ICPC Notebook – UIT.BTC

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
Vu Gia Bao, Nguyen Khanh Tai, Huynh Trung Cuong | Ngày 7 tháng 11 năm 2026

Mục lục

Tối ưu hằng số 1

1 Cấu trúc dữ liệu 1
1.1 Iterative Segment Tree . . . 1
1.2 Lazy Segment Tree (lưu ý cài đặt) . . . 1
1.3 Sparse Segment Tree . . . 1
1.4 Cartesian Tree . . . 2
1.5 Persistent Segment Tree . . . 2
1.6 Parallel Binary Search . . . 2
1.7 Wavelet Tree . . . 2
1.8 Segment Tree Beat . . . 2
1.9 Kd Tree . . . 3
1.10 BIT2D . . . 3
1.11 Treap . . . 3
1.12 Splay Tree / Link - cut Tree . . . 3
1.13 Trie . . . 4
1.14 Lichao Tree . . . 4
1.15 Line Container . . . 4
1.16 Arpa Trick . . . 4
1.17 Chia căn (Square Root Decomposition) . . . 4
1.18 Hash Map (SplitMix64) . . . 5
1.19 Erasable Priority Queue . . . 5
1.20 Cấu trúc mở rộng (Binary Doubling) . . . 5
1.21 Interval Set . . . 5

2 Đồ thị 5
2.1 Matroid: nhận diện và các loại . . . 5
2.2 Matroid intersection (maximum cardinality) . . . 5
2.3 Đếm thứ tự topo trên cây/rừng . . . 6
2.4 Minimum Cost Flow . . . 6
2.5 MinCut - MaxFlow . . . 6
2.6 Bipartite Matching . . . 6
2.7 Hungarian Matching . . . 7
2.8 General Matching . . . 7
2.9 Biconnected Component . . . 7
2.10 Functional Cycle . . . 8
2.11 Strongly Connected Components . . . 8
2.12 Xóa offline (DSU Rollback) . . . 8
2.13 2-SAT . . . 8
2.14 Floyd . . . 8
2.15 Eulerian Path . . . 8
2.16 Định lý Vizing . . . 9
2.17 Kiểm tra hai phía online . . . 9
2.18 Gomory-Hu Tree . . . 9
2.19 Critical (Dominator Tree) . . . 9
2.20 Prufer . . . 9
2.21 Steiner Tree . . . 10
2.22 Stable Marriage . . . 10

3 Cây 10
3.1 Tree Isomorphism . . . 10
3.2 Centroid Decomposition . . . 10
3.3 Heavy-Light Decomposition . . . 10

4 Chuỗi 10
4.1 Chiến lược giải bài chuỗi . . . 10
4.2 Rolling Hash (61-bit Mersenne) . . . 11
4.3 KMP . . . 11
4.4 Z-Algorithm . . . 11
4.5 Aho Corasick . . . 11
4.6 Suffix Array . . . 11
4.7 Suffix Automaton . . . 12
4.8 Lyndon Factorization . . . 12
4.9 Manacher . . . 12
4.10 Palindrome Tree . . . 12
4.11 De Bruijn (FKM) . . . 12

5 Quy hoạch động (DP) 12
5.1 Reroot DP . . . 12
5.2 Divide-and-Conquer DP và bất đẳng thức tứ giác . . . 13
5.3 Tối ưu DP 1D/1D . . . 13
5.4 Alien DP . . . 13
5.5 Convex Hull Trick . . . 13
5.6 Slope Trick . . . 13
5.7 Bitmask DP . . . 13
5.8 SOS DP . . . 13
5.9 Tối ưu bằng monotonic queue . . . 14
5.10 Expected Value Trick . . . 14

6 Toán học 14
6.1 Công thức số học . . . 14
6.2 Like-Euclid Floor Sum . . . 14
6.3 Rabin-Miller . . . 14
6.4 Sàng phân đoạn (Segmented Sieve) . . . 14
6.5 Đếm nguyên tố (Prime Counting) . . . 14
6.6 Pollard Rho . . . 15
6.7 Primitive Root . . . 15
6.8 Baby-step Giant-Step . . . 15
6.9 Căn modulo (Tonelli-Shanks) . . . 15
6.10 Extended GCD . . . 15
6.11 Diophantine . . . 15
6.12 Stern-Brocot Tree . . . 15
6.13 Hàm nhân tính và Möbius Inversion . . . 16
6.14 Công thức tổ hợp . . . 16
6.15 Hàm sinh (Generating Functions) và chuỗi Taylor . . . 16
6.16 Truy hồi tuyến tính (Berlekamp-Massey) . . . 16
6.17 FWHT . . . 16
6.18 Lagrange . . . 17
6.19 NTT (Number Theoretic Transform) . . . 17
6.20 FFT (Fast Fourier Transform) . . . 17
6.21 Công thức Formal Power Series (FPS) . . . 17
6.22 Đại số tuyến tính (Gauss) . . . 17
6.23 XOR Linear Basis . . . 18
6.24 Burnside Lemma & Pólya Theorem . . . 18
6.25 Game: Invariant, Symmetric, Nim, DP . . . 18
6.26 Xác suất và kỳ vọng . . . 18
6.27 Bignum . . . 18

7 Hình học 18
7.1 Primitive Geometry . . . 18
7.2 Convex Hull . . . 19
7.3 Rotating Calipers . . . 19
7.4 Minkowski Sum . . . 19
7.5 Point in Convex Polygon . . . 19
7.6 Line Hull Intersection . . . 19
7.7 Polygon Raycast . . . 19
7.8 Half-plane Intersection . . . 19
7.9 Smallest Enclosing Circle . . . 20
7.10 Circle Tangents . . . 20
7.11 Công thức hình học . . . 20
7.12 AREA . . . 20
7.13 Manhattan MST . . . 21
7.14 Planar Dual Graph . . . 21

8 Tiện ích và kỹ thuật 21
8.1 Template . . . 21
8.2 Debugging . . . 21
8.3 Script biên dịch và chạy . . . 21
8.4 Phân tích biểu thức (Expression Parsing) . . . 21
8.5 Lập lịch Johnson . . . 22
8.6 Fracturing Search (k-best) . . . 22
8.7 Monte Carlo và Las Vegas . . . 22
8.8 Lịch (Calendar) . . . 22
8.9 Josephus . . . 22
8.10 Kỹ thuật interactive . . . 22
8.11 Giao tiếp hai lần chạy (Run-Twice) . . . 23
8.12 Random, Bit Tricks, Bitset . . . 23
8.13 Biến đổi bài toán (Transformations) . . . 23
8.14 Phím tắt VS Code (Linux) . . . 24
8.15 Checklist trước khi nộp . . . 24

Tối ưu hằng số

```
#include <bits/stdc++.h>
#pragma GCC optimize("O3,unroll-loops")
// Chi bat target neu CPU judge ho tro.
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

Đặt sau include, trước các hàm. Chỉ bật `target` khi CPU judge hỗ trợ; đo lại vì `O3`/unroll có thể chậm hơn `O2`. `Ofast` nới quy tắc số thực.

Đo: xác định nghẽn tính toán, bộ nhớ hay I/O; cùng input/compiler, thay một yếu tố mỗi lần, đo vài lượt. Test lại với brute force; máy local và judge có thể khác tốc độ.

Cache/thứ tự duyệt: ưu tiên mảng liên tiếp; chỉ số cuối ở vòng trong. Nếu đổi chiều lưu trữ, đổi đồng bộ mọi truy cập; chỉ đổi thứ tự xử lý khi không vi phạm phụ thuộc dữ liệu.

Kiểu dữ liệu: dùng `int32_t` cho chỉ số/giá trị vừa 32-bit khi template có macro `int`; tổng/tích lớn dùng 64-bit hoặc `__int128`, ép kiểu trước phép nhân. Không dựa vào tràn số có dấu.

Cấp phát: tái sử dụng buffer; `reserve(n)` trước nhiều lần `push_back`. Reserve chỉ đặt capacity; cần `resize` trước khi truy cập phần tử bằng chỉ số. Nhiều node nhỏ: cân nhắc pool/mảng node thay cấp phát từng node.

Array vs vector: mảng C/`std::array<T,N>` có kích thước cố định lúc biên dịch; `vector<T>` có kích thước động. Cả ba lưu liên tiếp (trừ `vector<bool>`); truy cập `[]` đều $O(1)$, thường tương đương khi bật tối ưu. Vector tốn thêm cấp phát/quản lý capacity; `vector<int>(n)` khởi tạo n số 0, mảng số nguyên cục bộ không tự khởi tạo. Biết cận cố định, tái dùng nhiều: mảng; cần đúng kích thước input/thay đổi size: vector. Mảng lớn nên đặt global/static để tránh tràn stack; vector nhiều chiều lồng nhau không bảo đảm toàn khối liên tiếp, có thể dùng vector phẳng.

Template vector nhiều chiều: `Vec<D,T>`, D là số chiều, T là kiểu phần tử. Constructor nhận kích thước từng chiều, cuối cùng là giá trị khởi tạo (có thể bỏ, mặc định `T()`). Đây vẫn là vector lồng nhau, không phải một khối phẳng.

```
template<int D, typename T>
struct Vec : public vector<Vec<D - 1, T>> {
  static_assert(D >= 1, "Vector dimension must be greater than zero!");
  template<typename... Args>
  Vec(int n = 0, Args... args) : vector<Vec<D - 1, T>>(n, Vec<D - 1, T>(args...))
      {
  }
};
template<typename T>
struct Vec<1, T> : public vector<T> {
  Vec(int n = 0, const T& val = T()) : vector<T>(n, val) {
  }
};
//Usage: Vec<3, long long> a;
```

```
Vec<2, int> mat(rows, cols, 0);
Vec<3, int> cube(rows, cols, depth, -1);
cube[xx][yy][zz] = 5;
```

Container: ID nhỏ/nén được thì dùng mảng/vector; cần thứ tự mới dùng map/set. Với `unordered_map`, thử `reserve` và chỉnh load factor; không mặc định hash luôn nhanh hơn. Đồ thị lớn: cân nhắc mảng cạnh liên tiếp.

Hash table: GNU `gp_hash_table` là lựa chọn đáng đo thử khi nghẽn hash; không suy rộng tốc độ từ benchmark của người khác. Collision có thể làm tổng thời gian thành $O(n^2)$; `reserve` không ngăn hack. Cân nhắc hash trộn tốt với seed ngẫu nhiên; chỉ XOR/cộng một hằng ngẫu nhiên chưa đủ.

Tránh sao chép/gọi gián tiếp: truyền container bằng `const&`; duyệt phần tử lớn bằng `const auto&`. Hàm gọi rất nhiều lần: ưu tiên hàm thường/template/lambda cụ thể nếu không cần `std::function`.

Vòng lặp nóng: tính biểu thức bất biến/base một lần, tái dùng kết quả, tiền xử lý đại lượng dùng nhiều lần. Bỏ cập nhật đóng góp 0 nếu không có tác dụng phụ; dừng sớm khi đã đủ kết luận. Không tự unroll/branchless nếu chưa đo.

Số học: tránh `pow` cho bình phương số nguyên; dùng nhân với kiểu đủ rộng. Với $0 \le a, b < mod$, cộng modulo bằng cộng rồi trừ mod nếu cần; tổng trung gian phải không tràn. Giảm số lần chia/modulo trong vòng nóng nếu có công thức tương đương đúng.

Bit song song: giao hai tập bằng `(maskA & maskB).count()`; subset sum boolean với trọng số không âm: `can[0]=1`, mỗi phần tử `can |= can << weight`. Cấp đủ số bit cho miền tổng cần xét; mỗi phép trên bitset vẫn quét nhiều word, không phải $O(1)$.

Duyệt bit 1: dùng `__builtin_popcountll` cho mask 64-bit. Mask không âm: `while(mask){ bit=__builtin_ctzll(mask); mask&=mask-1; }`; không gọi `ctz/clz` với 0. GNU bitset có `_Find_first()`, `_Find_next(pos)`; hết bit 1 trả `size()`. Đây là extension, vẫn có thể quét các word trống.

Reset: chỉ xóa vị trí đã chạm hoặc dùng timestamp nếu mỗi test chỉ dùng ít phần tử trên miền lớn; xử lý giá trị mặc định và tràn timestamp. Có thể tái sử dụng vùng nhớ giữa các pha.

I/O: `ios::sync_with_stdio(false); cin.tie(nullptr);`, dùng `'\n'` thay `endl`. Dữ liệu lớn: thử `fread/fwrite`, xuất theo khối; tránh trộn C/C++ I/O sau khi tắt sync. Interactive vẫn phải flush khi gửi câu hỏi.

1 Cấu trúc dữ liệu

Quy ước C++17: Chỉ số từ 0, đoạn $[l, r)$. File nguồn có sẵn header chuẩn và namespace; phần bọc lặp lại được ẩn khi in.

1.1 Iterative Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/cập nhật, $\mathcal{O}(n)$ dựng

Segment tree trên monoid có thứ tự: gán điểm, gộp đoạn, tìm biên; bản dual cộng đoạn/truy vấn điểm

```
// Use: int mergeSum(int a,int b) { return a+b; }
//      SegmentTree st({1,2,3},0,mergeSum); auto s = st.fold(0,3);
//      RangeAddPointQuery lazy(3); lazy.add(0,2,4);
// Associative merge with identity; order is preserved (noncommutative supported
    ).
struct SegmentTree {
  int n, base = 1; int identity; function<int(int,int)> merge; vector<int> tree;
  SegmentTree(const vector<int>&a,int identity,function<int(int,int)> merge):n(a
     .size()),identity(identity),merge(merge){
    while (base < n) base *= 2;
    tree.assign(2*base,identity); copy(a.begin(),a.end(),tree.begin()+base);
    for(int u=base-1; u; --u)tree[u]=merge(tree[u*2],tree[u*2+1]);
  }
  void set(int p, int value) {
    assert(0 <= p && p < n); tree[p += base] = value;
    while (p /= 2) tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
  }
  int fold(int l, int r) const {
    assert(0<=l&&l<=r&&r<=n); int left=identity,right=identity;
    for (l += base, r += base; l < r; l /= 2, r /= 2) {
      if (l & 1) left = merge(left, tree[l++]);
      if (r & 1) right = merge(tree[--r], right);
    }
    return merge(left, right);
  }
  // First failing position r for predicate(fold(l,r+1)); n if none.
  // predicate(identity) true, predicate monotone under appending elements.
  int max_right(int l,function<bool(int)> predicate)const {
    assert(0 <= l && l <= n && predicate(identity));
    if (l == n) return n;
    int value = identity; int u = l + base;
    do {
      while (!(u & 1)) u /= 2;
      int candidate = merge(value, tree[u]);
      if (!predicate(candidate)) {
        while (u < base) {
          u *= 2; candidate = merge(value, tree[u]);
          if (predicate(candidate)) { value = candidate; ++u; }
        }
        return min(n, u - base);
      }
      value = candidate; ++u;
    }
    while ((u & -u) != u);
    return n;
  }
};
struct RangeAddPointQuery {
  int n; vector<int> tree;
  RangeAddPointQuery(int n) : n(n), tree(2 * max<int>(1, n)){}
  void add(int l, int r, int value) {
    assert(0 <= l && l <= r && r <= n);
    for (l += n, r += n; l < r; l /= 2, r /= 2) {
      if (l & 1) tree[l++] += value;
      if (r & 1) tree[--r] += value;
    }
  }
  int get(int p) const {
    assert(0 <= p && p < n); int value = 0;
    for (p += n; p; p /= 2) value += tree[p];
    return value;
  }
};
```

1.2 Lazy Segment Tree (lưu ý cài đặt)

Quy tắc lazy propagation, hợp thành tag (Affine, Range Set) và Tree Walk $O(\log n)$

- **Quy tắc push/pull**: Gọi `push(id, l, r)` đầu `update/query`; gọi `pull(id)` (`st[id] = merge(st[2*id], st[2*id+1])`) sau khi cập nhật các con.
- **Nhiều lazy tag**:
  - **Affine** ($x \mapsto a \cdot x + b$): Áp dụng tag mới (c, d) sau $(a, b) \implies c(ax + b) + d = (ca)x + (cb + d)$.
    Tag: `laz_a = laz_a * c; laz_b = laz_b * c + d;`
    Node: `val = val * c + (r - l + 1) * d;`
  - **Range Set và Range Add**: Range Set ($x \leftarrow v$) ghi đè Range Add trước đó $\implies$ đặt lại `add = 0`.
- **Tree Walk** ($O(\log n)$): Nếu con trái thỏa (max $\ge X$ hoặc tổng $\ge S$), đẩy lazy và đi trái; nếu không, điều chỉnh mục tiêu rồi đi phải.
- **Giới hạn**: Cấp phát $4N$ khi đánh số từ 1 (`id << 1` và `id << 1 | 1`).

1.3 Sparse Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn

Segment tree động, chỉ cấp phát node khi cần

```
// Use: SparseRangeSum st(0,1000000000LL); st.add(42,5);
// Sparse point-add / range-sum on integer domain [lo,hi); O(log(domain))
    operations.
struct SparseRangeSum {
  struct Node { int left=-1,right=-1; int sum=0; }; vector<Node>t {1};
  int lo, hi;
  int child(int u, bool right) {
    int v = right ? t[u].right : t[u].left;
    if (v < 0) {
      v = t.size(); t.emplace_back();
      if (right) t[u].right = v;
```

```
      else t[u].left = v;
    }
    return v;
  }
  void add(int u, int l, int r, int p, int x) {
    t[u].sum += x;
    if (r - l == 1) return;
    int m = l + (r - l) / 2;
    if (p < m) add(child(u, false), l, m, p, x);
    else add(child(u, true), m, r, p, x);
  }
  int query(int u,int l,int r,int a,int b)const {
    if (u < 0 || b <= l || r <= a) return 0;
    if (a <= l && r <= b) return t[u].sum;
    int m=l+(r-l)/2; return query(t[u].left,l,m,a,b)+query(t[u].right,m,r,a,b);
  }
  SparseRangeSum(int lo, int hi) : lo(lo), hi(hi) {
    assert(lo < hi && (__int128) hi - lo <= LLONG_MAX);
  }
  void add(int p,int x){ assert(lo<=p&&p<hi); add(0,lo,hi,p,x); }
  int sum(int l, int r) const {
    assert(lo <= l && l <= r && r <= hi); return query(0, lo, hi, l, r);
  }
  size_t node_count() const { return t.size(); }
};
```

1.4 Cartesian Tree

Độ phức tạp: $\mathcal{O}(n)$ dựng bằng monotonic stack

Cây nhị phân có thứ tự heap theo giá trị và thứ tự BST theo chỉ số

- **Tính chất**: Min-heap theo giá trị ($val[\text{parent}] \le val[\text{child}]$) + duyệt in-order cho chỉ số mảng gốc ($1..n$). Gốc là giá trị nhỏ nhất. Cây con u ứng với đoạn cực đại $[L_u, R_u]$ mà $A[u]$ là nhỏ nhất ($size(u) = R_u - L_u + 1$).
- $\mathcal{O}(n)$ **Dựng bằng stack**: Giữ nhánh phải trong stack. Với $i = 1..n$: pop u có $A[u] > A[i]$; node pop cuối thành $left[i] = u$; đỉnh còn lại có $right[\text{top}] = i$. Push i.
- **Ứng dụng chính**:
  - **RMQ $\iff$ LCA**: Giá trị nhỏ nhất trên đoạn $\min_{l \le k \le r} A[k] = A[\text{LCA}(l, r)]$.
  - **Hình chữ nhật lớn nhất trong histogram**: $\max_u(A[u] \cdot size(u))$.
  - **D&C → Tree DP**: Chuyển D&C trên mảng, chia tại phần tử nhỏ nhất, thành DP trên cây con.

1.5 Persistent Segment Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/cập nhật

Persistent segment tree: cập nhật điểm, truy vấn các phiên bản cũ

```
// Use: PersistentRangeSum pst(3); int root = pst.add(0,1,5);
//      auto s = pst.sum(root,0,3); // root 0: all zeros
// Immutable point-add versions; root 0 is the all-zero array. Ranges [l,r).
struct PersistentRangeSum {
  struct Node { int left=0,right=0; int sum=0; }; vector<Node>t {1}; int n;
  int update(int root, int l, int r, int p, int x) {
    int u = t.size(); t.push_back(t[root]); t[u].sum += x;
    if (r - l > 1) {
      int m = (l + r) / 2;
      if (p < m) {
        int v = update(t[u].left, l, m, p, x); t[u].left = v;
      } else { int v = update(t[u].right, m, r, p, x); t[u].right = v; }
    }
    return u;
  }
  int query(int u, int l, int r, int a, int b) const {
    if (!u || b <= l || r <= a) return 0;
    if (a <= l && r <= b) return t[u].sum;
    int m=(l+r)/2; return query(t[u].left,l,m,a,b)+query(t[u].right,m,r,a,b);
  }
  PersistentRangeSum(int n) : n(n) { assert(n >= 0); }
  int add(int root, int p, int x) {
    assert(0<=root&&root<(int)t.size()&&0<=p&&p<n); return update(root,0,n,p,x);
  }
  int sum(int root, int l, int r) const {
    assert(0<=root&&root<(int)t.size()&&0<=l&&l<=r&&r<=n); return query(root,0,n
      ,l,r);
  }
};
```

1.6 Parallel Binary Search

Độ phức tạp: $\mathcal{O}((N + Q)\log(\text{range})\log N)$

Binary search song song offline cho nhiều truy vấn đơn điệu

```
// Use: auto ans = parallel_binary_search(3,2,reset,apply,test);
//      reset(): void; apply(eventID): void; test(queryID): bool.
// Monotone test(query) after an event prefix. Returns first prefix length 0..M,
// or M+1 if never true. reset() restores prefix 0; apply(i) applies event i.
vector<int> parallel_binary_search(int M,int Q,function<void()> reset,
    function<void(int)> apply,function<bool(int)> test){
  assert(M >= 0 && Q >= 0); vector<int> lo(Q, 0), hi(Q, M + 1);
  for (;;) {
    vector<vector<int>> bucket(M + 1); bool active = false;
    for (int q = 0; q < Q; ++q) if (lo[q] < hi[q]) {
      active = true; bucket[(lo[q] + hi[q]) / 2].push_back(q);
    }
    if (!active) return lo;
    reset();
    for (int prefix = 0; prefix <= M; ++prefix) {
      if (prefix) apply(prefix - 1);
      for (int q : bucket[prefix]) if (test(q)) hi[q] = prefix;
      else lo[q] = prefix + 1;
    }
  }
}
```

1.7 Wavelet Tree

Độ phức tạp: $\mathcal{O}(\log(\text{alphabet}))$ mỗi truy vấn

Wavelet tree: phần tử thứ k, tần suất và đếm giá trị trong $[L, R]$ trên đoạn

```
// Use: WaveletTree wt({3,1,2}); auto x = wt.kth(0,3,0);
// Immutable array; ranges [l,r), k is zero-based. O(log sigma) queries.
struct WaveletTree {
  struct Node { int lo,hi,left=-1,right=-1; vector<int>pref; }; vector<Node>
    nodes;
  vector<int> values; int n;
  int build(vector<int> a, int lo, int hi) {
    int id = nodes.size(); nodes.push_back( { lo, hi, -1, -1, {0} });
    if (hi - lo <= 1 || a.empty()) return id;
    int mid = (lo + hi) / 2; vector<int> left, right;
    for (int x : a) {
      nodes[id].pref.push_back(nodes[id].pref.back() + (x < mid));
      (x < mid ? left : right).push_back(x);
    }
    int l=build(move(left),lo,mid),r=build(move(right),mid,hi); nodes[id].left=l
    ;
    nodes[id].right = r; return id;
  }
  int less(int id, int l, int r, int rank) const {
    const auto& t = nodes[id];
    if (l == r || rank <= t.lo) return 0;
    if (t.hi <= rank) return r - l;
    int a = t.pref[l], b = t.pref[r];
    return less(t.left, a, b, rank) + less(t.right, l - a, r - b, rank);
  }
  WaveletTree(const vector<int>&a):values(a),n(a.size()){
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(),values.end()),values.end()); vector<int>
     ranks;
    for(auto x:a)ranks.push_back(lower_bound(values.begin(),values.end(),x)-
     values.begin());
    build(move(ranks), 0, max<int>(1, static_cast<int>(values.size())));
  }
  int kth(int l, int r, int k) const {
    assert(0<=l&&l<=r&&r<=n&&0<=k&&k<r-l); int id=0;
    while (nodes[id].hi - nodes[id].lo > 1) {
      const auto& t = nodes[id]; int a = t.pref[l], b = t.pref[r];
      if(k<b-a){ l=a; r=b; id=t.left; } else { k-=b-a; l-=a; r-=b; id=t.right; }
    }
    return values[nodes[id].lo];
  }
  int count(int l, int r, int low, int high) const {
    assert(0 <= l && l <= r && r <= n);
    if (low > high) return 0;
    int a=lower_bound(values.begin(),values.end(),low)-values.begin();
    int b=upper_bound(values.begin(),values.end(),high)-values.begin();
    return less(0, l, r, b) - less(0, l, r, a);
  }
};
```

1.8 Segment Tree Beat

Độ phức tạp: $\mathcal{O}((n + q)\log^2 n)$ khấu hao

Segment Tree Beats: chmin/chmax, cộng và truy vấn tổng trên đoạn

```
// Use: SegmentTreeBeats st({1,2,3}); st.chmin(0,3,2);
// [l,r); values, shifts and sums must remain well inside +/- INF.
// chmin/chmax/sum: amortized logarithmic; with add: log^2 bound.
struct SegmentTreeBeats {
  static constexpr int INF = 1LL << 60;
  struct Node {
    int sum=0,mx=-INF,mx2=-INF,mn=INF,mn2=INF,add=0; int cmx=0,cmn=0,len=0;
  }; int n; vector<Node> t;
  void pull(int p) {
    const auto&a=t[p*2],&b=t[p*2+1]; auto&c=t[p]; c.sum=a.sum+b.sum;
    c.len=a.len+b.len; c.mx=max(a.mx,b.mx); c.mn=min(a.mn,b.mn);
    c.cmx = (a.mx == c.mx ? a.cmx : 0) + (b.mx == c.mx ? b.cmx : 0);
    c.cmn = (a.mn == c.mn ? a.cmn : 0) + (b.mn == c.mn ? b.cmn : 0);
    c.mx2=max(a.mx==c.mx?a.mx2:a.mx,b.mx==c.mx?b.mx2:b.mx);
    c.mn2=min(a.mn==c.mn?a.mn2:a.mn,b.mn==c.mn?b.mn2:b.mn);
  }
  void add_node(int p, int x) {
    auto&a=t[p]; a.sum+=x*a.len; a.mx+=x; a.mn+=x; a.add+=x;
    if (a.mx2 != -INF) a.mx2 += x;
    if (a.mn2 != INF) a.mn2 += x;
  }
  void cap(int p, int x) {
    auto& a = t[p];
    if (x >= a.mx) return;
    a.sum += (x - a.mx) * a.cmx;
    if (a.mn == a.mx) a.mn = x;
    else if (a.mn2 == a.mx) a.mn2 = x;
    a.mx = x;
  }
  void floor_node(int p, int x) {
    auto& a = t[p];
    if (x <= a.mn) return;
    a.sum += (x - a.mn) * a.cmn;
    if (a.mx == a.mn) a.mx = x;
    else if (a.mx2 == a.mn) a.mx2 = x;
    a.mn = x;
  }
  void push(int p) {
    int children[] = {p * 2, p * 2 + 1};
    for (int c : children) {
      if (t[p].add) add_node(c, t[p].add);
      cap(c, t[p].mx); floor_node(c, t[p].mn);
    }
    t[p].add = 0;
  }
  void build(int p, int l, int r, const vector<int>& a) {
    if(r-l==1){ t[p].sum=t[p].mx=t[p].mn=a[l]; t[p].cmx=t[p].cmn=t[p].len=1;
     return; }
    int m=(l+r)/2; build(p*2,l,m,a); build(p*2+1,m,r,a); pull(p);
  }
  void update(int p,int l,int r,int ql,int qr,int x,int kind){
    if(qr<=l||r<=ql||(kind==0&&t[p].mx<=x)||(kind==1&&t[p].mn>=x))return;
    if (ql <= l && r <= qr) {
      if (kind == 2) { add_node(p, x); return; }
      if (kind == 0 && t[p].mx2 < x) { cap(p, x); return; }
      if (kind == 1 && x < t[p].mn2) { floor_node(p, x); return; }
    }
    push(p); int m = (l + r) / 2; update(p * 2, l, m, ql, qr, x, kind);
    update(p * 2 + 1, m, r, ql, qr, x, kind); pull(p);
  }
  int query(int p, int l, int r, int ql, int qr) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return t[p].sum;
    push(p); int m=(l+r)/2; return query(p*2,l,m,ql,qr)+query(p*2+1,m,r,ql,qr);
```

```
    }
    SegmentTreeBeats(const vector<int>&a):n(a.size()),t(4*max<int>(1,n)){
      if (n) build(1, 0, n, a);
    }
    void chmin(int l, int r, int x) {
      if (l < r) update(1, 0, n, l, r, x, 0);
    }
    void chmax(int l, int r, int x) {
      if (l < r) update(1, 0, n, l, r, x, 1);
    }
    void add(int l, int r, int x) {
      if (l < r) update(1, 0, n, l, r, x, 2);
    }
    int sum(int l,int r){ return l<r?query(1,0,n,l,r):0; }
  };
```

1.9 Kd Tree

Độ phức tạp: $O(n)$ mỗi truy vấn trong trường hợp xấu nhất; cắt tỉa không gian trên cây cân bằng

Cây k chiều tĩnh: điểm gần nhất và truy vấn hộp đóng song song trục

```
  // Use: KDTree kd({{0.,0.},{1.,2.}}); auto p = kd.nearest({0.,1.});
  // Balanced static k-d tree. Worst-case nearest/range query O(n).
  struct KDTree {
    static const int D = 2; // Change D to the number of dimensions.
    using Point = array<double, D>;
    struct Node { int id,left=-1,right=-1; Point low,high; }; vector<Point>points;
    vector<Node> t; int root;
    int build(vector<int>& ids, int l, int r, int depth) {
      if (l == r) return-1;
      int m = (l + r) / 2, axis = depth % D;
      struct AxisOrder {
        const vector<Point>& points; int axis;
        bool operator()(int a,int b) const { return points[a][axis]<points[b][axis
      ]; }
      };
      nth_element(ids.begin()+l,ids.begin()+m,ids.begin()+r,AxisOrder{points,axis
      }); int u=t.size(); t.push_back( {ids[m],-1,-1,points[ids[m]],points[ids[m
      ]]});
      int a=build(ids,l,m,depth+1),b=build(ids,m+1,r,depth+1); t[u].left=a; t[u].
      right=b;
      int children[] = {a, b};
      for (int v : children) if (v >= 0) for (size_t d = 0; d < D; ++d) {
        t[u].low[d] = min(t[u].low[d], t[v].low[d]);
        t[u].high[d] = max(t[u].high[d], t[v].high[d]);
      }
      return u;
    }
    double lower_bound(int u, const Point& q) const {
      if (u < 0) return numeric_limits<double>::infinity();
      double ans = 0;
      for (size_t d = 0; d < D; ++d) {
        double z=max( {0.,t[u].low[d]-q[d],q[d]-t[u].high[d]}); ans+=z*z;
      }
      return ans;
    }
    KDTree(vector<Point> p) : points(move(p)) {
      vector<int> ids(points.size()); iota(ids.begin(), ids.end(), 0);
      root = build(ids, 0, ids.size(), 0);
    }
    void nearest_visit(int u,const Point& q,int excluded,
        int& best,double& distance) const {
        if (u < 0 || lower_bound(u, q) > distance) return;
        int id = t[u].id;
        if (id != excluded) {
          double v = 0;
          for(size_t d=0;d<D;++d)v+=(points[id][d]-q[d])*(points[id][d]-q[d]);
          if(v<distance||(v==distance&&(best<0||id<best)))distance=v,best=id;
        }
        int a = t[u].left, b = t[u].right;
        if (lower_bound(a, q) > lower_bound(b, q)) swap(a, b);
        nearest_visit(a,q,excluded,best,distance); nearest_visit(b,q,excluded,best
      ,distance);

    }
    optional<pair<int,double>>nearest(const Point&q,int excluded=-1)const {
      int best = -1; double distance = numeric_limits<double>::infinity();
      nearest_visit(root,q,excluded,best,distance);
      if (best < 0) return nullopt;
      return pair {best, distance};
    }
    void range_visit(int u,const Point& low,const Point& high,
        vector<int>& out) const {
        if (u < 0) return;
        for(size_t d=0;d<D;++d)if(t[u].high[d]<low[d]||high[d]<t[u].low[d])return;
        bool inside = true;
        for(size_t d=0;d<D;++d)inside&=low[d]<=points[t[u].id][d]&&points[t[u].id
      ][d]<=high[d];
        if (inside) out.push_back(t[u].id);
        range_visit(t[u].left,low,high,out); range_visit(t[u].right,low,high,out);

    }
    vector<int> range(const Point& low, const Point& high) const {
      vector<int> out;
      range_visit(root,low,high,out); return out;
    }
  };
```

1.10 BIT2D

Độ phức tạp: $\mathcal{O}(\log^2 n)$ mỗi truy vấn

Fenwick 2D: tổng hình chữ nhật và cập nhật điểm

```
  // Use: Fenwick2D fw(3,4); fw.add(0,1,5); auto s = fw.sum(0,0,3,4);
  struct Fenwick2D {
    int n, m; vector<vector<int>> t;
    Fenwick2D(int n,int m):n(n),m(m),t(n+1,vector<int>(m+1)){
      assert(n >= 0 && m >= 0);
    }
    void add(int x, int y, int value) {
      assert(0 <= x && x < n && 0 <= y && y < m);
      for(int i=x+1;i<=n;i+=i&-i)for(int j=y+1;j<=m;j+=j&-j)t[i][j]+=value;
    }
    int prefix(int x, int y) const {
      assert(0 <= x && x <= n && 0 <= y && y <= m); int ans = 0;
      for(int i=x; i; i-=i&-i)for(int j=y; j; j-=j&-j)ans+=t[i][j];
      return ans;
    }
    int sum(int x1, int y1, int x2, int y2) const {
      assert(x1 <= x2 && y1 <= y2);
      return prefix(x2,y2)-prefix(x1,y2)-prefix(x2,y1)+prefix(x1,y1);
    }
  };
```

1.11 Treap

Độ phức tạp: $\mathcal{O}(\log n)$ kỳ vọng mỗi thao tác

BST ngẫu nhiên: split/merge mảng và thao tác theo khóa ẩn

```
  // Use: ImplicitTreap tr(123456); tr.insert(0,7); tr.reverse(0,1);
  // Implicit sequence with insert/erase/reverse/range-sum. RNG seed is explicit.
  // Erased nodes remain in the arena; storage O(total insertions), no dangling
      pointers.
  struct ImplicitTreap {
    struct Node {
      int value,sum; int priority; int left=-1,right=-1,size=1; bool rev=false;
    }; vector<Node> t; int root = -1; mt19937_64 rng;
    int size(int u) const { return u < 0 ? 0 : t[u].size; }
    int sum(int u) const { return u < 0 ? 0 : t[u].sum; }
    void flip(int u){ if(u>=0){ swap(t[u].left,t[u].right); t[u].rev^=1; } }
    void push(int u) {
      if(u>=0&&t[u].rev){ flip(t[u].left); flip(t[u].right); t[u].rev=false; }
    }
    void pull(int u) {
      t[u].size = 1 + size(t[u].left) + size(t[u].right);
      t[u].sum = t[u].value + sum(t[u].left) + sum(t[u].right);
    }
    pair<int, int> split(int u, int k) {
      if (u < 0) return {-1, -1};
      push(u);
      if (size(t[u].left) >= k) {
        auto[a,b]=split(t[u].left,k); t[u].left=b; pull(u); return {a,u};
      }
      auto[a,b]=split(t[u].right,k-size(t[u].left)-1); t[u].right=a; pull(u);
      return {u, b};
    }
    int merge(int a, int b) {
      if (a < 0) return b;
      if (b < 0) return a;
      if (t[a].priority > t[b].priority) {
        push(a); t[a].right = merge(t[a].right, b); pull(a); return a;
      }
      push(b); t[b].left = merge(a, t[b].left); pull(b); return b;
    }
    ImplicitTreap(int seed) : rng(seed) {}
    int size() const { return size(root); }
    void insert(int p, int x) {
      assert(0<=p&&p<=size()); int u=t.size(); t.push_back( {x,x,static_cast<int>(
      rng() >> 1)});
      auto[a, b] = split(root, p); root = merge(merge(a, u), b);
    }
    void erase(int l, int r) {
      assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,discard]=
      split(a,l);
      (void) discard; root = merge(b, c);
    }
    void reverse(int l, int r) {
      assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,m]=split(a,l)
      ;
      flip(m); root = merge(merge(b, m), c);
    }
    int sum(int l, int r) {
      assert(0<=l&&l<=r&&r<=size()); auto[a,c]=split(root,r); auto[b,m]=split(a,l)
      ;
      int value=sum(m); root=merge(merge(b,m),c); return value;
    }
  };
```

1.12 Splay Tree / Link - cut Tree

Độ phức tạp: $\mathcal{O}(\log n)$ khấu hao mỗi thao tác

Link-Cut Tree: truy vấn đường đi động, link, cut và đổi gốc trên rừng

```
  // Use: LinkCutTree lct({1,2,3}); lct.link(0,1);
  // Dynamic forest, vertex values, path sum. 0-based IDs; -1 is no node.
  struct LinkCutTree {
    struct Node {
      int child[2] {-1,-1},parent=-1; bool rev=false; int value=0,sum=0;
    }; vector<Node> t;
    bool is_root(int u) const {
      int p=t[u].parent; return p<0||(t[p].child[0]!=u&&t[p].child[1]!=u);
    }
    int sum(int u) const { return u < 0 ? 0 : t[u].sum; }
    void pull(int u){ t[u].sum=t[u].value+sum(t[u].child[0])+sum(t[u].child[1]); }
    void flip(int u){ if(u>=0){ swap(t[u].child[0],t[u].child[1]); t[u].rev^=1; }
      }
    void push(int u) {
      if(t[u].rev){ flip(t[u].child[0]); flip(t[u].child[1]); t[u].rev=false; }
    }
    void rotate(int u) {
      int p=t[u].parent,g=t[p].parent,d=t[p].child[1]==u,v=t[u].child[d^1];
      if (!is_root(p)) t[g].child[t[g].child[1] == p] = u;
      t[u].parent=g; t[u].child[d^1]=p; t[p].parent=u; t[p].child[d]=v;
      if (v >= 0) t[v].parent = p;
      pull(p); pull(u);
    }
    void splay(int u) {
      vector<int> path {u};
      for (int v = u; !is_root(v);) path.push_back(v = t[v].parent);
      for (auto it = path.rbegin(); it != path.rend(); ++it) push(*it);
      while (!is_root(u)) {
        int p = t[u].parent, g = t[p].parent;
        if(!is_root(p))rotate((t[p].child[1]==u)==(t[g].child[1]==p)?p:u);
        rotate(u);
      }
    }
    int access(int u) {
      int last = -1;
      for (int v = u; v >= 0; v = t[v].parent) {
        splay(v); t[v].child[1] = last;
        if (last >= 0) t[last].parent = v;
        pull(v); last = v;
      }
      splay(u); return last;
    }
    int root(int u) {
      access(u);
      while (push(u), t[u].child[0] >= 0) u = t[u].child[0];
      splay(u); return u;
```

```
    }
    LinkCutTree(const vector<int>&values):t(values.size()){
      for(int i=0; i<(int)t.size(); ++i)t[i].value=t[i].sum=values[i];
    }
    void make_root(int u) { access(u); flip(u); }
    bool connected(int u, int v) { return u == v || root(u) == root(v); }
    bool link(int u, int v) {
      make_root(u);
      if (root(v) == u) return false;
      t[u].parent = v; return true;
    }
    bool cut(int u, int v) {
      make_root(u); access(v);
      if (t[v].child[0] != u || t[u].child[1] >= 0) return false;
      t[v].child[0] = -1; t[u].parent = -1; pull(v); return true;
    }
    void set(int u,int value){ access(u); t[u].value=value; pull(u); }
    optional<int> path_sum(int u, int v) {
      if (!connected(u, v)) return nullopt;
      make_root(u); access(v); return t[v].sum;
    }
    optional<int> lca(int rooted_at, int u, int v) {
      if (!connected(rooted_at, u) || !connected(u, v)) return nullopt;
      make_root(rooted_at); access(u); return access(v);
    }
  };
```

1.13 Trie

Độ phức tạp: $\mathcal{O}$(length) mỗi truy vấn

Trie nhị phân multiset 64-bit: thêm, xóa, đếm, XOR lớn nhất

```
  // Use: BinaryTrie tr; tr.insert(7); auto best = tr.max_xor(3);
  // Multiset of nonnegative int keys; duplicates and erasure supported.
  struct BinaryTrie {
    static constexpr int BITS = numeric_limits<int>::digits;
    struct Node { int child[2] {-1,-1}; int count=0; }; vector<Node>t {1};
    int size() const { return t[0].count; }
    int count(int x) const {
      assert(x >= 0);
      int u = 0;
      for (int b = BITS - 1; b >= 0; --b) {
        u = t[u].child[x >> b & 1];
        if (u < 0) return 0;
      }
      return t[u].count;
    }
    void insert(int x) {
      assert(x >= 0);
      int u = 0; ++t[u].count;
      for (int b = BITS - 1; b >= 0; --b) {
        int c = x >> b & 1, v = t[u].child[c];
        if (v < 0) { v = t.size(); t.emplace_back(); t[u].child[c] = v; }
        u = v; ++t[u].count;
      }
    }
    bool erase(int x) {
      if (!count(x)) return false;
      int u = 0; --t[u].count;
      for(int b=BITS-1; b>=0; --b){ u=t[u].child[x>>b&1]; --t[u].count; }
      return true;
    }
    optional<int> max_xor(int x) const {
      assert(x >= 0);
      if (!size()) return nullopt;
      int answer = 0; int u = 0;
      for (int b = BITS - 1; b >= 0; --b) {
        int c = x >> b & 1, v = t[u].child[c ^ 1];
        if(v>=0&&t[v].count){ answer|=static_cast<int>(1)<<b; u=v; }else u=t[u].
      child[c];
      }
      return answer;
    }
  };
```

1.14 Lichao Tree

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi truy vấn/thêm

Li Chao Tree: thêm đường thẳng online, truy vấn min/max

```
  // Use: LiChao lc(-100,101); lc.add(2,3); auto y = lc.minimum(5);
  // Integer-domain minimum envelope [lo,hi); query nullopt before first insertion
     .
  struct LiChao {
    struct Line {
      int m, b;
      __int128 at(int x) const { return(__int128) m * x + b; }
    }; struct Node { Line line; int left=-1,right=-1; }; vector<Node>t; int root
      =-1;
    int lo, hi;
    int add(int u, int l, int r, Line line) {
      if (u < 0) { u = t.size(); t.push_back( {line}); return u; }
      int mid = l + (r - l) / 2;
      bool left=line.at(l)<t[u].line.at(l),middle=line.at(mid)<t[u].line.at(mid);
      if (middle) swap(line, t[u].line);
      if (r - l == 1) return u;
      if (left != middle) {
        int v = add(t[u].left, l, mid, line); t[u].left = v;
      } else { int v = add(t[u].right, mid, r, line); t[u].right = v; }
      return u;
    }
    optional<__int128>query(int u,int l,int r,int x)const {
      if (u < 0) return nullopt;
      __int128 ans = t[u].line.at(x);
      if (r - l > 1) {
        int m = l + (r - l) / 2;
        auto child=x<m?query(t[u].left,l,m,x):query(t[u].right,m,r,x);
        if (child) ans = min(ans, *child);
      }
      return ans;
    }
    LiChao(int lo, int hi) : lo(lo), hi(hi) {
      assert(lo < hi && (__int128) hi - lo <= LLONG_MAX);
    }
    void add(int m,int b){ root=add(root,lo,hi,{m,b}); }
    optional<__int128> minimum(int x) const {
      assert(lo <= x && x < hi); return query(root, lo, hi, x);
    }
  };
```

1.15 Line Container

Độ phức tạp: $\mathcal{O}(\log n)$ khấu hao mỗi thao tác

Convex hull của các đường thẳng động, hệ số góc và điểm truy vấn x tùy ý

```
  // Use: LineContainer lc; lc.add(2,3); auto y = lc.maximum(5);
  // Arbitrary slopes and x, maximum queries. Integer intersections use int128.
  struct LineContainer {
    struct Line {
      int m, b; mutable __int128 end;
      bool operator<(const Line& other) const { return m < other.m; }
      bool operator<(__int128 x) const { return end < x; }
    }; multiset<Line,less<>>h; static constexpr __int128 INF=(__int128)1<<126;
    static __int128 floor_div(__int128 a, __int128 b) {
      __int128 q = a / b;
      if (a % b && ((a < 0) != (b < 0))) --q;
      return q;
    }
    bool intersect(multiset<Line,less<>>::iterator x,multiset<Line,less<>>::
      iterator y){
      if (y == h.end()) { x->end = INF; return false; }
      if (x->m == y->m) x->end = x->b > y->b ? INF : -INF;
      else x->end=floor_div((__int128)y->b-x->b,(__int128)x->m-y->m);
      return x->end >= y->end;
    }
    void add(int m, int b) {
      auto z = h.insert( {m, b, 0}), y = z++, x = y;
      while (intersect(y, z)) z = h.erase(z);
      if(x!=h.begin()&&intersect(--x,y))intersect(x,y=h.erase(y));
      while((y=x)!=h.begin()&&(--x)->end>=y->end)intersect(x,h.erase(y));
    }
    optional<__int128> maximum(int x) const {
      if (h.empty()) return nullopt;
      auto line=h.lower_bound((__int128)x); return(__int128)line->m*x+line->b;
    }
  };
```

1.16 Arpa Trick

Độ phức tạp: $\mathcal{O}(n\alpha(n))$

Truy vấn min/max đoạn offline bằng DSU

```
  // Use: auto ids = range_max_indices({3,1},{{0,2}});
  // Offline maximum indices; queries [l,r) nonempty; ties choose rightmost.
  vector<int>range_max_indices(const vector<int>&a,const vector<pair<int,int>>&
      queries){
    int n = a.size(); vector<vector<pair<int, int>>> ending(n);
    for (int i = 0; i < static_cast<int>(queries.size()); ++i) {
      auto [l,r]=queries[i]; assert(0<=l&&l<r&&r<=n); ending[r-1].push_back( {l,i
      });
    }
    vector<int> parent(n), stack, answer(queries.size());
    iota(parent.begin(), parent.end(), 0);
    struct DSUFind {
      vector<int>& parent;
      int find(int u) {
      int v = u;
      while (parent[v] != v) v = parent[v];
      while(parent[u]!=u){ int next=parent[u]; parent[u]=v; u=next; }
      return v;
    }
    } dsu{parent};
    for (int r = 0; r < n; ++r) {
      while (!stack.empty() && a[stack.back()] <= a[r]) {
        parent[stack.back()] = r; stack.pop_back();
      }
      stack.push_back(r);
      for (auto [l, id] : ending[r]) answer[id] = dsu.find(l);
    }
    return answer;
  }
```

1.17 Chia căn (Square Root Decomposition)

Độ phức tạp: $\mathcal{O}(n\sqrt{n})$

Chia căn: mảng chia block và thuật toán Mo theo thứ tự Hilbert

```
  // Use: BlockArray a({1,2,3});
  struct BlockArray {
    int n, block; vector<int> a, lazy;
    BlockArray(vector<int>values):n(values.size()),block(max<int>(1,(int)sqrt(n)))
      ,a(move(values)),lazy((n+block-1)/block){}
    void add(int l, int r, int x) {
      assert(0 <= l && l <= r && r <= n);
      while (l < r && l % block) a[l++] += x;
      while (l + block <= r) { lazy[l / block] += x; l += block; }
      while (l < r) a[l++] += x;
    }
    int get(int i)const { assert(0<=i&&i<n); return a[i]+lazy[i/block]; }
  };
  int hilbert_order(int x,int y,int power,int rotation=0){
    assert(0 <= power && power <= 31);
    if (!power) return 0;
    int half=static_cast<int>(1)<<(power-1); int segment=x<half?(y<half?0:3):(y<
      half?1:2);
    segment=(segment+rotation)&3; static constexpr int delta[] {3,0,0,1};
    int size = static_cast<int>(1) << (2 * power - 2);
    auto sub=hilbert_order(x&(half-1),y&(half-1),power-1,(rotation+delta[segment])
      &3);
    return segment*size+((segment==1||segment==2)?sub:size-sub-1);
  }
  // Current range starts empty; add/remove receive array indices, answer receives
       query ID.
  vector<int> mo_queries(int n,const vector<pair<int,int>>& ranges,
      function<void(int)> add,function<void(int)> remove,
      function<int(int)> answer){
    int power = 0;
    while ((static_cast<int>(1) << power) <= (int) n) ++power;
    assert(power <= 31); vector<pair<int, int>> order;
    for (int id = 0; id < (int) ranges.size(); ++id) {
      auto[l, r] = ranges[id]; assert(0 <= l && l <= r && r <= n);
      order.push_back( {hilbert_order(l, r, power), id});
    }
    sort(order.begin(), order.end());
    vector<int> result(ranges.size()); int l=0,r=0;
    for (auto[key, id] : order) {
      (void) key; auto[a, b] = ranges[id];
      while (l > a) add(--l);
```

```
    while (r < b) add(r++);
    while (l < a) remove(l++);
    while (r > b) remove(--r);
    result[id] = answer(id);
  }
  return result;
}
```

1.18 Hash Map (SplitMix64)

Độ phức tạp: $\mathcal{O}(1)$ kỳ vọng mỗi lần tra cứu

Hash map khóa không dấu, hash SplitMix64 với seed do người gọi chọn

```
// Use: HashMap cnt; cnt.reserve(1000); ++cnt[42];
// Signed 63-bit mixer: mask wide products instead of overflowing int.
struct IntegerHash {
  int seed;
  IntegerHash(int seed = 0x712367) : seed(seed) {}
  static int mix(int x) {
    const int MASK = LLONG_MAX;
    x = ((__int128)(x & MASK) + 0x1e3779b97f4a7c15LL) & MASK;
    x = ((__int128)(x ^ (x >> 30)) * 0x3f58476d1ce4e5b9LL) & MASK;
    x = ((__int128)(x ^ (x >> 27)) * 0x14d049bb133111ebLL) & MASK;
    return x ^ (x >> 31);
  }
  size_t operator()(int x) const {
    int sign = x < 0 ? 0x517cc1b727220a95LL : 0;
    return mix((x & LLONG_MAX) ^ (seed & LLONG_MAX)) ^ sign;
  }
};
using HashMap = unordered_map<int,int,IntegerHash>;
// Use: HashMap cnt(0,IntegerHash(seed)); cnt.reserve(expected_size);
```

1.19 Erasable Priority Queue

Độ phức tạp: $\mathcal{O}(\log n)$ mỗi thao tác

Priority queue max/min, xóa lazy phần tử tùy ý

```
// Use: ErasablePriorityQueue q; q.insert(3); // max-heap
// erase removes one occurrence if present. O(log n) insert/erase/pop amortized.
struct ErasablePriorityQueue {
  priority_queue<int> q,deleted; map<int,size_t> counts;
  size_t live = 0;
  void flush() {
    while (!deleted.empty() && !q.empty() && q.top() == deleted.top()) {
      q.pop(); deleted.pop();
    }
  }
  // Max-heap; negate keys for a min-heap.
  size_t size() const { return live; }
  bool empty() const { return live == 0; }
  void insert(const int& x) { q.push(x); ++counts[x]; ++live; }
  bool erase(const int& x) {
    auto it = counts.find(x);
    if (it == counts.end()) return false;
    if (!--it->second) counts.erase(it);
    deleted.push(x); --live; flush(); return true;
  }
  const int& top() { flush(); assert(live); return q.top(); }
  void pop() {
    int value = top(); q.pop(); auto it = counts.find(value);
    if (!--it->second) counts.erase(it);
    --live; flush();
  }
};
```

1.20 Cấu trúc mở rộng (Binary Doubling)

Độ phức tạp: $\mathcal{O}(A \log n)$ thêm khấu hao, với cấu trúc tĩnh dựng được trong $O(A)$

Kỹ thuật gộp block theo lũy thừa 2: thêm online cho cấu trúc tĩnh (Aho, CHT)

```
// Use: ExpandableIndex idx(buildIndex); idx.insert(3);
//      buildIndex(const vector<int>&): vector<int>.
// Binary-counter buckets. Builder(const vector<Value>&) returns a static index
    by value.
// visit(index) called once per occupied bucket; caller combines query results.
struct ExpandableIndex {
  using Value = int;
  using Index = vector<Value>; // Replace with your static index type.
  function<Index(const vector<Value>&)> build;
  vector<vector<Value>> buckets; vector<optional<Index>> indices;
  ExpandableIndex(function<Index(const vector<Value>&)> builder) : build(move(
    builder)){}
  void insert(Value value) {
    vector<Value> carry; carry.push_back(move(value)); size_t level = 0;
    for (;; ++level) {
      if(level==buckets.size()){ buckets.emplace_back(); indices.emplace_back();
      }
      if (buckets[level].empty()) {
        auto index = build(carry); buckets[level] = move(carry);
        indices[level].emplace(move(index)); break;
      }
      for (auto& x : buckets[level]) carry.push_back(move(x));
      buckets[level].clear(); indices[level].reset();
    }
  }
  void query(function<void(const Index&)> visit) const {
    for (const auto& index : indices) if (index) visit(*index);
  }
};
```

1.21 Interval Set

Độ phức tạp: $O((k+1)\log s)$ mỗi lần gán, k đoạn bị xóa, s đoạn đang lưu

Phân hoạch đoạn: gán đoạn, gộp các đoạn kề nhau cùng giá trị.

```
// Use: IntervalSet seg(0,10,0); seg.assign(2,5,7);
// Complete interval partition of [lo,hi), adjacent equal values merged.
// Assign takes O((erased_intervals+1)*log(number_of_intervals)) worst case.
struct IntervalSet {
  struct Entry { int end; int value; }; map<int,Entry>segments;
  int lo, hi;
  auto split(int x) {
    if (x == hi) return segments.end();
    auto it = prev(segments.upper_bound(x));
    if (it->first == x) return it;
    int end=it->second.end; int value=it->second.value; it->second.end=x;
    return segments.emplace(x, Entry {end, move(value)}).first;
  }
  struct Interval { int l, r; int value; };
  IntervalSet(int lo,int hi,int initial):lo(lo),hi(hi){
    assert(lo < hi); segments.emplace(lo, Entry {hi, move(initial)});
  }
  const int& get(int x) const {
    assert(lo<=x&&x<hi); return prev(segments.upper_bound(x))->second.value;
  }
  void assign(int l, int r, int value) {
    assert(lo <= l && l <= r && r <= hi);
    if (l == r) return;
    auto right = split(r), left = split(l); segments.erase(left, right);
    auto it = segments.emplace(l, Entry {r, move(value)}).first;
    if (it != segments.begin()) {
      auto p = prev(it);
      if (p->second.value == it->second.value) {
        p->second.end = it->second.end; segments.erase(it); it = p;
      }
    }
    auto next = std::next(it);
    if(next!=segments.end()&&it->second.value == next->second.value){
      it->second.end = next->second.end; segments.erase(next);
    }
  }
  vector<Interval> intervals() const {
    vector<Interval> out;
    for (auto& [l, e] : segments) out.push_back( {l, e.end, e.value});
    return out;
  }
};
```

2 Đồ thị

2.1 Matroid: nhận diện và các loại

Ví dụ: chọn cạnh không tạo cycle là **graphic matroid**; thêm quota mỗi màu là giao với **partition matroid**. Một matroid dùng greedy; hai matroid cần trao đổi cả chuỗi phần tử.

Định nghĩa: $M = (E, \mathcal{I})$, $\mathcal{I}$ là các tập independent:

- $\varnothing \in \mathcal{I}$; $A \in \mathcal{I}, B \subseteq A \Rightarrow B \in \mathcal{I}$.
- **Exchange**: $A, B \in \mathcal{I}, |A| < |B| \Rightarrow \exists x \in B \setminus A : A + x \in \mathcal{I}$.

Rank $r(X) = \max_{S \subseteq X, S \in \mathcal{I}} |S|$; **basis**: independent tối đại, luôn có $r(E)$ phần tử. **Circuit**: tập dependent tối tiểu. Greedy xét trọng số giảm dần, thêm nếu còn independent, cho maximum-weight basis; nếu không buộc đủ rank thì bỏ trọng số âm.

- **Uniform**: $|S| \le k$.
- **Partition**: E chia nhóm rời nhau E_i, $|S \cap E_i| \le c_i$. Oracle: đếm quota.
- **Graphic**: E là cạnh, S là forest. Oracle: DSU; circuit là cycle.
- **Linear**: E là vector trên cùng field, S độc lập tuyến tính. Oracle: Gauss / XOR basis trên $\mathbb{F}_2$.
- **Laminar**: quota $|S \cap A| \le c_A$, các A đôi một lồng nhau hoặc rời nhau. Ví dụ quota trên mọi prefix.
- **Transversal**: E là phía trái của bipartite graph; S ghép được vào các đỉnh phải khác nhau. Oracle: matching phủ S.

Cue: xóa phần tử vẫn hợp lệ chưa đủ; phải có exchange. Các *tập cạnh matching* không là matroid: trên path ba cạnh, matching gồm cạnh giữa không thêm được cạnh nào từ matching gồm hai cạnh ngoài.

2.2 Matroid intersection (maximum cardinality)

Hai matroid trên **cùng** E; cần oracle $I_j(S)$ kiểm tra independent. Bắt đầu $S = \varnothing$. Mỗi vòng dựng exchange graph với $x \notin S$, $y \in S$:

Cạnh	Điều kiện
$s \to x$	$I_1(S + x)$
$y \to x$	$I_1(S - y + x)$
$x \to y$	$I_2(S - y + x)$
$x \to t$	$I_2(S + x)$

BFS lấy shortest path $s \rightsquigarrow t$: đảo trạng thái chọn của mọi phần tử trên path (bỏ s, t). Khi đó $|S|$ tăng 1 và vẫn independent ở cả hai matroid. Dựng lại graph rồi lặp; không còn path $\Rightarrow$ tối ưu.

Principle khi code: giữ nguyên S khi gọi oracle/dựng graph; chỉ cập nhật sau khi tìm *shortest* path. Không dùng một path tùy ý. Với $N = |E|$, $r = \min(r_1(E), r_2(E))$, oracle tốn T: cách dựng trực tiếp dùng $O(Nr^2)$ oracle calls, thời gian $O(Nr^2(T+1))$. Weighted intersection cần thuật toán riêng; thay BFS bằng heap chưa đủ.

Mô hình: rainbow forest = graphic + quota màu; bipartite matching = hai partition theo hai đầu cạnh. Giao hai matroid thường không còn là matroid nên greedy trực tiếp không đúng.

Interface: `MatroidIntersection(n, oracle1, oracle2).solve()` trả về các ID được chọn. Oracle nhận `const vector<int>&` gồm ID của tập cần kiểm tra, trả về `bool`; không giữ tham chiếu vào tập này.

```
// Use: MatroidIntersection mi(n,oracle1,oracle2); auto ids = mi.solve();
//      Each oracle: bool(const vector<int>& selectedIDs).
struct MatroidIntersection {
    // Oracle receives element IDs of a proposed set; no duplicates.
    using Oracle = function<bool(const vector<int>&)>;
    int n;
```

```
    Oracle independent1, independent2;

    MatroidIntersection(int n, Oracle first, Oracle second)
        : n(n), independent1(move(first)), independent2(move(second)) {}

    // Maximum cardinality; ground set is [0,n). Not weighted.
    vector<int> solve() const {
        vector<char> chosen(n, false);
        vector<int> selected;
        while (true) {
            int source = n, sink = n + 1;
            vector<vector<int>> graph(n + 2);
            vector<int> trial = selected;
            for (int x = 0; x < n; ++x) if (!chosen[x]) {
                trial.push_back(x);
                if (independent1(trial)) graph[source].push_back(x);
                if (independent2(trial)) graph[x].push_back(sink);
                trial.pop_back();
                for (int i = 0; i < static_cast<int>(selected.size()); ++i) {
                    int y = selected[i];
                    trial[i] = x;
                    if (independent1(trial)) graph[y].push_back(x);
                    if (independent2(trial)) graph[x].push_back(y);
                    trial[i] = y;
                }
            }
            vector<int> parent(n + 2, -1);
            queue<int> q;
            parent[source] = source;
            q.push(source);
            while (!q.empty() && parent[sink] == -1) {
                int u = q.front(); q.pop();
                for (int v : graph[u]) if (parent[v] == -1) {
                    parent[v] = u;
                    q.push(v);
                }
            }
            if (parent[sink] == -1) return selected;
            for (int x = parent[sink]; x != source; x = parent[x])
                chosen[x] ^= 1;
            selected.clear();
            for (int x = 0; x < n; ++x)
                if (chosen[x]) selected.push_back(x);
        }
    }
};
```

2.3 Đếm thứ tự topo trên cây/rừng

Cạnh hướng cha → con: số thứ tự topo là $\dfrac{n!}{\prod_v sz(v)}$, với $sz(v)$ là kích thước subtree kể cả v. Ghép các thứ tự con bằng multinomial; đảo toàn bộ cạnh vẫn cùng số cách. Không áp dụng cho DAG tổng quát.

2.4 Minimum Cost Flow

Độ phức tạp: $O(nm + Am \log n)$, A lần tăng luồng

Min-cost k-flow: chuyển k đơn vị qua đồ thị có capacity và chi phí

```
// Use: MinCostFlow flow(3); flow.add_edge(0,1,5,2);
//      auto ans = flow.send(0,1,3); // ans.flow, ans.cost
// Capacities nonnegative; no reachable negative-cost cycle in residual graph.
// Bellman-Ford initializes potentials; Dijkstra for each augmentation.
struct MinCostFlow {
  struct Edge { int to,rev; int cap,cost; }; vector<vector<Edge>>g;
  struct Result { int flow, cost; };
  MinCostFlow(int n) : g(n) {}
  void add_edge(int u, int v, int cap, int cost) {
    assert(cap >= 0); int a = g[u].size(), b = g[v].size();
    g[u].push_back( {v,b+(u==v),cap,cost}); g[v].push_back( {u,a,0,-cost});
  }
  Result send(int s, int t, int limit) {
    assert(s!=t&&limit>=0); int n=g.size(); constexpr int inf=1LL<<60;
    vector<int> potential(n, inf); potential[s] = 0;
    for (int iter = 0; iter < n; ++iter) {
      bool changed = false;
      for(int u=0;u<n;++u)if(potential[u]<inf)for(auto&e:g[u])if(e.cap&&
     potential[e.to]>potential[u]+e.cost){
        potential[e.to] = potential[u] + e.cost; changed = true;
      }
      if (!changed) break;
      if (iter == n - 1) throw invalid_argument("negative cost cycle");
    }
    for (auto& x : potential) if (x == inf) x = 0;
    Result result {0, 0};
    while (result.flow < limit) {
      vector<int>dist(n,inf); vector<int>pv(n),pe(n); dist[s]=0;
      priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>
     q;
      q.push( {0, s});
      while (!q.empty()) {
        auto [d, u] = q.top(); q.pop();
        if (d != dist[u]) continue;
        for (int i = 0; i < static_cast<int>(g[u].size()); ++i) {
          auto& e = g[u][i];
          if (!e.cap) continue;
          int nd = d + e.cost + potential[u] - potential[e.to];
          if (nd < dist[e.to]) {
            dist[e.to]=nd; pv[e.to]=u; pe[e.to]=i; q.push( {nd,e.to});
          }
        }
      }
      if (dist[t] == inf) break;
      for(int u=0; u<n; ++u)if(dist[u]<inf)potential[u]+=dist[u];
      int f = limit - result.flow, cost = 0;
      for(int v=t; v!=s; v=pv[v])f=min(f,g[pv[v]][pe[v]].cap);
      for (int v = t; v != s; v = pv[v]) {
        auto&e=g[pv[v]][pe[v]]; cost+=e.cost; e.cap-=f; g[v][e.rev].cap+=f;
      }
      result.flow += f; result.cost += f * cost;
    }
    return result;
  }
};
```

2.5 MinCut - MaxFlow

Độ phức tạp: $\mathcal{O}(EV^2)$

Dinic: số đường đi/đường đi không chung đỉnh lớn nhất từ source đến sink.

- **Tô hai màu lưới**: Chia ô theo $(r + c) \bmod 2 \implies$ đồ thị hai phía. Dùng lát domino, đặt quân không tấn công nhau, bắn tia laser trên lưới.
- **Tách đỉnh (Vertex Splitting)** ($u_{in} \to u_{out}$): Giới hạn capacity đỉnh hoặc buộc đi qua đỉnh (capacity c_u, chi phí w_u).
- **Max-Weight Closure (Project Selection)**:
  - Quy tắc: chọn u thì phải chọn v (cạnh có hướng $u \to v$ với cap $= \infty$).
  - Trọng số dương $W_u > 0$: thêm cạnh $S \to u$ với cap $= W_u$.
  - Trọng số âm $W_v < 0$: thêm cạnh $v \to T$ với cap $= -W_v$.
  - Max Profit $= \sum_{W_u>0} W_u - \text{MinCut}(S, T)$.
- **Phân nhóm với hai lựa chọn**: Gán i vào S (chi phí A_i) hoặc T (B_i). Nếu $i \in S, j \in T$, phạt $C_{i,j}$. Thêm $S \to i$ (B_i), $i \to T$ (A_i), $i \to j$ ($C_{i,j}$). Tổng chi phí = min-cut.
- **Circulation có cận dưới/trên** $[L_e, R_e]$: Thay cạnh $e = (u, v)$ bằng capacity $R_e - L_e$. Đặt $d(u) = \sum_{in} L - \sum_{out} L$. Thêm super-source S^*, super-sink T^*. Với $d(u) > 0$, thêm $S^* \to u$ capacity $d(u)$; với $d(u) < 0$, thêm $u \to T^*$ capacity $-d(u)$. Khả thi $\iff$ mọi cạnh từ S^* bão hòa.

```
// Use: Dinic flow(3); flow.add_edge(0,1,5); flow.add_edge(1,2,5);
//      auto f = flow.max_flow(0,2);
// Directed residual network. add_edge returns forward edge ID; self-loops
    supported.
struct Dinic {
  struct Edge { int from,to; int capacity,flow; }; vector<Edge>edges;
  vector<vector<int>> adj; vector<int> level, next;
  int dfs(int u, int sink, int pushed) {
    if (u == sink) return pushed;
    for (int& i = next[u]; i < static_cast<int>(adj[u].size()); ++i) {
      int id = adj[u][i]; auto& e = edges[id];
      if (level[e.to] != level[u] + 1 || e.flow == e.capacity) continue;
      int f = dfs(e.to, sink, min(pushed, e.capacity - e.flow));
      if (f) { e.flow += f; edges[id ^ 1].flow -= f; return f; }
    }
    return 0;
  }
  Dinic(int n) : adj(n), level(n), next(n) {}
  int add_edge(int u, int v, int capacity) {
    assert(capacity >= 0); int id = edges.size(); adj[u].push_back(id);
    adj[v].push_back(id + 1); edges.push_back( {u, v, capacity, 0});
    edges.push_back( {v, u, 0, 0}); return id;
  }
  int max_flow(int source, int sink, int limit = LLONG_MAX){
    assert(source != sink); int total = 0;
    while (total < limit) {
      fill(level.begin(),level.end(),-1); queue<int>q; q.push(source); level[
     source]=0;
      while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int id : adj[u]) {
          auto& e = edges[id];
          if (level[e.to] < 0 && e.flow < e.capacity) {
            level[e.to] = level[u] + 1; q.push(e.to);
          }
        }
      }
      if (level[sink] < 0) break;
      fill(next.begin(), next.end(), 0);
      while (total < limit) {
        int f = dfs(source, sink, limit - total);
        if (!f) break;
        total += f;
      }
    }
    return total;
    // additional flow; mutates residual network.
  }
  vector<bool> source_side(int source) const {
    vector<bool>seen(adj.size()); queue<int>q; q.push(source); seen[source]=true
     ;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int id : adj[u]) {
        auto& e = edges[id];
        if(!seen[e.to]&&e.flow<e.capacity){ seen[e.to]=true; q.push(e.to); }
      }
    }
    return seen;
  }
};
```

2.6 Bipartite Matching

Độ phức tạp: $\mathcal{O}(E\sqrt{V})$

Hopcroft-Karp: vertex cover, path cover không chung đỉnh/tổng quát, antichain (Dilworth).

- **Định lý Kőnig**: Max Matching = Min Vertex Cover.
- **Max Independent Set**: $|V|$ − Min Vertex Cover $= |V|$ − Max Matching.
- **Min Edge Cover** (không có đỉnh cô lập): $|V|$ − Max Matching.
- **Path cover nhỏ nhất trên DAG**:
  - *Không chung đỉnh*: Tách $u \to u_{out}, u_{in}$; dựng matching hai phía $u_{out} \to v_{in}$ cho cạnh $u \to v$. Kết quả: $|V|$ − Max Matching.
  - *Tổng quát (được chung đỉnh)*: Tính transitive closure (Floyd-Warshall/reachability), rồi giải path cover không chung đỉnh.
- **Định lý Dilworth**: Trong poset hữu hạn (reachability trên DAG), kích thước **maximum antichain** (tập lớn nhất gồm các đỉnh không đến được nhau) bằng **minimum path cover**.

- **Erdős–Gallai**: $n-1 \geq d_1 \geq \cdots \geq d_n \geq 0$ là dãy bậc đồ thị đơn vô hướng iff $\sum d_i$ chẵn và $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$ $(\forall 1 \leq k \leq n)$. Dựng hình → **Havel–Hakimi**: lấy $\max d$, nối d đỉnh lớn kế tiếp rồi trừ 1.

- **Gale–Ryser**: Hai dãy nguyên không âm $a_1 \geq \cdots \geq a_n$, $b_1, \ldots, b_m$ là bậc hai phía của đồ thị đơn iff $\sum a_i = \sum b_j$ và $\sum_{i=1}^{k} a_i \leq \sum_{j=1}^{m} \min(b_j, k)$ với mọi $1 \leq k \leq n$. Dùng cho ma trận 0/1 có tổng hàng/cột cho trước.

- **Định lý Hall**: Đồ thị hai phía (X, Y) có matching phủ hết $X \iff \forall S \subseteq X, |N(S)| \geq |S|$.

- **Kích thước maximum matching qua Hall**: $|X| - \max_{S \subseteq X}(|S| - |N(S)|)$.

```cpp
// Use: HopcroftKarp hk(2,3); hk.add_edge(0,1); int k = hk.solve();
//      hk.left[u], hk.right[v]: matched ID; -1 if unmatched.
// Left vertices [0,L), right [0,R). solve() returns maximum cardinality.
struct HopcroftKarp {
  vector<vector<int>> g; vector<int> distance; int shortest;
  bool augment(int u) {
    for (int v : g[u]) {
      int next = right[v];
      if((next<0&&distance[u]+1==shortest)||(next>=0&&distance[next]==distance[u]+1&&augment(next))){
        left[u] = v; right[v] = u; return true;
      }
    }
    distance[u] = -1; return false;
  }
  vector<int> left, right;
  HopcroftKarp(int L,int R):g(L),distance(L),left(L,-1),right(R,-1){}
  void add_edge(int u, int v) { g[u].push_back(v); }
  int solve() {
    int count=0; for (int x:left) if (x>=0) ++count;
    for (;;) {
      fill(distance.begin(),distance.end(),-1); queue<int>q; shortest=INT_MAX;
      for (int u = 0; u < static_cast<int>(g.size()); ++u) if (left[u] < 0) {
        distance[u] = 0; q.push(u);
      }
      while (!q.empty()) {
        int u = q.front(); q.pop();
        if (distance[u] + 1 > shortest) continue;
        for (int v : g[u]) if (right[v] < 0) shortest = distance[u] + 1;
        else if (distance[right[v]] < 0) {
          distance[right[v]] = distance[u] + 1; q.push(right[v]);
        }
      }
      if (shortest == INT_MAX) break;
      for(int u=0; u<static_cast<int>(g.size()); ++u)if(left[u]<0&&augment(u))++count;
    }
    return count;
  }
  // Call after solve(); returns {left IDs,right IDs} in a minimum vertex cover.
  pair<vector<int>, vector<int>> min_vertex_cover() const {
    vector<bool> l(left.size()), r(right.size()); queue<int> q;
    for (int u = 0; u < static_cast<int>(left.size()); ++u) if (left[u] < 0) {
      l[u] = true; q.push(u);
    }
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : g[u]) if (left[u] != v && !r[v]) {
        r[v] = true; int next = right[v];
        if (next >= 0 && !l[next]) { l[next] = true; q.push(next); }
      }
    }
    pair<vector<int>, vector<int>> answer;
    for(int u=0; u<static_cast<int>(l.size()); ++u)if(!l[u])answer.first.push_back(u);
    for(int v=0; v<static_cast<int>(r.size()); ++v)if(r[v])answer.second.push_back(v);
    return answer;
  }
};
```

2.7 Hungarian Matching

Độ phức tạp: $\mathcal{O}(V^3)$

Ghép cặp hoàn hảo chi phí nhỏ nhất trên đồ thị hai phía

```cpp
// Use: auto ans = hungarian({{3,1},{2,4}}); // ans.cost, ans.column
struct Assignment { int cost; vector<int> column; };
// Minimum rectangular assignment, rows<=columns; costs may be negative.
Assignment hungarian(const vector<vector<int>>& a) {
  int n = a.size();
  if (!n) return {
    0, {}
  }; int m=a[0].size(); assert(n<=m); vector<int>u(n+1),v(m+1);
  vector<int> p(m + 1), way(m + 1);
  for (int i = 1; i <= n; ++i) {
    p[0]=i; int j0=0; vector<int>best(m+1,1LL<<60); vector<bool>used(m+1);
    do {
      used[j0]=true; int row=p[j0],j1=0; int delta=1LL<<60;
      for (int j = 1; j <= m; ++j) if (!used[j]) {
        int value = a[row - 1][j - 1] - u[row] - v[j];
        if (value < best[j]) { best[j] = value; way[j] = j0; }
        if (best[j] < delta) { delta = best[j]; j1 = j; }
      }
      for (int j = 0; j <= m; ++j) if (used[j]) {
        u[p[j]] += delta; v[j] -= delta;
      } else best[j] -= delta;
      j0 = j1;
    }
    while (p[j0]);
    do { int previous = way[j0]; p[j0] = p[previous]; j0 = previous; }
    while (j0);
  }
  vector<int> column(n);
  for (int j = 1; j <= m; ++j) if (p[j]) column[p[j] - 1] = j - 1;
  return {-v[0], column};
}
```

2.8 General Matching

Độ phức tạp: $\mathcal{O}(V^3)$

Edmonds Blossom: maximum matching trên đồ thị tổng quát.

- **Tutte Matrix & Rabin-Vazirani ($O(V^3)$)**: Với $(u, v) \in E$ $(u < v)$, đặt $T_{u,v} = r \in [1, p-1]$, $T_{v,u} = -r \pmod p$. Kích thước $= \frac{1}{2}\text{rank}(T)$ (Gauss mod p).
  - *Khôi phục Rabin-Vazirani*: Tính $A = T^{-1} \pmod p$ (trên cơ sở $2K \times 2K$). Cạnh (u, v) thuộc một matching $\iff A_{u,v} \neq 0$. Chọn cặp đó, ghép lại rồi cập nhật A trong $O(V^2)$ bằng Schur complement: $A'_{x,y} = A_{x,y} + \frac{A_{x,u}A_{y,v} - A_{x,v}A_{y,u}}{A_{u,v}} \pmod p$. Lặp cho K cặp $\implies$ tổng $O(V^3)$.

- **Quy về max b-matching** $(\deg(i) \leq a_i)$: Đỉnh $i \to a_i$ slot $(a_i \leftarrow \min(a_i, \deg(i)))$. Cạnh $e = (u, v) \to L_e, R_e$, nối $L_e - R_e$, $L_e - \text{slot}(u)$, $R_e - \text{slot}(v)$. Bỏ e: match $L_e - R_e$ $(+1)$. Chọn e: match L_e, R_e với slot $(+2$, lợi thêm $+1)$. Đồ thị phụ $|V'| \leq 4M \implies$ Blossom.

```cpp
// Use: auto mate = general_matching({{1},{0,2},{1}});
// Edmonds blossom; simple undirected adjacency. Returns mate[u] or -1.
struct BlossomMatching {
  const vector<vector<int>>& adjacency;
  int n; mt19937 rng{712367};
  vector<int> match,aux,label,orig,parent; queue<int> q; int aux_time=-1;
  BlossomMatching(const vector<vector<int>>& g):adjacency(g),n(g.size()),
      match(n,-1),aux(n,-1),label(n),orig(n),parent(n,-1) {}
  int lca(int v, int u) {
    aux_time++;
    while (true) {
      if (v != -1) {
        if (aux[v] == aux_time) { return v; }
        aux[v] = aux_time;
        if(match[v]==-1){ v=-1; } else { v=orig[parent[match[v]]]; }
      }
      swap(v, u);
    }
  }
  // lca
  void blossom(int v, int u, int a) {
    while (orig[v] != a) {
      parent[v] = u; u = match[v];
      if (label[u] == 1) { label[u] = 0; q.push(u); }
      orig[v] = orig[u] = a; v = parent[u];
    }
  }
  // blossom
  void augment(int v) {
    while (v != -1) {
      int pv=parent[v]; int next_v=match[pv]; match[v]=pv; match[pv]=v;
      v = next_v;
    }
  }
  // augment
  bool bfs(int root) {
    fill(label.begin(),label.end(),-1); iota(orig.begin(),orig.end(),0);
    while (!q.empty()) { q.pop(); }
    q.push(root); label[root] = 0;
    while (!q.empty()) {
      int v = q.front(); q.pop();
      for (int u : adjacency[v]) {
        if (label[u] == -1) {
          label[u] = 1; parent[u] = v;
          if (match[u] == -1) { augment(u); return true; }
          label[match[u]] = 0; q.push(match[u]); continue;
        } else if (label[u] == 0 && orig[v] != orig[u]) {
          int a=lca(orig[v],orig[u]); blossom(u,v,a); blossom(v,u,a);
        }
      }
    }
    return false;
  }
  // bfs
  void greedy() {
    vector<int> order(n); iota(order.begin(), order.end(), 0);
    shuffle(order.begin(), order.end(), rng);
    for (int i : order) {
      if (match[i] == -1) {
        for (int to : adjacency[i]) {
          if (match[to] == -1) { match[i] = to; match[to] = i; break; }
        }
      }
    }
  }
  vector<int> solve() {
    greedy();
    for (int i=0;i<n;++i) if (match[i]==-1) bfs(i);
    return match;
  }
};
vector<int> general_matching(const vector<vector<int>>& adjacency) {
  BlossomMatching solver(adjacency); return solver.solve();
}
```

2.9 Biconnected Component

Độ phức tạp: $\mathcal{O}(V + E)$

BCC: khớp, cầu và thành phần song liên thông

```cpp
// Use: auto bcc = biconnected_components(3,{{0,1},{1,2}});
struct BiconnectedResult {
  vector<int>bridges; vector<bool>articulation; vector<vector<int>>edge_blocks;
};
// Undirected multigraph; IDs are positions in edges. Self-loops form singleton blocks.
BiconnectedResult biconnected_components(int n,const vector<pair<int,int>>&edges){
  vector<vector<pair<int, int>>> g(n);
  BiconnectedResult result { {}, vector<bool>(n), {} };
  for (int id = 0; id < static_cast<int>(edges.size()); ++id) {
    auto [u, v] = edges[id];
    if (u == v) { result.edge_blocks.push_back( {id}); continue; }
    g[u].push_back( {v, id}); g[v].push_back( {u, id});
  }
  vector<int> tin(n, -1), low(n), stack; int timer = 0;
  struct BCCDFS {
    const vector<vector<pair<int,int>>>& g;
    vector<int>& tin; vector<int>& low; vector<int>& stack;
    int& timer; BiconnectedResult& result;
```

```
    void dfs(int u,int parent_edge) {
      tin[u] = low[u] = timer++; int children = 0;
      for (auto [v, id] : g[u]) if (id != parent_edge) {
        if (tin[v] < 0) {
          ++children; stack.push_back(id); dfs(v,id); low[u]=min(low[u],low[v]);
          if (low[v] > tin[u]) result.bridges.push_back(id);
          if (low[v] >= tin[u]) {
            if (parent_edge >= 0) result.articulation[u] = true;
            result.edge_blocks.push_back( {}); int e;
            do {
              e=stack.back(); stack.pop_back(); result.edge_blocks.back().
    push_back(e);
            }
            while (e != id);
          }
        } else if(tin[v]<tin[u]){ stack.push_back(id); low[u]=min(low[u],tin[v])
    ; }
      }
      if (parent_edge < 0) result.articulation[u] = children > 1;
    }
    } helper{g,tin,low,stack,timer,result};
    for (int u = 0; u < n; ++u) if (tin[u] < 0) helper.dfs(u, -1);
    return result;
  }
```

2.10 Functional Cycle

Độ phức tạp: $O(\text{dist} + \text{len})$

Floyd Tortoise and Hare: tìm chu trình trong functional graph $f(x)$

```
  // Use: int nextState(int x) { return (x+1)%3; }
  //      auto cycle = find_cycle(0,nextState);
  using CycleState = int; // Change the state type if needed.
  struct CycleInfo {
    CycleState entry; size_t prefix_length, cycle_length;
  };
  // f must eventually cycle; equality compares complete states. O(1) extra space.
  CycleInfo find_cycle(CycleState start,function<CycleState(CycleState)> f){
    CycleState slow = f(start), fast = f(f(start));
    while (slow != fast) { slow = f(slow); fast = f(f(fast)); }
    size_t prefix = 0; slow = start;
    while (slow != fast) { slow = f(slow); fast = f(fast); ++prefix; }
    size_t length = 1; fast = f(slow);
    while (slow != fast) { fast = f(fast); ++length; }
    return {slow, prefix, length};
  }
```

2.11 Strongly Connected Components

Độ phức tạp: $\mathcal{O}(V + E)$

Tarjan SCC trên đồ thị có hướng

```
  // Use: auto scc = strongly_connected_components({{1},{0,2},{}});
  struct SCCResult { vector<int>component; vector<vector<int>>vertices; };
  // O(V+E). IDs assigned sink-first: every condensation edge has ID[from]>ID[to].
  SCCResult strongly_connected_components(const vector<vector<int>>& g){
    int n=g.size(),timer=0; vector<int>number(n,-1),low(n),stack; vector<bool>
      active(n);
    SCCResult result { vector<int>(n, -1), {} };
    struct SCCDFS {
      const vector<vector<int>>& g;
      vector<int>& number; vector<int>& low; vector<int>& stack;
      vector<bool>& active; int& timer; SCCResult& result;
      void dfs(int u) {
        number[u] = low[u] = timer++; stack.push_back(u); active[u] = true;
        for (int v : g[u]) {
          if (number[v] < 0) {
            dfs(v); low[u] = min(low[u], low[v]);
          } else if (active[v]) low[u] = min(low[u], number[v]);
        }
        if (low[u] == number[u]) {
          result.vertices.push_back( {}); int id=result.vertices.size()-1;
          for (;;) {
            int v=stack.back(); stack.pop_back(); active[v]=false; result.
    component[v]=id;
            result.vertices.back().push_back(v);
            if (v == u) break;
          }
        }
      }
      helper{g,number,low,stack,active,timer,result};
      for (int u = 0; u < n; ++u) if (number[u] < 0) helper.dfs(u);
      return result;
    }
```

2.12 Xóa offline (DSU Rollback)

Độ phức tạp: $\mathcal{O}(T(n) \log n)$

DSU rollback kết hợp D&C trên segment tree

```
  // Use: RollbackDSU dsu(3); auto snap = dsu.snapshot();
  //      dsu.unite(0,1); dsu.rollback(snap);
  // No path compression; O(log n) find. Roll back to a snapshot, including no-op
      unions.
  struct RollbackDSU {
    vector<int>parent,size; struct Change { int child,root,old_size; };
    vector<Change> history; int components;
    RollbackDSU(int n) : parent(n), size(n, 1), components(n) {
      iota(parent.begin(), parent.end(), 0);
    }
    int find(int u) const {
      while (parent[u] != u) u = parent[u];
      return u;
    }
    size_t snapshot() const { return history.size(); }
    bool unite(int u, int v) {
      u = find(u); v = find(v);
      if (u == v) return false;
      if (size[u] < size[v]) swap(u, v);
      history.push_back( {v,u,size[u]}); parent[v]=u; size[u]+=size[v]; --
      components;
      return true;
    }
    void rollback(size_t snapshot) {
      assert(snapshot <= history.size());
      while (history.size() > snapshot) {
        auto c=history.back(); history.pop_back(); parent[c.child]=c.child;
        size[c.root] = c.old_size; ++components;
      }
    }
  };
```

2.13 2-SAT

Độ phức tạp: $\mathcal{O}(n + m)$

Giải 2-SAT bằng đồ thị kéo theo (implication graph) và SCC.

- **Ít nhất một biến đúng** ($X \vee Y$): Thêm các cạnh $\neg X \to Y$ và $\neg Y \to X$.
- **Nếu** X **thì** Y ($X \implies Y$): Thêm các cạnh $X \to Y$ và $\neg Y \to \neg X$.
- **Đúng một biến đúng** (X **XOR** $Y = 1$): $(X \vee Y) \wedge (\neg X \vee \neg Y)$. Các cạnh: $X \leftrightarrow \neg Y$ và $\neg X \leftrightarrow Y$.
- X **và** Y **phải bằng nhau** (X **XOR** $Y = 0$): $(X \vee \neg Y) \wedge (\neg X \vee Y)$. Các cạnh: $X \leftrightarrow Y$ và $\neg X \leftrightarrow \neg Y$.
- **Buộc** X **đúng/sai**: Thêm cạnh $\neg X \to X$ (buộc đúng), hoặc $X \to \neg X$ (buộc sai).
- **Nhiều nhất một biến trong** $\{X_1, X_2, \ldots, X_k\}$ **đúng**: Thêm biến prefix $P_i = P_{i-1} \vee X_i$. Các cạnh: $X_i \to P_i$, $P_{i-1} \to P_i$, và $X_i \to \neg P_{i-1}$ (nếu X_i đúng thì mọi biến trước phải sai).

```
  // Use: TwoSAT sat(3); sat.add_or(sat.literal(0,true),
  //                                sat.literal(1,false));
  //      auto ans = sat.solve(); // optional<vector<bool>>
  // Literal 2*v=true, 2*v+1=false. add_or accepts encoded literals.
  struct TwoSAT {
    vector<vector<int>> g;
    TwoSAT(int variables) : g(2 * variables) {}
    static int literal(int variable,bool value){ return 2*variable+!value; }
    void imply(int a,int b){ g[a].push_back(b); g[b^1].push_back(a^1); }
    void add_or(int a, int b) { imply(a ^ 1, b); }
    void force(int a) { add_or(a, a); }
    optional<vector<bool>> solve() const {
      auto s=strongly_connected_components(g); vector<bool>answer(g.size()/2);
      for (int v = 0; v < static_cast<int>(answer.size()); ++v) {
        if (s.component[2 * v] == s.component[2 * v + 1]) return nullopt;
        answer[v] = s.component[2 * v] < s.component[2 * v + 1];
      }
      return answer;
    }
  };
```

2.14 Floyd

Độ phức tạp: $O(n^3)$

Đường đi ngắn nhất mọi cặp/transitive closure/đường đi min-max

```
  // Use: bool ok = floyd_warshall(dist); // vector<vector<int>>
  // Matrix includes diagonal 0, inf for absent edges. Reports whether a negative
      cycle exists.
  // Distances involving a negative cycle require separate -infinity propagation.
  bool floyd_warshall(vector<vector<int>>&d,int inf=1LL<<60){
    int n = d.size();
    for(int k=0;k<n;++k)for(int i=0;i<n;++i)if(d[i][k]<inf)for(int j=0;j<n;++j)if(
      d[k][j]<inf)d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
    for (int i = 0; i < n; ++i) if (d[i][i] < 0) return true;
    return false;
  }
```

2.15 Eulerian Path

Độ phức tạp: $\mathcal{O}(m)$

Hierholzer: tìm đường đi/chu trình Euler.

- **Nối từ/chuỗi (De Bruijn/domino)**: Nối từ sao cho đuôi từ trước khớp đầu từ sau. Đỉnh là *trạng thái* (prefix/suffix dài $L-1$ hoặc nửa domino); cạnh có hướng là *đối tượng*. Đường đi Euler đi qua mỗi đối tượng đúng một lần.
- **Định hướng cạnh vô hướng**: Định hướng cạnh để cân bằng bậc vào/ra (ví dụ $|in - out| \leq 1$). Thêm cạnh giả (hoặc nối đỉnh bậc lẻ tới đỉnh giả) để mọi bậc chẵn; tìm chu trình Euler rồi định hướng theo lượt duyệt.
- **Phân rã chu trình không chung cạnh**: Phân rã thành chu trình không chung cạnh. Khả thi $\iff$ đồ thị liên thông và mọi đỉnh bậc chẵn. Tìm chu trình Euler rồi tách các chu trình.
- **Duyệt lưới bằng đồ thị hàng-cột**: Bài ma trận di chuyển luân phiên hàng/cột: dựng đồ thị hai phía gồm hàng và cột. Ô (r, c) là cạnh nối hàng r với cột c. Đường đi Euler duyệt các ô theo hàng-cột-hàng.

```
  // Use: auto trail = euler_trail(3,{{0,1},{1,2}},false);
  struct EulerTrail { vector<int> vertices, edge_ids; };
  // Directed or undirected multigraph, self-loops allowed. nullopt if no complete
      trail.
  optional<EulerTrail>euler_trail(int n,const vector<pair<int,int>>&edges,bool
      directed=false,optional<int>requested_start=nullopt){
    if(n==0)return edges.empty()?optional<EulerTrail>(EulerTrail {}):nullopt;
    vector<vector<pair<int, int>>> g(n); vector<int> in(n), out(n);
    for (int id = 0; id < static_cast<int>(edges.size()); ++id) {
      auto [u,v]=edges[id]; g[u].push_back( {v,id}); ++out[u]; ++in[v];
      if (!directed) { g[v].push_back( {u, id}); ++out[v]; ++in[u]; }
    }
    int forced = -1, ends = 0;
    if (directed) {
      int sinks = 0;
      for (int u = 0; u < n; ++u) {
        if (out[u] - in[u] == 1) {
          forced = u; ++ends;
        } else if (in[u] - out[u] == 1) ++sinks;
        else if (in[u] != out[u]) return nullopt;
      }
      if (ends != sinks || ends > 1) return nullopt;
    } else for (int u = 0; u < n; ++u) if (out[u] & 1) {
```

```
    ++ends; forced = u;
  }
  if (!directed && ends != 0 && ends != 2) return nullopt;
  int start = requested_start.value_or(forced >= 0 ? forced : 0);
  if(!requested_start&&forced<0)for(int u=0; u<n; ++u)if(out[u]){
    start = u; break;
  }
  if (start < 0 || start >= n) return nullopt;
  if (directed && forced >= 0 && start != forced) return nullopt;
  if (!directed && ends && !(out[start] & 1)) return nullopt;
  vector<int> next(n); vector<bool> used(edges.size());
  vector<pair<int, int>> stack { {start, -1} }; EulerTrail answer;
  while (!stack.empty()) {
    int u = stack.back().first;
    while(next[u]<static_cast<int>(g[u].size())&&used[g[u][next[u]].second])++
     next[u];
    if (next[u] == static_cast<int>(g[u].size())) {
      auto [v,id]=stack.back(); stack.pop_back(); answer.vertices.push_back(v);
      if (id >= 0) answer.edge_ids.push_back(id);
    }else { auto [v,id]=g[u][next[u]++]; used[id]=true; stack.push_back( {v,id})
     ; }
  }
  if (answer.edge_ids.size() != edges.size()) return nullopt;
  reverse(answer.vertices.begin(), answer.vertices.end());
  reverse(answer.edge_ids.begin(),answer.edge_ids.end()); return answer;
}
```

2.16 Định lý Vizing

Độ phức tạp: $O(nm)$ thời gian, $O(n^2)$ bộ nhớ

Tô cạnh đồ thị đơn bằng tối đa $\Delta + 1$ màu

```
// Use: auto colors = vizing_coloring(3,{{0,1},{1,2},{0,2}});
// Simple undirected graph; returns edge colors 0..Delta. O(n^2) storage.
struct VizingColoring {
  int N,a; vector<pair<int,int>> E;
  vector<vector<int>> C,G; vector<int> X;
  VizingColoring(int n,const vector<pair<int,int>>& edges):N(n),
      C(n+1,vector<int>(n+2)),G(n+1,vector<int>(n+1)),X(n+1) {
    for (auto [u,v]:edges) { assert(u!=v); E.push_back({u+1,v+1}); }
  }
  void update(int u) {
    for (X[u] = 1; C[u][X[u]]; X[u]++);
  }
  int color(int u, int v, int c) {
    int p = G[u][v];
    if (p == c) return p;
    G[u][v]=G[v][u]=c; C[u][c]=v; C[v][c]=u; C[u][p]=C[v][p]=0;
    if ( p ) X[u] = X[v] = p;
    else update(u), update(v);
    return p;
  }
  // end of function : color
  int flip(int u, int c1, int c2) {
    int p = C[u][c1]; swap(C[u][c1], C[u][c2]);
    if ( p ) G[u][p] = G[p][u] = c2;
    if ( !C[u][c1] ) X[u] = c1;
    if ( !C[u][c2] ) X[u] = c2;
    return p;
  }
  // end of function : flip
  vector<int> solve() {
  for (int i = 1; i <= N; i++) X[i] = 1;
  for (int t = 0; t < (int) E.size(); t++) {
    int u=E[t].first,v0=E[t].second,v=v0,c0=X[u],c=c0,d; vector<pair<int,int>>L;
    vector<int> vst(N + 2);
    while (!G[u][v0]) {
      L.emplace_back(v, d = X[v]);
      if(!C[v][c])for(a=(int)L.size()-1; a>=0; a--)c=color(u,L[a].first,c);
      else if(!C[u][d])for(a=(int)L.size()-1;a>=0;a--)color(u,L[a].first,L[a].
     second);
      else if ( vst[d] ) break;
      else vst[d] = 1, v = C[u][d];
    }
    if ( !G[u][v0] ) {
      for (; v; v = flip(v, c, d), swap(c, d));
      if (C[u][c0]) {
        for (a = (int) L.size() - 2; a >= 0 && L[a].second != c; a--);
        for (; a >= 0; a--) color(u, L[a].first, L[a].second);
      } else t--;
    }
  }
  vector<int> result;
  for (auto [u, v] : E) result.push_back(G[u][v] - 1);
  return result;
  }
};
vector<int> vizing_coloring(int n,const vector<pair<int,int>>& edges) {
  VizingColoring solver(n,edges); return solver.solve();
}
```

2.17 Kiểm tra hai phía online

Độ phức tạp: $\mathcal{O}(\alpha(n))$

Kiểm tra hai phía online khi thêm cạnh bằng parity DSU

```
// Use: BipartiteDSU dsu(3); dsu.add_edge(0,1);
// Append undirected edges; is_bipartite(u) tests the current component.
struct BipartiteDSU {
  vector<int> parent, size, parity; vector<bool> good;
  pair<int, int> find(int u) {
    if (parent[u] == u) return {u, 0};
    auto [root, p] = find(parent[u]); parity[u] ^= p; parent[u] = root;
    return {root, parity[u]};
  }
  BipartiteDSU(int n):parent(n),size(n,1),parity(n),good(n,true){
    iota(parent.begin(), parent.end(), 0);
  }
  bool add_edge(int u, int v) {
    auto [a, x] = find(u); auto [b, y] = find(v);
    if (a == b) {
      if (x == y) good[a] = false;
      return good[a];
    }
    if (size[a] < size[b]) { swap(a, b); swap(x, y); }
    parent[b]=a; parity[b]=x^y^1; size[a]+=size[b]; good[a]=good[a]&&good[b];
    return good[a];
  }
  bool is_bipartite(int u) { return good[find(u).first]; }
};
```

2.18 Gomory-Hu Tree

Độ phức tạp: $O(n \cdot \text{MaxFlow})$

Cây cut tương đương cho max-flow/min-cut mọi cặp

```
// Use: auto tree = gomory_hu(3,{{0,1,5},{1,2,7}});
struct CutTreeEdge { int u, v; int capacity; };
// Undirected nonnegative capacities; min edge on tree path gives pairwise min-
    cut.
vector<CutTreeEdge> gomory_hu(int n, const vector<CutTreeEdge>& edges){
  vector<int> parent(n, 0); vector<int> value(n);
  for (int s = 1; s < n; ++s) {
    int t = parent[s]; Dinic flow(n);
    for (auto e : edges) {
      flow.add_edge(e.u,e.v,e.capacity); flow.add_edge(e.v,e.u,e.capacity);
    }
    int cut=flow.max_flow(s,t); auto side=flow.source_side(s);
    for(int i=s+1; i<n; ++i)if(parent[i]==t&&side[i])parent[i]=s;
    if (t != 0 && side[parent[t]]) {
      parent[s]=parent[t]; parent[t]=s; value[s]=value[t]; value[t]=cut;
    } else value[s] = cut;
  }
  vector<CutTreeEdge> tree;
  for (int u = 1; u < n; ++u) tree.push_back( {u, parent[u], value[u]});
  return tree;
}
```

2.19 Critical (Dominator Tree)

Độ phức tạp: $O(m \log n)$

Lengauer-Tarjan: immediate dominator từ nguồn s

```
// Use: auto idom = immediate_dominators({{1},{2},{}},0);
// Immediate dominators from source; source dominates itself; unreachable=-1.
// Lengauer-Tarjan; recursive eval compression, O((n+m) log n) bound.
vector<int>immediate_dominators(const vector<vector<int>>&g,int source){
  int n=g.size(); vector<int>index(n),vertex(1,-1),parent(1),order {source};
  struct Frame { int u,next; }; vector<Frame>stack { {source,0} }; index[source
    ]=1;
  vertex.push_back(source); parent.push_back(0);
  while (!stack.empty()) {
    auto& f = stack.back();
    if (f.next == static_cast<int>(g[f.u].size())) { stack.pop_back(); continue;
     }
    int v = g[f.u][f.next++];
    if (index[v]) continue;
    int p=index[f.u]; index[v]=vertex.size(); vertex.push_back(v); parent.
     push_back(p);
    stack.push_back( {v, 0});
  }
  int count = vertex.size() - 1;
  vector<int>semi(count+1),label(count+1),ancestor(count+1),idom(count+1);
  iota(semi.begin(),semi.end(),0); iota(label.begin(),label.end(),0);
  vector<vector<int>> pred(count + 1), bucket(count + 1);
  for(int u=0;u<n;++u)if(index[u])for(int v:g[u])if(index[v])pred[index[v]].
    push_back(index[u]);
  struct DominatorDSU {
    vector<int>& ancestor; vector<int>& label; vector<int>& semi;
    void compress(int v) {
      if (ancestor[ancestor[v]]) {
        compress(ancestor[v]);
        if (semi[label[ancestor[v]]]<semi[label[v]]) label[v]=label[ancestor[v
     ]];
        ancestor[v]=ancestor[ancestor[v]];
      }
    }
    int eval(int v) {
      if (ancestor[v]) compress(v);
      return label[v];
    }
  } dsu{ancestor,label,semi};
  for (int w = count; w > 1; --w) {
    for (int v : pred[w]) semi[w] = min(semi[w], semi[dsu.eval(v)]);
    bucket[semi[w]].push_back(w); ancestor[w] = parent[w];
    for(int v:bucket[parent[w]]){ int u=dsu.eval(v); idom[v]=semi[u]<semi[v]?u:
     parent[w]; }
    bucket[parent[w]].clear();
  }
  for(int w=2; w<=count; ++w)if(idom[w]!=semi[w])idom[w]=idom[idom[w]];
  vector<int> answer(n, -1); answer[source] = source;
  for (int w = 2; w <= count; ++w) answer[vertex[w]] = vertex[idom[w]];
  return answer;
}
```

2.20 Prufer

Độ phức tạp: $O(n \log n)$

Mã hóa/giải mã cây có nhãn gồm n đỉnh

```
// Use: auto edges = prufer_decode({0,1}); // n = code.size()+2
// Labels 0..n-1; tree n>=2. O(n log n); decode uses n=code.size()+2.
vector<int> prufer_encode(const vector<vector<int>>& g) {
  int n = g.size(); assert(n >= 2); vector<int> degree(n);
  priority_queue<int, vector<int>, greater<int>> leaves;
  for(int u=0; u<n; ++u)if((degree[u]=g[u].size())==1)leaves.push(u);
  vector<int> code;
  for (int i = 0; i < n - 2; ++i) {
    int u = leaves.top(); leaves.pop(); degree[u] = 0;
    for (int v : g[u]) if (degree[v]) {
      code.push_back(v);
      if (--degree[v] == 1) leaves.push(v);
      break;
    }
  }
  return code;
}
vector<pair<int, int>> prufer_decode(const vector<int>& code) {
  int n = code.size() + 2; vector<int> degree(n, 1);
  for (int u : code) { assert(0 <= u && u < n); ++degree[u]; }
  priority_queue<int, vector<int>, greater<int>> leaves;
  for (int u = 0; u < n; ++u) if (degree[u] == 1) leaves.push(u);
  vector<pair<int, int>> edges;
  for (int u : code) {
    int v = leaves.top(); leaves.pop(); edges.push_back( {u, v});
    if (--degree[u] == 1) leaves.push(u);
  }
  int a=leaves.top(); leaves.pop(); edges.push_back( {a,leaves.top()}); return
    edges;
```

```
}
```

2.21 Steiner Tree

Độ phức tạp: $O(n \cdot 3^k + m \cdot 2^k \log m)$

Steiner tree nhỏ nhất nối k đỉnh terminal

```
// Use: auto cost = steiner_tree(g,{0,2});
//      g: vector<vector<pair<int,int>>>, undirected.
// Nonnegative undirected weights; at most 20 terminals (practically <=12).
// Returns nullopt when disconnected, zero for no terminals.
optional<int>steiner_tree(const vector<vector<pair<int,int>>>&g,const vector<int
    >&terminals){
  int n = g.size(), k = terminals.size();
  if (!k) return 0;
  assert(k <= 20); constexpr int inf = 1LL << 60;
  vector<vector<int>> dp(1 << k, vector<int>(n, inf));
  for (int i = 0; i < k; ++i) dp[1 << i][terminals[i]] = 0;
  for (int mask = 1; mask < (1 << k); ++mask) {
    for(int sub=(mask-1)&mask;sub;sub=(sub-1)&mask)if(sub<(mask^sub))for(int u
     =0;u<n;++u)dp[mask][u]=min(dp[mask][u],dp[sub][u]+dp[mask^sub][u]);
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>q;
    for(int u=0; u<n; ++u)if(dp[mask][u]<inf)q.push( {dp[mask][u],u});
    while (!q.empty()) {
      auto [d, u] = q.top(); q.pop();
      if (d != dp[mask][u]) continue;
      for (auto [v, w] : g[u]) if (d + w < dp[mask][v]) {
        dp[mask][v] = d + w; q.push( {d + w, v});
      }
    }
  }
  int answer = *min_element(dp.back().begin(), dp.back().end());
  return answer == inf ? nullopt : optional<int>(answer);
}
```

2.22 Stable Marriage

Độ phức tạp: $O(n^2)$

Gale-Shapley giải bài ghép đôi ổn định (Stable Marriage)

```
// Use: auto wives = stable_marriage({{0,1},{1,0}},{{0,1},{1,0}});
// Complete strict preference permutations, equal side sizes; proposer-optimal.
vector<int>stable_marriage(const vector<vector<int>>&men,const vector<vector<int
    >>&women){
  int n = men.size(); assert(women.size() == men.size());
  vector<vector<int>> rank(n, vector<int>(n));
  for(int w=0; w<n; ++w)for(int i=0; i<n; ++i)rank[w][women[w][i]]=i;
  vector<int> husband(n, -1), wife(n, -1), next(n); queue<int> free;
  for (int m = 0; m < n; ++m) free.push(m);
  while (!free.empty()) {
    int m = free.front(); free.pop(); int w = men[m][next[m]++];
    if (husband[w] < 0 || rank[w][m] < rank[w][husband[w]]) {
      if(husband[w]>=0){ wife[husband[w]]=-1; free.push(husband[w]); }
      husband[w] = m; wife[m] = w;
    } else free.push(m);
  }
  return wife;
}
```

3 Cây

3.1 Tree Isomorphism

Độ phức tạp: $\mathcal{O}(n \log n)$

Nhãn chuẩn hóa chính xác từ các con; so sánh cây không gốc tại tâm

```
// Use: TreeIsomorphism iso; bool same = iso.isomorphic({{1},{0}},
//                                                      {{1},{0}});
// Shared interning dictionary gives exact rooted IDs (not randomized hashes).
struct TreeIsomorphism {
  map<vector<int>, int> ids;
  vector<int> centers(const vector<vector<int>>& g) const {
    int n = g.size();
    if (n < 2) return n ? vector<int> {0}
    :vector<int> {}; vector<int> degree(n), leaves;
    for(int u=0; u<n; ++u)if((degree[u]=g[u].size())<=1)leaves.push_back(u);
    int remaining = n;
    while (remaining > 2) {
      remaining -= leaves.size(); vector<int> next;
      for(int u:leaves)for(int v:g[u])if(--degree[v]==1)next.push_back(v);
      leaves.swap(next);
    }
    return leaves;
  }
  int rooted_id(const vector<vector<int>>& g, int root) {
    vector<int> parent(g.size(), -1), order {root}, id(g.size());
    for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(v!=parent[order[i
    ]]){
      parent[v] = order[i]; order.push_back(v);
    }
    for (auto it = order.rbegin(); it != order.rend(); ++it) {
      int u = *it; vector<int> children;
      for (int v : g[u]) if (v != parent[u]) children.push_back(id[v]);
      sort(children.begin(), children.end());
      auto [pos,inserted]=ids.emplace(children,ids.size()+1); id[u]=pos->second;
    }
    return id[root];
  }
  bool isomorphic(const vector<vector<int>>&a,const vector<vector<int>>&b){
    if (a.size() != b.size()) return false;
    if (a.empty()) return true;
    auto ca = centers(a), cb = centers(b);
    if (ca.size() != cb.size()) return false;
    int id = rooted_id(a, ca[0]);
    for (int c : cb) if (rooted_id(b, c) == id) return true;
    return false;
  }
};
```

3.2 Centroid Decomposition

Độ phức tạp: $\mathcal{O}(n \log n)$

Phân rã cây thành hệ phân cấp sâu $O(\log n)$ để truy vấn đường đi bằng D&C

```
// Use: CentroidDecomposition cd(g,onCentroid);
//      void onCentroid(int c,int parent,const vector<bool>& blocked)
// Builds centroid parent forest for a tree; original adjacency is unchanged.
// on_centroid(c,parent,blocked) may process the current component.
struct CentroidDecomposition {
  vector<int> parent;
  CentroidDecomposition(const vector<vector<int>>& g,
      function<void(int,int,const vector<bool>&)> on_centroid):parent(g.size()
     ,-1){
    int n = g.size();
    if (!n) return;
    vector<bool> blocked(n); vector<int> p(n), size(n);
    vector<pair<int, int>> tasks { {0, -1} };
    while (!tasks.empty()) {
      auto [entry,ancestor]=tasks.back(); tasks.pop_back(); vector<int>order {
     entry};
      p[entry] = -1;
      for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(!blocked[v]&&v
     !=p[order[i]]){
        p[v] = order[i]; order.push_back(v);
      }
      int total = order.size(), c = entry;
      for (auto it = order.rbegin(); it != order.rend(); ++it) {
        int u = *it; size[u] = 1; int largest = 0;
        for (int v : g[u]) if (!blocked[v] && p[v] == u) {
          size[u] += size[v]; largest = max(largest, size[v]);
        }
        if (max(largest, total - size[u]) <= total / 2) c = u;
      }
      parent[c]=ancestor; blocked[c]=true; on_centroid(c,ancestor,blocked);
      for (int v : g[c]) if (!blocked[v]) tasks.push_back( {v, c});
    }
  }
};
```

3.3 Heavy-Light Decomposition

Độ phức tạp: $\mathcal{O}(n \log^2 n)$

Chia cây thành heavy chain: truy vấn/cập nhật đường đi $O(\log^2 n)$, cây con $O(\log n)$

```
// Use: HeavyLight hld({{1},{0,2},{1}},0); auto seg = hld.subtree(1);
//      Store vertex values at hld.pos[u]; seg = [l,r).
// Zero-based positions; segments [l,r). rev=true means traverse r-1 down to l.
// path_segments returns segments in u-to-v order, supports noncommutative folds
    .
struct HeavyLight {
  struct Segment { int l, r; bool reversed; };
  vector<int> parent, depth, size, heavy, head, pos, vertex;
  HeavyLight(const vector<vector<int>>&g,int root=0):parent(g.size(),-1),depth(g
     .size()),size(g.size(),1),heavy(g.size(),-1),head(g.size()),pos(g.size()),
     vertex(g.size()){
    if (g.empty()) return;
    vector<int> order {root};
    for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(v!=parent[order[i
    ]]){
      parent[v]=order[i]; depth[v]=depth[order[i]]+1; order.push_back(v);
    }
    for(auto it=order.rbegin();it!=order.rend();++it)for(int v:g[*it])if(parent[
    v]==*it){
      size[*it] += size[v];
      if (heavy[*it] < 0 || size[v] > size[heavy[*it]]) heavy[*it] = v;
    }
    vector<pair<int, int>> tasks { {root, root} }; int timer = 0;
    while (!tasks.empty()) {
      auto [u, h] = tasks.back(); tasks.pop_back();
      for (; u != -1; u = heavy[u]) {
        head[u] = h; pos[u] = timer; vertex[timer++] = u;
        for(int v:g[u])if(parent[v]==u&&v!=heavy[u])tasks.push_back( {v,v});
      }
    }
  }
  int lca(int u, int v) const {
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]]) swap(u, v);
      u = parent[head[u]];
    }
    return depth[u] < depth[v] ? u : v;
  }
  pair<int,int>subtree(int u)const { return {pos[u],pos[u]+size[u]}; }
  vector<Segment>path_segments(int u,int v,bool edges=false)const {
    vector<Segment> left, right;
    while (head[u] != head[v]) {
      if (depth[head[u]] >= depth[head[v]]) {
        left.push_back( {pos[head[u]],pos[u]+1,true}); u=parent[head[u]];
      }else { right.push_back( {pos[head[v]],pos[v]+1,false}); v=parent[head[v
    ]]; }
    }
    if (depth[u] >= depth[v]) {
      if(pos[v]+edges<pos[u]+1)left.push_back( {pos[v]+edges,pos[u]+1,true});
    } else if(pos[u]+edges<pos[v]+1)right.push_back( {pos[u]+edges,pos[v]+1,
    false});
    reverse(right.begin(), right.end());
    left.insert(left.end(), right.begin(), right.end()); return left;
  }
};
```

4 Chuỗi

4.1 Chiến lược giải bài chuỗi

Phân loại bài chuỗi, chu kỳ, LCP, substring phân biệt, palindrome và định lý chính

- **Problem Taxonomy & Core Approches**:
  - **LCP với toàn chuỗi/tìm một mẫu**: Dùng **Z-Algorithm** hoặc **KMP**.
  - **Tìm nhiều mẫu/DP trên tập chuỗi**: Dùng **Aho-Corasick Automaton**.
  - **Substring phân biệt**: Suffix Array ($\frac{n(n+1)}{2} - \sum LCP_i$) hoặc SAM ($\sum len[u] - len[link[u]]$).
  - **Substring thứ k/LCS của nhiều chuỗi**: Dùng **Suffix Automaton (SAM)** hoặc **Suffix Array + LCP**.

– **Substring đối xứng (palindrome)**: Dùng **Manacher** (bán kính tại mỗi vị trí) hoặc **EERTREE / PAM** (cây palindrome).

– **Phép xoay chuỗi nhỏ nhất theo thứ tự từ điển**: Chạy **Lyndon Factorization (Duval)** trên $S + S$.

– **So sánh substring $O(1)$/chuỗi 2D**: Dùng **Rolling Hash** ($2^{61} - 1$ số nguyên tố Mersenne).

• **Chu kỳ và tính chất border**:

– Chu kỳ nhỏ nhất của chuỗi dài n là $p = n - \pi[n-1]$ (nếu $n \bmod p == 0 \implies S = (S[0..p-1])^{n/p}$).

– **Fine-Wilf (bổ đề chu kỳ)**: Nếu S có chu kỳ p, q và $|S| \geq p + q - \gcd(p, q) \implies S$ có chu kỳ $\gcd(p, q)$.

– **Dãy border**: Mọi border chia thành $\leq O(\log n)$ cấp số cộng $\implies$ Tối ưu DP border xuống $O(n \log n)$.

• **Cận lý thuyết**:

– Số palindrome phân biệt trong chuỗi dài n luôn $\leq n$.

– Suffix Automaton của chuỗi dài n có $\leq 2n - 1$ trạng thái và $\leq 3n - 4$ cạnh chuyển.

4.2 Rolling Hash (61-bit Mersenne)

Độ phức tạp: $O(n)$ dựng, $O(1)$ truy vấn substring

Hash kép/Mersenne 61-bit ($2^{61} - 1$), base chung do người gọi chọn (so sánh có xác suất lỗi)

```
// Use: RollingHash h("abc"); auto hash = h.get(0,2);
struct RollingHash {
  static constexpr int MOD=(static_cast<int>(1)<<61)-1; vector<int>h,p;
  static int mul(int a, int b) {
    __int128 x=(__int128)a*b; int v=(int)(x&MOD)+(int)(x>>61); return v>=MOD?v-
    MOD:v;
  }
  // Use the same base for hashes to be compared; equality is probabilistic.
     Randomize the shared base for adversarial inputs.
  RollingHash(string_view s,int base=911382323):h(s.size()+1),p(s.size()+1,1){
    assert(base > 256 && base < MOD);
    for (size_t i = 0; i < s.size(); ++i) {
      h[i+1]=(mul(h[i],base)+(unsigned char)s[i]+1)%MOD; p[i+1]=mul(p[i],base);
    }
  }
  int get(int l, int r) const {
    assert(0<=l&&l<=r&&r<(int)h.size()); return (h[r]+MOD-mul(h[l],p[r-l]))%MOD;
  }
};
```

4.3 KMP

Độ phức tạp: $O(n + m)$

Prefix function π và tìm mẫu tuyến tính

```
// Use: auto positions = kmp_matches("ababa","aba");
vector<int> prefix_function(string_view s) {
  vector<int> p(s.size());
  for (int i = 1; i < (int) s.size(); ++i) {
    int j = p[i - 1];
    while (j && s[i] != s[j]) j = p[j - 1];
    if (s[i] == s[j]) ++j;
    p[i] = j;
  }
  return p;
}
vector<int> kmp_matches(string_view text, string_view pattern) {
  vector<int> out;
  if (pattern.empty()) {
    for (int i = 0; i <= (int) text.size(); ++i) out.push_back(i);
    return out;
  }
  auto p = prefix_function(pattern); int j = 0;
  for (int i = 0; i < (int) text.size(); ++i) {
    while (j && text[i] != pattern[j]) j = p[j - 1];
    if (text[i] == pattern[j]) ++j;
    if(j==(int)pattern.size()){ out.push_back(i-j+1); j=p[j-1]; }
  }
  return out;
}
```

4.4 Z-Algorithm

Độ phức tạp: $O(n + m)$

Z-array: $z[i] = \text{LCP}(S, S[i..n-1])$ và tìm mẫu qua $S = P + \# + T$

```
// Use: auto z = z_function("ababa");
vector<int> z_function(string_view s) {
  int n = s.size(); vector<int> z(n);
  for (int i = 1, l = 0, r = 0; i < n; ++i) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  if (n) z[0] = n;
  return z;
}
```

4.5 Aho Corasick

Độ phức tạp: $O(26 \sum |P_i| + |T| + \text{matches})$

Tìm nhiều mẫu trên trie, dựng failure/exit link bằng BFS

```
// Use: AhoCorasick ac; ac.add("aba",0); ac.build();
//      ac.scan("ababa",onMatch); // void onMatch(int end,int id)
struct AhoCorasick {
  struct Node {
    array<int, 26> next; int link = 0, exit = -1; vector<int> ids;
    Node() { next.fill(-1); }
  }; vector<Node> t {1}; bool built = false;
  // Lowercase, nonempty patterns; IDs may repeat across different patterns.
  void add(string_view s, int id) {
    assert(!built && !s.empty()); int u = 0;
    for (char c : s) {
      assert('a' <= c && c <= 'z'); int x = c - 'a';
      if(t[u].next[x]<0){ t[u].next[x]=t.size(); t.emplace_back(); }
      u = t[u].next[x];
    }
    t[u].ids.push_back(id);
  }
  void build() {
    assert(!built); queue<int> q;
    for (int c = 0; c < 26; ++c) {
      int v = t[0].next[c];
      if (v < 0) t[0].next[c] = 0;
      else q.push(v);
    }
    while (!q.empty()) {
      int u=q.front(); q.pop(); int f=t[u].link; t[u].exit=t[f].ids.empty()?t[f
    ].exit:f;
      for (int c = 0; c < 26; ++c) {
        int v = t[u].next[c];
        if (v < 0) t[u].next[c] = t[f].next[c];
        else { t[v].link = t[f].next[c]; q.push(v); }
      }
    }
    built = true;
  }
  // emit(end_exclusive, pattern_id); includes overlaps and duplicate patterns.
  void scan(string_view s,function<void(int,int)> emit) const {
    assert(built); int u = 0;
    for (int i = 0; i < (int) s.size(); ++i) {
      assert('a' <= s[i] && s[i] <= 'z'); u = t[u].next[s[i] - 'a'];
      for(int v=u; v>=0; v=t[v].exit)for(int id:t[v].ids)emit(i+1,id);
    }
  }
};
```

4.6 Suffix Array

Độ phức tạp: $O(n \log n)$ dựng, $O(n)$ LCP

Dựng suffix array bằng doubling, tính LCP bằng Kasai

```
// Use: SuffixArray sa("banana"); // sa.sa, sa.lcp
struct SuffixArray {
  vector<int> sa, rank, lcp;
  SuffixArray(string_view s) {
    int n=s.size(),N=n+1; vector<int>a(N),c(N),p(N),cnt(max<int>(N,257));
    for (int i = 0; i < n; ++i) a[i] = (unsigned char) s[i] + 1;
    for (int x : a) ++cnt[x];
    for (int i = 1; i < (int) cnt.size(); ++i) cnt[i] += cnt[i - 1];
    for (int i = N - 1; i >= 0; --i) p[--cnt[a[i]]] = i;
    int classes = 1;
    for (int i = 1; i < N; ++i) {
      if (a[p[i]] != a[p[i - 1]]) ++classes;
      c[p[i]] = classes - 1;
    }
    for (int k = 1; k < N; k *= 2) {
      vector<int> q(N), d(N); fill(cnt.begin(), cnt.end(), 0);
      for(int i=0; i<N; ++i){ q[i]=(p[i]-k+N)%N; ++cnt[c[q[i]]]; }
      for (int i = 1; i < classes; ++i) cnt[i] += cnt[i - 1];
      for (int i = N - 1; i >= 0; --i) p[--cnt[c[q[i]]]] = q[i];
      int nc = 1;
      for (int i = 1; i < N; ++i) {
        if (pair {c[p[i]], c[(p[i] + k) % N]}
        != pair {c[p[i - 1]], c[(p[i - 1] + k) % N]}) ++nc;
        d[p[i]] = nc - 1;
      }
      c.swap(d); classes = nc;
      if (k > N / 2) break;
    }
    sa.assign(p.begin() + 1, p.end()); rank.resize(n); lcp.resize(n);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    for (int i = 0, h = 0; i < n; ++i) {
      int r = rank[i];
      if (!r) { h = 0; continue; }
      int j = sa[r - 1];
      while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
      lcp[r] = h;
      if (h) --h;
    }
  }
};
```

• **Ứng dụng LCP trong thi đấu**:

– **LCP của hai suffix bất kỳ**: $\text{LCP}(S[i..], S[j..]) = \min_{k=\min(pos[i],pos[j])+1}^{\max(pos[i],pos[j])} LCP[k]$ qua RMQ/Sparse Table, truy vấn $O(1)$.

– **Substring phân biệt**: Tổng substring phân biệt là $\frac{n(n+1)}{2} - \sum_{i=1}^{n-1} LCP[i]$.

– **Substring thứ k theo thứ tự từ điển**: Suffix $SA[i]$ đóng góp $n - SA[i] - LCP[i]$ chuỗi mới, có độ dài $LCP[i] + 1 \ldots n - SA[i]$.

– **LCS của hai chuỗi**: Dựng SA trên $S_1 + \# + S_2$; đáp án là $\max LCP[i]$ với $SA[i], SA[i-1]$ thuộc hai chuỗi khác nhau.

– **LCS của K chuỗi**: Dựng SA trên $S_1 + \#_1 + \cdots + S_K + \#_K$; two pointers/sliding window $[L, R]$ chứa đủ K chuỗi $\implies \min_{k=L+1}^{R} LCP[k]$.

– **Khoảng xuất hiện mẫu**: Các suffix có prefix P tạo đoạn liên tiếp $[L, R]$ trong SA; tìm bằng hai binary search trong $O(|P| \log n)$, số lần $= R - L + 1$.

– **Substring dài nhất xuất hiện $\geq K$ lần**: $\max_i(\min_{j=i+1}^{i+K-1} LCP[j])$ bằng monotonic queue/sliding RMQ trong $O(n)$.

4.7 Suffix Automaton

Độ phức tạp: $O(n \log |\Sigma|)$ dựng

DAWG tối tiểu, mỗi đường đi từ gốc ứng với một substring của S

```cpp
// Use: SuffixAutomaton sam("banana");
struct SuffixAutomaton {
  struct State {
    int length=0,link=-1; map<unsigned char,int>next; int ends=0;
  }; vector<State> states {1}; int last = 0;
  void extend(unsigned char c) {
    int cur = states.size(); states.emplace_back();
    states[cur].length=states[last].length+1; states[cur].ends=1; int p=last;
    while(p>=0&&!states[p].next.count(c)){ states[p].next[c]=cur; p=states[p].link; }
    if (p < 0) states[cur].link = 0;
    else {
      int q = states[p].next[c];
      if(states[p].length+1==states[q].length)states[cur].link=q;
      else {
        int clone = states.size(); states.push_back(states[q]);
        states[clone].length=states[p].length+1; states[clone].ends=0;
        while(p>=0&&states[p].next[c]==q){ states[p].next[c]=clone; p=states[p].link; }
        states[q].link = states[cur].link = clone;
      }
    }
    last = cur;
  }
  SuffixAutomaton(string_view s = "") {
    for (unsigned char c : s) extend(c);
  }
  bool contains(string_view s) const {
    int u = 0;
    for (unsigned char c : s) {
      auto it = states[u].next.find(c);
      if (it == states[u].next.end()) return false;
      u = it->second;
    }
    return true;
  }
  int distinct_substrings() const {
    int ans = 0;
    for(int i=1;i<(int)states.size();++i)ans+=states[i].length-states[states[i].link].length;
    return ans;
  }
  // Returned counts do not mutate the automaton; may be recomputed after extensions.
  vector<int> occurrences() const {
    vector<int>order(states.size()); iota(order.begin(),order.end(),0);
    vector<pair<int,int>> lengths;
    for (int id:order) lengths.push_back({states[id].length,id});
    sort(lengths.rbegin(),lengths.rend());
    for (int i=0;i<(int)order.size();++i) { order[i]=lengths[i].second; }
    vector<int> cnt;
    for (auto& s : states) cnt.push_back(s.ends);
    for(int u:order)if(states[u].link>=0)cnt[states[u].link]+=cnt[u];
    return cnt;
  }
};
```

4.8 Lyndon Factorization

Độ phức tạp: $O(n)$ thời gian; các đoạn trả về/chuỗi nhân đôi dùng $O(n)$ bộ nhớ

Duval: phân tích chuỗi thành từ Lyndon $s_1 \geq s_2 \geq \cdots \geq s_k$ và tìm phép xoay nhỏ nhất

```cpp
// Use: int start = minimum_rotation("baca");
vector<pair<int, int>> lyndon_factors(string_view s) {
  vector<pair<int, int>> out; int n = s.size();
  for (int i = 0; i < n;) {
    int j = i + 1, k = i;
    while (j < n && (unsigned char) s[k] <= (unsigned char) s[j]) {
      k = (unsigned char) s[k] < (unsigned char) s[j] ? i : k + 1; ++j;
    }
    while (i <= k) { out.push_back( {i, i + j - k}); i += j - k; }
  }
  return out;
}
int minimum_rotation(string_view s) {
  if (s.empty()) return 0;
  std::string t=std::string(s)+std::string(s); int n=s.size(),ans=0;
  for (int i = 0; i < n;) {
    ans = i; int j = i + 1, k = i;
    while (j < 2 * n && (unsigned char) t[k] <= (unsigned char) t[j]) {
      k = (unsigned char) t[k] < (unsigned char) t[j] ? i : k + 1; ++j;
    }
    while (i <= k) i += j - k;
  }
  return ans;
}
```

4.9 Manacher

Độ phức tạp: $O(n)$

Tìm bán kính palindrome lớn nhất tại mỗi vị trí

```cpp
// Use: auto radii = manacher("ababa"); // radii.odd, radii.even
struct PalindromeRadii { vector<int> odd, even; };
PalindromeRadii manacher(string_view s) {
  int n=s.size(); PalindromeRadii out {vector<int>(n),vector<int>(n)};
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 1 : min(out.odd[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
    out.odd[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
  }
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 0 : min(out.even[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
    out.even[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
  }
  return out;
}
```

4.10 Palindrome Tree

Độ phức tạp: $O(n)$

EERTREE: biểu diễn mọi palindrome phân biệt và các link chuyển

```cpp
// Use: PalindromeTree pt; for (char c : string("aba")) pt.append(c);
//      auto counts = pt.occurrences();
struct PalindromeTree {
  struct Node {
    int length, link; map<unsigned char, int> next; int ends = 0;
  }; vector<Node>nodes { {-1,0,{},0 },{ 0,0,{},0 } }; std::string text; int last=1;
  bool fits(int v,int i,unsigned char c) const {
    int j=i-1-nodes[v].length; return j>=0 && (unsigned char)text[j]==c;
  }
  int append(unsigned char c) {
    text += char(c); int i = text.size() - 1, u = last;

    while (!fits(u,i,c)) u = nodes[u].link;
    auto it = nodes[u].next.find(c);
    if (it != nodes[u].next.end()) last = it->second;
    else {
      int v=nodes.size(),length=nodes[u].length+2; nodes.push_back( { length,1,{},0 });
      nodes[u].next[c] = v;
      if (length > 1) {
        int p = nodes[u].link;
        while (!fits(p,i,c)) p = nodes[p].link;
        nodes[v].link = nodes[p].next.at(c);
      }
      last = v;
    }
    ++nodes[last].ends; return last;
  }
  vector<int> occurrences() const {
    vector<int> cnt;
    for (auto& v : nodes) cnt.push_back(v.ends);
    for(int i=(int)nodes.size()-1; i>=2; --i)cnt[nodes[i].link]+=cnt[i];
    return cnt;
  }
};
```

4.11 De Bruijn (FKM)

Độ phức tạp: $O(k^n)$ thời gian, $O(n)$ bộ nhớ phụ; $O(1)$ khấu hao mỗi ký tự sinh ra

Sinh dãy vòng chứa mỗi từ dài n đúng một lần: nối các từ Lyndon có độ dài là ước của n theo thứ tự từ điển. Dạng tuyến tính: thêm $n-1$ ký tự đầu của dãy vòng.

```cpp
// Use: de_bruijn(2,3,onDigit); // void onDigit(int digit)
void de_bruijn(int k,int n,function<void(int)> emit) {
  assert(k >= 1 && n >= 1);
  if (k == 1) { emit(0); return; }
  vector<int> a(n + 1);
  struct DeBruijnDFS {
    int k,n; vector<int>& a; function<void(int)> emit;
    void go(int t,int p) {
      if (t > n) {
        if (n % p == 0) for (int i = 1; i <= p; ++i) emit(a[i]);
        return;
      }
      a[t] = a[t - p]; go(t + 1, p);
      for (int c = a[t - p] + 1; c < k; ++c) { a[t] = c; go(t + 1, t); }
    }
  } helper{k,n,a,emit}; helper.go(1, 1);
}
```

5 Quy hoạch động (DP)

5.1 Reroot DP

Độ phức tạp: $\mathcal{O}(n)$ cho lượt DFS

Template reroot trên cây: DFS hai lượt, chuyển trạng thái bằng prefix/suffix

```cpp
// Use: auto ans = reroot(g,identity,merge,finish,lift);
//      RerootState: edit state type and write three named functions.
// Associative merge, identity; finish(aggregate,u) adds vertex contribution.
// lift(state,from,to) transforms a complete state across an edge. O(n) callbacks.
using RerootState = pair<int,int>; // Change state for your DP.
vector<RerootState> reroot(const vector<vector<int>>& g,RerootState identity,
    function<RerootState(RerootState,RerootState)> merge,
    function<RerootState(RerootState,int)> finish,
    function<RerootState(RerootState,int,int)> lift,int root=0){
  int n = g.size();
  if (!n) return {};
  vector<int> parent(n, -1), order {root};
  for(size_t i=0;i<order.size();++i)for(int v:g[order[i]])if(v!=parent[order[i]]){
    parent[v] = order[i]; order.push_back(v);
  }
  vector<RerootState> down(n, identity), up(n, identity), answer(n, identity);
  for (auto it = order.rbegin(); it != order.rend(); ++it) {
    int u = *it; RerootState value = identity;
    for(int v:g[u])if(v!=parent[u])value=merge(value,lift(down[v],v,u));
    down[u] = finish(value, u);
  }
  for (int u : order) {
    int degree = g[u].size();
    vector<RerootState>pref(degree+1,identity),suff(degree+1,identity),messages;
    for(int v:g[u])messages.push_back(v==parent[u]?up[u]:lift(down[v],v,u));
    for(int i=0; i<degree; ++i)pref[i+1]=merge(pref[i],messages[i]);
    for(int i=degree-1; i>=0; --i)suff[i]=merge(messages[i],suff[i+1]);
    answer[u] = finish(pref[degree], u);
    for(int i=0;i<degree;++i)if(g[u][i]!=parent[u])up[g[u][i]]=lift(finish(merge(pref[i],suff[i+1]),u),u,g[u][i]);
  }
  return answer;
}
```

5.2 Divide-and-Conquer DP và bất đẳng thức tứ giác

Độ phức tạp: $\mathcal{O}(kn \log n)$ với DnC / $\mathcal{O}(n^2)$ với Knuth

Bất đẳng thức tứ giác (Monge): $\forall a \le b \le c \le d \implies C(a,c) + C(b,d) \le C(a,d) + C(b,c)$ (Cross $\le$ Nest).

- **Kiểm tra cục bộ cần và đủ**: Chỉ cần kiểm tra các ô 2×2 kề nhau (tổng rút gọn):

$$C(i, j+1) - C(i,j) \ge C(i+1, j+1) - C(i+1, j) \quad (\forall i < j)$$

(Tương đương, độ tăng chi phí $\Delta_j(i) = C(i, j+1) - C(i,j)$ không tăng theo i; trong trường hợp liên tục $\frac{\partial^2 C}{\partial x \partial y} \le 0$).

- **Các hàm chi phí thường thỏa QI**:
  - $C(l,r) = f(S_r - S_{l-1})$ với f lồi ($f'' \ge 0$), ví dụ $(S_r - S_{l-1})^2$, $(S_r - S_{l-1})^k$ ($k \ge 1$).
  - $C(l,r) = \sum_{k=l}^{r} w_k = W_r - W_{l-1}$ ($w_k \ge 0$, dấu bằng trong Knuth Optimal BST).
  - $C(l,r) = \text{inv}(l,r)$ (số nghịch thế trong đoạn $[l,r]$, hoặc tính chất trên cặp chéo).
  - $C(l,r) = |x_r - x_l|^p$ ($p \ge 1$ với x đã sắp xếp), hoặc phương sai/tổng bình phương chênh lệch.

- **Đơn điệu**: Tính chất Monge $\implies$ $opt(i_1) \le opt(i_2)$ với $i_1 \le i_2$ (DnC: $O(kn \log n)$) và $opt[i][j-1] \le opt[i][j] \le opt[i+1][j]$ (Knuth: $O(n^2)$).

```cpp
// Use: auto next = divide_conquer_layer(previous,cost);
//      auto ans = knuth_partition(n,cost); // cost(1,r): int
// next[i]=min_{j<i}(previous[j]+cost(j,i)); cost describes [j,i).
// Requires nondecreasing optimal j as i increases. O(n log n) cost calls.
vector<int> divide_conquer_layer(const vector<int>& previous,
    function<int(int,int)> cost,int inf=1LL<<60){
  int n = previous.size() - 1; vector<int> next(n + 1, inf);
  struct LayerSolver {
    const vector<int>& previous; vector<int>& next;
    function<int(int,int)> cost; int inf;
    void solve(int l,int r,int lo,int hi) {
      if (l > r) return;
      int mid = (l + r) / 2, best = lo;
      for (int j = lo; j <= min(mid - 1, hi); ++j) if (previous[j] < inf){
        int value = previous[j] + cost(j, mid);
        if (value < next[mid]) { next[mid] = value; best = j; }
      }
      solve(l, mid - 1, lo, best); solve(mid + 1, r, best, hi);
  }
  } helper{previous,next,cost,inf};
  if (n) helper.solve(1, n, 0, n - 1);
  return next;
}
// Interval DP: split [l,r) into two nonempty intervals and add cost(l,r).
// Requires Knuth inequalities; O(n^2) time/space. Singleton cost is zero.
int knuth_partition(int n,function<int(int,int)> cost) {
  if (!n) return 0;
  vector<vector<int>> dp(n, vector<int>(n));
  vector<vector<int>> opt(n, vector<int>(n));
  for (int i = 0; i < n; ++i) opt[i][i] = i;
  for (int len = 2; len <= n; ++len) for (int l = 0; l + len <= n; ++l){
    int r = l + len - 1; dp[l][r] = 1LL << 60;
    for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r]); ++k) {
      int value = dp[l][k] + dp[k + 1][r] + cost(l, r + 1);
      if (value < dp[l][r]) { dp[l][r] = value; opt[l][r] = k; }
    }
  }
  return dp[0][n - 1];
}
```

5.3 Tối ưu DP 1D/1D

Độ phức tạp: $\mathcal{O}(n \log n)$

Tối ưu $dp[i] = \min_{j<i}(dp[j] + cost(j,i))$ thỏa bất đẳng thức tứ giác bằng deque các khoảng tối ưu

```cpp
// Use: auto dp = monotone_partition_dp(n,cost); // cost(j,i): int
// dp[i]=min_{j<i}(dp[j]+cost(j,i)), dp[0]=initial.
// Requires single crossing: a newer candidate wins a suffix of future indices.
vector<int> monotone_partition_dp(int n,
    function<int(int,int)> cost,int initial=0){
  struct Candidate { int j,l,r; }; deque<Candidate>q; vector<int>dp(n+1);
  dp[0] = initial;
  if (n) q.push_back( {0, 1, n});

  for (int i = 1; i <= n; ++i) {
    while (q.front().r < i) q.pop_front();
    dp[i] = (dp[q.front().j]+cost(q.front().j,i));
    if (i == n) break;
    if (q.front().r == i) q.pop_front();
    else q.front().l = i + 1;
    while(!q.empty()&&(dp[i]+cost(i,q.back().l))<=(dp[q.back().j]+cost(q.back().
     j,q.back().l)))q.pop_back();
    if (q.empty()) { q.push_back( {i, i + 1, n}); continue; }
    int l = q.back().l, r = q.back().r + 1;
    while (l < r) {
      int mid = (l + r) / 2;
      if ((dp[i] + cost(i,mid)) <= (dp[q.back().j]+cost(q.back().j,mid))) r =
     mid;
      else l = mid + 1;
    }
    if (l <= n) { q.back().r = l - 1; q.push_back( {i, l, n}); }
  }
  return dp;
}
```

5.4 Alien DP

Độ phức tạp: $\mathcal{O}(n \log(\max_val))$

DP trên hàm lồi/lõm (WQS Binary Search):

- Chi phí lồi/lõm $\rightarrow$ Chọn greedy.
- DP lồi/lõm $\rightarrow$ Binary search hệ số phạt λ.

```cpp
// Use: auto ans = aliens_exact(k,lo,hi,oracle);
//      oracle(lambda): PenalizedResult{minCost, count}.
struct PenalizedResult { int value; int count; };
// oracle(lambda) minimizes cost-lambda*count, breaking ties toward MORE count.
// Exact-count optimum must be discrete convex, attainable; lo/hi bracket count=
    k.
int aliens_exact(int k,int lo,int hi,
    function<PenalizedResult(int)> oracle){
  assert(lo <= hi && oracle(hi).count >= k);
  while (lo < hi) {
    int mid=lo+static_cast<int>((__int128(hi)-lo)/2);
    if (oracle(mid).count >= k) hi = mid;
    else lo = mid + 1;
  }
  auto result = oracle(lo); return result.value + lo * k;
}
```

5.5 Convex Hull Trick

Độ phức tạp: $\mathcal{O}(n \log n)$ hoặc $\mathcal{O}(n)$ khi hệ số góc đơn điệu

Tối ưu DP với hàm tuyến tính $y = mx + c$

```cpp
// Use: MonotoneMinHull cht; cht.add(3,1); auto y = cht.query(2);
// Add nonincreasing slopes; query nondecreasing x. Products fit int128;
// coefficients fit int64 and their products/differences fit signed int128.
struct MonotoneMinHull {
  struct Line {
    int m, b;
    __int128 at(int x) const { return __int128(m) * x + b; }
  }; deque<Line> q;
  void add(int m, int b) {
    Line c {m, b};
    if (!q.empty() && q.back().m == m) {
      if (q.back().b <= b) return;
      q.pop_back();
    }
    while (q.size() > 1) {
      auto a = q[q.size() - 2], z = q.back();
      if((__int128(z.b)-a.b)*(__int128(z.m)-c.m)<(__int128(c.b)-z.b)*(__int128(a
     .m)-z.m))break;
      q.pop_back();
    }
    q.push_back(c);
  }
  __int128 query(int x) {
    assert(!q.empty());
    while (q.size() > 1 && q[0].at(x) >= q[1].at(x)) q.pop_front();
    return q.front().at(x);
  }
};
```

5.6 Slope Trick

Độ phức tạp: $\mathcal{O}(n \log n)$

Duy trì hàm lồi tuyến tính từng đoạn liên tục $f(x)$: cộng $|x-a|$, prefix min, dịch miền bằng hai heap

```cpp
// Use: SlopeTrick f; f.add_abs(3); auto ans = f.minimum();
// f starts at 0. Represents convex piecewise-linear f, slopes integral.
struct SlopeTrick {
  priority_queue<int> left;
  priority_queue<int,vector<int>,greater<int>>right;
  int lshift = 0, rshift = 0, value = 0;
  int minimum() const { return value; }
  // Minimizers [left endpoint,right endpoint]; missing endpoints mean infinity.
  pair<optional<int>, optional<int>> minimizers() const {
    return {
      left.empty()?nullopt:optional<int>(left.top()+lshift),right.empty()?
     nullopt:optional<int>(right.top()+rshift)
    };
  }
  void add_x_minus_a(int a) {
    if (!left.empty() && left.top() + lshift > a) {
      int x=left.top()+lshift; left.pop(); value+=x-a; left.push(a-lshift);
      right.push(x - rshift);
    } else right.push(a - rshift);
  }
  void add_a_minus_x(int a) {
    if (!right.empty() && right.top() + rshift < a) {
      int x=right.top()+rshift; right.pop(); value+=a-x; right.push(a-rshift);
      left.push(x - lshift);
    } else left.push(a - lshift);
  }
  void add_abs(int a) { add_x_minus_a(a); add_a_minus_x(a); }
  void add_constant(int c) { value += c; }
  void prefix_min() { right = {}; }
  // g(x)=min_{y<=x}f(y).
  void suffix_min() { left = {}; }
  // g(x)=min_{x-b<=y<=x+a} f(y), a,b>=0.
  void window_min(int a, int b) {
    assert(a >= 0 && b >= 0); lshift -= a; rshift += b;
  }
};
```

5.7 Bitmask DP

- **Matching**: Tập cặp không có thứ tự $\rightarrow$ cố định một đầu, chọn đầu còn lại. $O(n2^n)$ thời gian, $O(2^n)$ bộ nhớ.
- **Nén trạng thái**: $i = \text{popcount}(mask) \rightarrow dp[i][mask] \rightarrow dp[mask]$.

5.8 SOS DP

Độ phức tạp: $\mathcal{O}(N \cdot 2^N)$

Sum Over Subsets / Fast Zeta Transform: tính tổng trên submask/supermask trong $O(N \cdot 2^N)$ ($N \le 24$)

```cpp
// Use: vector<int> f(1<<3,1); subset_transform(f);
// In-place subset/superset zeta transform; inverse needs subtraction.
```

```cpp
void subset_transform(vector<int>&a,bool supersets=false,bool inverse=false){
  size_t n = a.size(); assert(n && (n & (n - 1)) == 0);
  for(size_t bit=1;bit<n;bit<<=1)for(size_t mask=0;mask<n;++mask)if(bool(mask&
    bit)!=supersets){
    int value = a[mask ^ bit];
    if (inverse) a[mask] -= value;
    else a[mask] += value;
  }
}
```

5.9 Tối ưu bằng monotonic queue

Độ phức tạp: $O(1)$ khấu hao mỗi push/pop

Deque chỉ số tăng dần, tìm min trong sliding window

```cpp
// Use: MonotoneMinQueue q; q.push(0,5); q.expire(0);
// Push strictly increasing indices; expire indices < first_valid.
struct MonotoneMinQueue {
  deque<pair<int, int>> q;
  void push(int index, int value) {
    while (!q.empty() && q.back().second >= value) q.pop_back();
    q.push_back( {index, value});
  }
  void expire(int first_valid) {
    while (!q.empty() && q.front().first < first_valid) q.pop_front();
  }
  bool empty() const { return q.empty(); }
  const int&minimum()const { assert(!q.empty()); return q.front().second; }
};
```

5.10 Expected Value Trick

Độ phức tạp: $\mathcal{O}(n\sqrt{n})$

Chọn tối ưu k trong n phần tử đã xáo trộn: bước i dùng heuristic dải ngẫu nhiên (có thể bỏ lỡ tối ưu), chỉ xét trạng thái trong $\left[\frac{i\cdot k}{n} - C\sqrt{n}, \frac{i\cdot k}{n} + C\sqrt{n}\right]$.

```cpp
// Use: mt19937_64 rng(123456);
//      auto ans = banded_pick_k(vector<int>{1,2,3},2,3,rng);
// Randomized band DP for additive pick-k; not a deterministic exact guarantee.
// Shuffles a COPY; caller provides half-width and RNG. O(n*(width+1)) work, O(
    width+1) space.
// nullopt means no feasible path stayed in the band; reaching k may still be
    suboptimal.
optional<int> banded_pick_k(vector<int> values,
    int k,int width,mt19937_64& rng){
  int n = values.size(); assert(0 <= k && k <= n && width >= 0);
  if (!n) return 0;
  shuffle(values.begin(),values.end(),rng); constexpr int neg=-(1LL<<60);
  vector<int> dp(1, 0); int prev_lo = 0, prev_hi = 0;
  for (int i = 1; i <= n; ++i) {
    int center = static_cast<int>(i) * k / n;
    int lo=max<int>( {0,center-width,k-(n-i)}),hi=min( {i,k,center+width});
    if (lo > hi) return nullopt;
    vector<int> next(hi - lo + 1, neg);
    for (int j = lo; j <= hi; ++j) {
      if (prev_lo <= j && j <= prev_hi) next[j - lo] = dp[j - prev_lo];
      if(j&&prev_lo<=j-1&&j-1<=prev_hi&&dp[j-1-prev_lo]!=neg)next[j-lo]=max(next
    [j-lo],dp[j-1-prev_lo]+values[i-1]);
    }
    dp.swap(next); prev_lo = lo; prev_hi = hi;
  }
  return dp[k-prev_lo]==neg?nullopt:optional<int>(dp[k-prev_lo]);
}
```

6 Toán học

6.1 Công thức số học

Số học modulo, rút gọn số mũ Euler mở rộng, Legendre, Wilson, Frobenius Coin, Floor Trick, CRT, Fibonacci

- **Rút gọn số mũ Euler mở rộng** $(\forall a, m)$:

$$a^b \equiv a^{(b \bmod \phi(m))+\phi(m)} \pmod{m} \quad (b \geq \phi(m))$$

- **Công thức Legendre** $(v_p(n!))$: $v_p(n!) = \sum_{k=1}^{\infty} \lfloor \frac{n}{p^k} \rfloor = \frac{n - S_p(n)}{p-1}$.
- **LTE**: $v_p(x)$ là số mũ của nguyên tố p trong $x \neq 0$. Với a, b, n nguyên dương, $a \neq b$, $n \geq 1$: nếu p lẻ, $p \mid a-b$, $p \nmid ab$, thì $v_p(a^n - b^n) = v_p(a-b) + v_p(n)$. Nếu p lẻ, $p \mid a+b$, $p \nmid ab$, n lẻ: $v_p(a^n + b^n) = v_p(a+b) + v_p(n)$. Với a, b lẻ và $p = 2$: n lẻ cho $v_2(a^n - b^n) = v_2(a-b)$; n chẵn cho $v_2(a^n - b^n) = v_2(a-b) + v_2(a+b) + v_2(n) - 1$. Dùng tính mức chia hết của hiệu/tổng lũy thừa mà không tính số lớn.
- **Định lý Wilson**: $(p-1)! \equiv -1 \pmod{p} \iff p$ nguyên tố $\implies (p-2)! \equiv 1 \pmod{p}$.
- **Frobenius Coin (Chicken McNugget)**: Với $\gcd(a, b) = 1$, số lớn nhất không biểu diễn được dạng $ax + by$ $(x, y \geq 0)$ là $ab - a - b$; số lượng không biểu diễn được là $\frac{(a-1)(b-1)}{2}$.
- **Floor Trick** $(O(\sqrt{n})$ **nhóm)**:

```cpp
for (int l = 1, r; l <= n; l = r + 1) {
    int v = n / l; r = n / v;
}
```

- **Định lý số dư Trung Hoa (CRT)**: $x \equiv a_i \pmod{m_i}$.

$$x \equiv \sum_{i=1}^{k} a_i M_i (M_i^{-1} \bmod m_i) \pmod{M} \quad (M = \prod m_i, M_i = M/m_i)$$

- **Fibonacci**: $\gcd(F_m, F_n) = F_{\gcd(m,n)}$, $F_{2n} = F_n(2F_{n+1} - F_n)$.

6.2 Like-Euclid Floor Sum

Độ phức tạp: $O(\log m)$

Tính $\sum_{i=0}^{n-1} \lfloor \frac{ai+b}{m} \rfloor$ bằng đổi tọa độ

```cpp
// Use: auto s = floor_sum(10,7,3,2); // sum floor((3*i+2)/7)
// Nonnegative n,a,b; m>0. All intermediates and the result must fit signed 128
    bits.
__int128 floor_sum(int n, int m, int a, int b) {
  assert(n >= 0 && a >= 0 && b >= 0 && m > 0); __int128 ans = 0;
  for (;;) {
    if (a >= m) { ans += (__int128) n * (n - 1) / 2 * (a / m); a %= m; }
    if (b >= m) { ans += (__int128) n * (b / m); b %= m; }
    __int128 y = (__int128) a * n + b;
    if (y < m) break;
    n = y / m; b = y % m; swap(m, a);
  }
  return ans;
}
```

6.3 Rabin-Miller

Độ phức tạp: $O(k \log n)$

Miller-Rabin tất định: 3 cơ sở khi $n < 2^{32}$, 7 cơ sở khi $n < 2^{63}$; dùng `int long long`

```cpp
// Dung int; them #define int long long neu can so 64-bit.
const int base_prime[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
const int rb_seed32[] = {2, 7, 61};
const int rb_seed[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
int mul_mod(int a, int b, int m) {
  return (__int128) a * b % m;
}
int pow_mod(int a, int e, int m) {
  assert(a >= 0 && e >= 0 && m > 0);
  int r = 1 % m;
  a %= m;
  while (e) {
    if (e & 1) r = mul_mod(r, a, m);
    a = mul_mod(a, a, m);
    e >>= 1;
  }
  return r;
}
bool is_prime(int n) {
  if (n < 2) return false;
  for (int p : base_prime) if (n % p == 0) return n == p;
  int d = n - 1, s = 0;
  while (d % 2 == 0) d >>= 1, ++s;
  const int *bases = n < (1LL << 32) ? rb_seed32 : rb_seed;
  int count = n < (1LL << 32) ? 3 : 7;
  for (int i = 0; i < count; ++i) {
    int a = bases[i];
    if (a % n == 0) continue;
    int x = pow_mod(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool pass = false;
    for (int r = 1; r < s; ++r) {
      x = mul_mod(x, x, n);
      if (x == n - 1) { pass = true; break; }
    }
    if (!pass) return false;
  }
  return true;
}
```

6.4 Sàng phân đoạn (Segmented Sieve)

Độ phức tạp: $O(\sqrt{R}\log\log R + W\log\log R + \lceil W/B \rceil \pi(\sqrt{R}))$

Lấy tối đa k nguyên tố trong $[L, R]$, $R \leq 10^9$; $W = R - L + 1$, B số/block; bộ nhớ $O(\sqrt{R} + B + k)$

```cpp
// Use: auto primes = segmented_sieve(2,1000000000,5000000);
// Returns up to k primes in [L,R], in increasing order; R <= 1e9.
const int SIEVE_BLOCK = 1000000;
vector<int> segmented_sieve(int L,int R,int k = INT_MAX) {
  assert(0 <= L && L <= R && R <= 1000000000 && k >= 0);
  vector<int> base, primes;
  if (R < 2 || k == 0) return primes;
  int root = sqrtl(R);
  while ((root+1)*(root+1) <= R) ++root;
  vector<bool> small(root+1);
  for (int i = 2; i <= root; ++i) if (!small[i]) {
    base.push_back(i);
    for (int j = i*i; j <= root; j += i) small[j] = true;
  }
  bitset<SIEVE_BLOCK> composite;
  for (int lo = max<int>(L,2); lo <= R; lo += SIEVE_BLOCK) {
    int hi = min(R,lo+SIEVE_BLOCK-1);
    composite.reset();
    for (int p : base) {
      if (p*p > hi) break;
      int start = max(p*p,((lo+p-1)/p)*p);
      for (int j = start; j <= hi; j += p) composite[j-lo] = true;
    }
    for (int x = lo; x <= hi; ++x) if (!composite[x-lo]) {
      primes.push_back(x);
      if ((int)primes.size() == k) return primes;
    }
  }
  return primes;
}
```

6.5 Đếm nguyên tố (Prime Counting)

Độ phức tạp: $O(N^{3/4})$ thời gian, $O(\sqrt{N})$ bộ nhớ

Sàng theo các giá trị thương: đếm nguyên tố $\leq N$

```cpp
// Use: auto count = prime_count(1000000);
// Quotient-value sieve: O(N^(3/4)/log N) typical bound, O(sqrt N) storage.
int prime_count(int n) {
  if (n < 2) return 0;
  int r = sqrtl(n);
  while ((__int128)(r + 1) * (r + 1) <= n) ++r;
  while ((__int128) r * r > n) --r;
```

```
    vector<int> v, g;
    for (int l = 1; l <= n;) {
      int x=n/l; v.push_back(x); g.push_back(x-1); int next=n/x;
      if (next == n) break;
      l = next + 1;
    }
    struct QuotientIndex {
      int n,r; size_t size;
      size_t get(int x) { return x<=r?size-x:n/x-1; }
    } index{n,r,v.size()};
    for (int p = 2; p <= r; ++p) {
      if (g[index.get(p)] == g[index.get(p - 1)]) continue;
      int before = g[index.get(p - 1)];
      for(size_t i=0;i<v.size()&&v[i]>=p*p;++i)g[i]-=g[index.get(v[i]/p)]-before;
    }
    return g[0];
  }
```

6.6 Pollard Rho

Độ phức tạp: $O(\sqrt[4]{n}\log n)$

Tìm ước không tầm thường của hợp số bằng phát hiện chu trình Brent

```
  // Use: auto factors = factorize(360);
  inline mt19937 rho_rng(chrono::steady_clock::now().time_since_epoch().count());
  int rho_step(int v,int c,int n) {
    return static_cast<int>((__int128(mul_mod(v,v,n))+c)%n);
  }
  int rho_factor(int n, mt19937 & rng) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    for (;;) {
      uniform_int_distribution<int> random(1,n-1);
      int c=random(rng),y=random(rng),g=1,r=1,x=0,ys=0;

      while (g == 1) {
        x = y;
        for (int i = 0; i < r; ++i) y = rho_step(y,c,n);
        for (int k = 0; k < r && g == 1; k += 128) {
          ys = y; int q = 1;
          for(int i=0;i<min<int>(128,r-k);++i){ y=rho_step(y,c,n); q=mul_mod(q,x>y
     ?x-y:y-x,n); }
          g = gcd(q, n);
        }
        r *= 2;
      }
      if (g == n) {
        do { ys = rho_step(ys,c,n); g = gcd(x > ys ? x - ys : ys - x, n); }
        while (g == 1);
      }
      if (g < n) return g;
    }
  }
  void factor_into(int n,mt19937& rng,vector<int>& out) {
    if (n==1) return;
    if (is_prime(n)) { out.push_back(n); return; }
    int d=rho_factor(n,rng); factor_into(d,rng,out); factor_into(n/d,rng,out);
  }
  vector<int> factorize(int n,mt19937& rng) {
    assert(n>=1); vector<int> out; factor_into(n,rng,out);
    sort(out.begin(),out.end()); return out;
  }
  vector<int> factorize(int n) { return factorize(n,rho_rng); }
```

6.7 Primitive Root

Độ phức tạp: $\mathcal{O}(\text{ans}\cdot\log p)$

Tìm căn nguyên thủy g nhỏ nhất modulo nguyên tố p

```
  // Use: mt19937 rng(123456); auto g = primitive_root(17,rng);
  int primitive_root(int p, mt19937 & rng) {
    assert(is_prime(p));
    if (p == 2) return 1;
    auto f=factorize(p-1,rng); f.erase(unique(f.begin(),f.end()),f.end());
    for (int g = 2; g < p; ++g) {
      bool ok = true;
      for (int q : f) if (pow_mod(g, (p - 1) / q, p) == 1) {
        ok = false; break;
      }
      if (ok) return g;
    }
    throw logic_error("prime required");
  }
```

6.8 Baby-step Giant-Step

Độ phức tạp: $O(\sqrt{m}\log m)$

Tìm x nhỏ nhất thỏa $a^x \equiv b \pmod m$ (Discrete Log)

```
  // Use: auto exponent = discrete_log(2,8,13);
  // Smallest exponent, including non-coprime a,m. Practical m limited by sqrt(m)
      memory.
  optional<int> discrete_log(int a, int b, int m) {
    assert(a >= 0 && b >= 0 && m > 0); a %= m; b %= m;
    if (b == 1 % m) return 0;
    int k = 1 % m, offset = 0;
    for (int g; (g = gcd(a, m)) > 1;) {
      if (b == k) return offset;
      if (b % g) return nullopt;
      b /= g; m /= g; k = mul_mod(k, a / g, m); ++offset;
    }
    int n = sqrtl(m) + 1; unordered_map<int, int> baby; int x = b;
    for (int q = 0; q <= n; ++q) { baby[x] = q; x = mul_mod(x, a, m); }
    int step = pow_mod(a, n, m); x = k;
    for (int p = 1; p <= n + 1; ++p) {
      x = mul_mod(x, step, m); auto it = baby.find(x);
      if (it != baby.end()) return p * n - it->second + offset;
    }
    return nullopt;
  }
```

6.9 Căn modulo (Tonelli-Shanks)

Độ phức tạp: $O(\log^2 p)$

Tonelli-Shanks: giải $x^2 \equiv a \pmod p$

```
  // Use: auto root = sqrt_mod(4,17);
  optional<int> sqrt_mod(int a, int p) {
    assert(a >= 0 && is_prime(p)); a %= p;
    if (p == 2 || a == 0) return a;
    if (pow_mod(a, (p - 1) / 2, p) != 1) return nullopt;
    if(p%4==3){ int x=pow_mod(a,p/4+1,p); return min(x,p-x); }
    int q = p - 1; int s = 0;
    while (!(q & 1)) q >>= 1, ++s;
    int z = 2;
    while (pow_mod(z, (p - 1) / 2, p) != p - 1) ++z;
    int c=pow_mod(z,q,p),x=pow_mod(a,(q+1)/2,p),t=pow_mod(a,q,p);
    while (t != 1) {
      int i = 0; int u = t;
      while (u != 1) { u = mul_mod(u, u, p); ++i; }
      int b=pow_mod(c,static_cast<int>(1)<<(s-i-1),p); x=mul_mod(x,b,p); c=mul_mod
      (b,b,p);
      t = mul_mod(t, c, p); s = i;
    }
    return min(x, p - x);
  }
```

Bézout (tổng quát): Với $a_1,\ldots,a_n \in \mathbb{Z}$ không đồng thời bằng 0, $g=\gcd(|a_1|,\ldots,|a_n|)$: tồn tại $c_i\in\mathbb{Z}$ sao cho $\sum_i c_i a_i = g$. Mọi tổ hợp nguyên tạo đúng các bội của g: $\sum_i c_i a_i = x$ có nghiệm iff $g \mid x$. **Cue**: cộng/trừ các bước tùy ý → gcd các bước chia hết độ dịch chuyển; nghịch đảo modulo → $ax+my=1$ (cần $\gcd(a,m)=1$). Hệ số được âm; yêu cầu $c_i\ge 0$ thì gcd chia hết chưa đủ. Extended GCD tìm hệ số cho hai số, ghép dần cho nhiều số.

GCD: cộng/trừ và đổi mốc

- $\gcd(u,v)=\gcd(u,v+ku)=\gcd(v,u \bmod v)$ ($k\in\mathbb{Z}$, $v\neq 0$ cho modulo); $\gcd(cu,cv)=|c|\gcd(u,v)$. Cộng/trừ bội giữ các ước chung.
- Đặt $d_i=a_{i+1}-a_i$, $g=\gcd|d_i|$: $\gcd(a_1,\ldots,a_n)=\gcd(a_1,g)$; tịnh tiến: $\gcd(a_1+b,\ldots,a_n+b)=\gcd(a_1+b,g)$ (Row GCD). Phải giữ mốc; $\gcd(z,0)=|z|$.
- $\gcd(pu+qv,ru+sv)=\gcd(u,v)$ nếu $ps-qr=\pm 1$ (phép ngược có hệ số nguyên). Tổ hợp tùy ý chưa chắc giữ gcd, vd. $(u+v,u-v)$.

6.10 Extended GCD

Độ phức tạp: $\mathcal{O}(\log\min(a,b))$

Tìm x,y thỏa phương trình $ax+by=\gcd(a,b)$

```
  // Use: auto b = extended_gcd(12,18); // b.gcd, b.x, b.y
  struct Bezout { __int128 gcd, x, y; };
  Bezout extended_gcd(int a, int b) {
    __int128 r = a, s = b, x = 1, u = 0, y = 0, v = 1;
    while (s) {
      __int128 q=r/s; tie(r,s)=pair {s,r-q*s}; tie(x,u)=pair {u,x-q*u};
      tie(y, v) = pair {v, y - q * v};
    }
    if (r < 0) r = -r, x = -x, y = -y;
    return {r, x, y};
  }
```

6.11 Diophantine

Độ phức tạp: $O(\log\min(a,b))$

Nghiệm tổng quát của phương trình Diophantine tuyến tính $ax+by=c$

```
  // Use: auto sol = diophantine(3,5,7);
  struct DiophantineSolution { bool all_pairs; __int128 x, y, dx, dy; };
  // Every solution is (x+t*dx,y+t*dy), except all_pairs for 0x+0y=0.
  optional<DiophantineSolution>diophantine(int a,int b,int c){
    auto z = extended_gcd(a, b);
    if (!z.gcd) {
      if (c) return nullopt;
      return DiophantineSolution {true, 0, 0, 0, 0};
    }
    if (c % z.gcd) return nullopt;
    return DiophantineSolution {
      false,z.x*(c/z.gcd),z.y*(c/z.gcd),b/z.gcd,-(__int128)a/z.gcd
    };
  }
```

6.12 Stern-Brocot Tree

Độ phức tạp: $\mathcal{O}(\log(p+q))$

Phân số tối giản, mã hóa/giải mã cây, LCA

```
  // Use: auto path = stern_encode(3,5); auto fraction = stern_decode(path);
  using SternPath = vector<pair<char, int>>;
  SternPath stern_encode(int p, int q) {
    assert(p > 0 && q > 0 && gcd(p, q) == 1); SternPath out;
    while (p != q) {
      if (p < q) {
        int k = (q - 1) / p; out.push_back( {'L', k}); q -= k * p;
      } else { int k=(p-1)/q; out.push_back( {'R',k}); p-=k*q; }
    }
    return out;
  }
  // Fractions and intermediate bounds must fit signed 128 bits.
  pair<__int128, __int128> stern_decode(const SternPath& path) {
    __int128 a = 0, b = 1, c = 1, d = 0;
    for (auto[dir, k] : path) {
      assert(k > 0 && (dir == 'L' || dir == 'R'));
      if (dir == 'L') c += k * a, d += k * b;
      else a += k * c, b += k * d;
    }
    return {a + c, b + d};
  }
  SternPath stern_lca(const SternPath& a, const SternPath& b) {
    SternPath out;
    for(size_t i=0; i<min(a.size(),b.size())&&a[i].first==b[i].first; ++i){
      out.push_back( {a[i].first, min(a[i].second, b[i].second)});
      if (a[i].second != b[i].second) break;
    }
    return out;
  }
```

6.13 Hàm nhân tính và Möbius Inversion

Tích chập Dirichlet, nghịch đảo Möbius, tổng GCD/LCM

- **Euler Phi & Möbius**: $\phi(n) = n\prod_{p|n}(1-1/p)$, $\sum_{d|n}\phi(d) = n$, $\sum_{d|n}\mu(d) = [n=1]$.
- **Dirichlet & Inversion**: $(f*g)(n) = \sum_{d|n} f(d)g(n/d)$.

$$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(d) f(n/d)$$

- **Đếm cặp nguyên tố cùng nhau**:

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1] = \sum_{d=1}^{\min(n,m)} \mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

- **Tổng GCD trên mọi cặp**:

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j) = \sum_{g=1}^{\min(n,m)} \phi(g)\left\lfloor\frac{n}{g}\right\rfloor\left\lfloor\frac{m}{g}\right\rfloor$$

- **Tổng LCM trên mọi cặp** ($S(x) = \frac{x(x+1)}{2}$):

$$\sum_{i\le n, j\le m}\operatorname{lcm}(i,j) = \sum_g g\sum_d d^2\mu(d) S\left(\left\lfloor\frac{n}{gd}\right\rfloor\right) S\left(\left\lfloor\frac{m}{gd}\right\rfloor\right)$$

6.14 Công thức tổ hợp

Đẳng thức nhị thức, Hockey-stick, Vandermonde, Inversion, Stars and Bars, Catalan, Stirling, Cayley, LGV

- **Đẳng thức nhị thức**: $\binom{n}{k} = \binom{n}{n-k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $k\binom{n}{k} = n\binom{n-1}{k-1}$; $\frac{1}{k+1}\binom{n}{k} = \frac{1}{n+1}\binom{n+1}{k+1}$; $\binom{n}{k}\binom{k}{m} = \binom{n}{m}\binom{n-m}{k-m}$; Đối số trên âm: $\binom{-n}{k} = (-1)^k\binom{n+k-1}{k}$.
- **Tổng nhị thức**: $\sum_{k=0}^{n}\binom{n}{k} = 2^n$; $\sum_{k=0}^{n}(-1)^k\binom{n}{k} = [n=0]$; $\sum_{k\text{ chẵn}}\binom{n}{k} = \sum_{k\text{ lẻ}}\binom{n}{k} = 2^{n-1}$; $\sum_{k=0}^{n}k\binom{n}{k} = n2^{n-1}$; $\sum_{k=0}^{n}k^2\binom{n}{k} = n(n+1)2^{n-2}$; $\sum_{k=0}^{n}\frac{1}{k+1}\binom{n}{k} = \frac{2^{n+1}-1}{n+1}$.
- **Hockey-Stick & Vandermonde**: $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$; $\sum_{i=0}^{k}\binom{n+i}{i} = \binom{n+k+1}{k}$; $\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r} \implies \sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}$.
- **Binomial Inversion và sai phân**: $f(n) = \sum_{k=0}^{n}\binom{n}{k}g(k) \iff g(n) = \sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}f(k)$; Dạng tổng phía trên: $f(n) = \sum_{k=n}^{N}\binom{k}{n}g(k) \iff g(n) = \sum_{k=n}^{N}(-1)^{k-n}\binom{k}{n}f(k)$; $\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}k^m = n!\left\{{m \atop n}\right\}$ (0 nếu $m<n$, $n!$ nếu $m=n$).
- **Stars and Bars**: $x_i \ge 0 \implies \binom{n+k-1}{k-1}$; $x_i \ge 1 \implies \binom{n-1}{k-1}$; $x_i \ge c_i \implies n' = n - \sum c_i$.
- **Derangements**: $D_0 = 1, D_1 = 0, D_n = (n-1)(D_{n-1}+D_{n-2}) = nD_{n-1} + (-1)^n \approx [n!/e]$.
- **Số Catalan** $C_n = \frac{1}{n+1}\binom{2n}{n}$: Dãy ngoặc đúng, cây nhị phân đầy đủ có $n+1$ lá, chia tam giác đa giác lồi $(n+2)$ đỉnh, đường Dyck không vượt $y=x$, hoán vị sinh bằng một stack.
- **Nguyên lý phản xạ (Reflection Principle)**: Đường đi $(0,0) \to (n,m)$ không chạm $y = x+k$ ($k>0$): $\binom{n+m}{m} - \binom{n+m}{m-k}$.
- **Định lý Ballot**: A có a phiếu, B có b ($a>b$). Xác suất A luôn dẫn trước nghiêm ngặt: $\frac{a-b}{a+b}$.
- **Số Stirling loại 1 và 2**: $\left[{n \atop k}\right] = \left[{n-1 \atop k-1}\right] + (n-1)\left[{n-1 \atop k}\right]$, $\left\{{n \atop k}\right\} = \left\{{n-1 \atop k-1}\right\} + k\left\{{n-1 \atop k}\right\}$. $x^n = \sum_{k=0}^{n}\left\{{n \atop k}\right\}x^{\underline{k}} \implies \sum_{i=1}^{n} i^k = \sum_{j=0}^{k}\left\{{k \atop j}\right\}j!\binom{n+1}{j+1}$.
- **Chu kỳ hoán vị**: Với hoán vị π của n phần tử có c chu kỳ (tính cả điểm cố định), số phép đổi chỗ hai vị trí bất kỳ ít nhất để về identity là $n-c$. Mỗi chu kỳ dài ℓ cần $\ell-1$ lần. Biến hoán vị A thành B: xét $\pi(i) = \text{pos}_B(A_i)$. Không áp dụng cho chỉ đổi hai vị trí kề nhau.
- **Eulerian numbers**: $E(n,k)$ đếm hoán vị n phần tử có đúng k vị trí giảm $a_i > a_{i+1}$. $E(0,0)=1$; với $n \ge 1$, $E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$, chỉ tính $0 \le k < n$, các chỉ số khác bằng 0. Với $n \ge 1$, cũng bằng số hoán vị có đúng k vị trí $a_i > i$ (chỉ số từ 1).
- **PIE**: $|\bigcup_{i=1}^{n}A_i| = \sum_{\emptyset\ne J}(-1)^{|J|-1}|\bigcap_{j\in J}A_j|$; Thuộc đúng r tập: $\sum_{m=r}^{n}(-1)^{m-r}\binom{m}{r}\sum_{|X|=m}|\bigcap_{i\in X}A_i|$.
- **Định lý Lucas**: $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$ (chữ số cơ số p).
- **Đếm cây và Cayley**: n^{n-2} cây có nhãn, rừng gồm k cây có gốc: kn^{n-k-1}.
- **LGV (Lindström–Gessel–Viennot)**: Trên DAG, đặt M_{ij} là số đường đi $A_i \to B_j$. Khi đó $\det M = \sum_{\sigma\in S_k}\text{sgn}(\sigma)N_\sigma$, với N_σ đếm các bộ đường đi đôi một không chung đỉnh, nối $A_i \to B_{\sigma(i)}$. Nếu mọi phép ghép khác đồng nhất đều buộc giao nhau, $\det M$ chính là số bộ nối $A_i \to B_i$. Nhận biết: nhiều đường đi đơn điệu trên lưới, không chung đỉnh; mỗi $M_{ij} = \binom{\Delta x+\Delta y}{\Delta x}$ nếu $\Delta x, \Delta y \ge 0$, ngược lại 0. Các bộ giao nhau triệt tiêu bằng cách đổi đuôi sau điểm giao, làm đổi dấu hoán vị.
- **Matrix–Tree (Kirchhoff)**: Đồ thị vô hướng có Laplacian $L_{ii} = \deg(i)$, $L_{ij} = -m_{ij}$ ($i \ne j$), với m_{ij} là số cạnh nối i,j. Xóa hàng và cột cùng chỉ số r được $L^{(r)}$; số cây khung $\tau(G) = \det L^{(r)}$, với mọi r. Đồ thị không liên thông cho 0; một đỉnh cho 1. Giữ cạnh song song, bỏ self-loop. Tính modulo nguyên tố bằng `determinant_mod`.
- **Đếm MST**: Kruskal theo nhóm cạnh cùng trọng số; co DSU từ các cạnh nhẹ hơn, bỏ cạnh nội bộ. Nhân $\tau(C)$ cho từng thành phần liên thông C của đồ thị co (giữ cạnh song song), rồi mới union nhóm hiện tại. Đồ thị gốc không liên thông: không có MST.

6.15 Hàm sinh (Generating Functions) và chuỗi Taylor

OGF, EGF, khai triển Taylor, tích phân số Simpson

- OGF: $A(x) = \sum a_n x^n$; EGF: $A(x) = \sum a_n \frac{x^n}{n!}$.
- $\frac{1}{1-x} = \sum_{n\ge0}x^n$, $\frac{1}{(1-x)^k} = \sum_{n\ge0}\binom{n+k-1}{k-1}x^n$.
- $e^x = \sum_{n\ge0}\frac{x^n}{n!}$, $\ln(1-x) = -\sum_{n\ge1}\frac{x^n}{n}$, $(1+x)^\alpha = \sum_{n\ge0}\binom{\alpha}{n}x^n$.
- **Công thức Simpson**: $\int_a^{a+2h} f(x)dx \approx \frac{h}{3}(f(a)+4f(a+h)+f(a+2h))$.

6.16 Truy hồi tuyến tính (Berlekamp-Massey)

Độ phức tạp: $O(N^2)$ tìm truy hồi, $\mathcal{O}(N^2\log K)$ tính số hạng thứ k

Tìm truy hồi tuyến tính tối tiểu từ $2N$ số hạng đầu, tính số hạng thứ K qua $x^K \bmod P(x)$

```cpp
// Use: vector<int> a = {0,1,1,2,3,5};
//      auto rec = berlekamp_massey(a); int x = recurrence_term(a,rec,10);
// Prime modulus 998244353; s[i]=sum recurrence[j]*s[i-1-j].
vector<int> berlekamp_massey(vector<int> s) {
  for (auto& x : s) x = (x % MOD + MOD) % MOD;
  vector<int> C {1}, B {1}; int L = 0, gap = 1; int previous = 1;
  for (int i = 0; i < (int) s.size(); ++i) {
    int d = (s[i] % MOD + MOD) % MOD;
    for (int j = 1; j <= L; ++j) d = (d + 1LL * C[j] * s[i - j]) % MOD;
    if (!d) { ++gap; continue; }
    auto old = C; int coef = d * power(previous, MOD - 2) % MOD;
    if (C.size() < B.size() + gap) C.resize(B.size() + gap);
    for(int j=0;j<(int)B.size();++j)C[j+gap]=(C[j+gap]-coef*B[j]%MOD+MOD)%MOD;
    if(2*L<=i){ L=i+1-L; B=move(old); previous=d; gap=1; } else ++gap;
  }
  C.resize(L + 1); C.erase(C.begin());
  for (auto& x : C) x = (MOD - x) % MOD;
  return C;
}
vector<int> recurrence_combine(const vector<int>& a,const vector<int>& b,
    const vector<int>& recurrence) {
  int n=recurrence.size();
    vector<int> c(2 * n - 1);
    for(int i=0;i<n;++i)for(int j=0;j<n;++j)c[i+j]=(c[i+j]+1LL*a[i]*b[j])%MOD;
    for(int i=2*n-2;i>=n;--i)for(int j=1;j<=n;++j)c[i-j]=(c[i-j]+1LL*c[i]*
     recurrence[j-1])%MOD;
    c.resize(n); return c;

}
int recurrence_term(const vector<int>&initial,const vector<int>&recurrence,int k
    ){
  assert(k >= 0);
  int n = recurrence.size();
  if (k < (int)initial.size()) return(initial[k] % MOD + MOD) % MOD;
  if (!n) return 0;
  assert(initial.size() >= (size_t) n);
vector<int> a(n), x(n); a[0] = 1;
  if (n == 1) x[0] = recurrence[0];
  else x[1] = 1;
  for (; k; k >>= 1, x = recurrence_combine(x,x,recurrence)) if (k & 1) a =
    recurrence_combine(a,x,recurrence);
  int ans = 0;
  for(int i=0; i<n; ++i)ans=(ans+1LL*a[i]*initial[i])%MOD;
  return(ans + MOD) % MOD;
}
```

6.17 FWHT

Độ phức tạp: $\mathcal{O}(n\log n)$
Fast Walsh-Hadamard Transform (XOR, AND, OR Convolution)

```cpp
// Use: vector<int> a(8,1); fwht(a,Walsh::Xor,998244353);
enum class Walsh {Xor, And, Or};
// mod must be prime; inverse XOR requires an odd mod.
void fwht(vector<int>& a, Walsh kind, int mod, bool inverse = false) {
  int n = a.size(); assert(n && !(n & (n - 1)) && mod > 1);
  assert(!inverse || kind != Walsh::Xor || (mod & 1));
  for (int& x : a) { x %= mod; if (x < 0) x += mod; }
  for (int len = 1; len < n; len <<= 1)
    for (int i = 0; i < n; i += 2 * len)
      for (int j = 0; j < len; ++j) {
        int& x = a[i + j]; int& y = a[i + j + len];
        int u = x, v = y;
        if (kind == Walsh::Xor) {
          x = (int)(((__int128)u + v) % mod);
          y = u >= v ? u - v : mod - (v - u);
        } else if (kind == Walsh::Or) {
          y = inverse ? (v >= u ? v - u : mod - (u - v))
                      : (int)(((__int128)u + v) % mod);
        } else {
          x = inverse ? (u >= v ? u - v : mod - (v - u))
                      : (int)(((__int128)u + v) % mod);
        }
      }
```

```
  if (inverse && kind == Walsh::Xor) {
    int inv_n = pow_mod(n % mod, mod - 2, mod);
    for (int& x : a) x = mul_mod(x, inv_n, mod);
  }
}
```

6.18 Lagrange

Độ phức tạp: $O(n)$ tính giá trị

Nội suy Lagrange tại X từ $n+1$ điểm cách đều có giá trị $y_0, \ldots, y_n$

```
// Use: auto y = lagrange(vector<int>{0,1,4},5,998244353);
// y[i]=f(i), degree < y.size() < prime modulus; input values reduced.
int lagrange(const vector<int>& y, int x, int p) {
  assert(x >= 0 && is_prime(p)&&!y.empty()&&(int)y.size()<p); int n=y.size(); x
    %=p;
  if (x < (int) n) return y[x] % p;
  vector<int> pre(n + 1, 1), suf(n + 1, 1), invfact(n, 1);
  for (int i = 0; i < n; ++i) pre[i + 1] = mul_mod(pre[i], x - i, p);
  for(int i=n-1; i>=0; --i)suf[i]=mul_mod(suf[i+1],x-i,p);
  int fact = 1;
  for (int i = 1; i < n; ++i) fact = mul_mod(fact, i, p);
  invfact[n - 1] = pow_mod(fact, p - 2, p);
  for(int i=n-1; i>0; --i)invfact[i-1]=mul_mod(invfact[i],i,p);
  int ans = 0;
  for (int i = 0; i < n; ++i) {
    int term = mul_mod(y[i] % p, mul_mod(pre[i], suf[i + 1], p), p);
    term = mul_mod(term, mul_mod(invfact[i], invfact[n - 1 - i], p), p);
    if ((n - 1 - i) & 1) term = term ? p - term : 0;
    ans = static_cast<int>((__int128(ans) + term) % p);
  }
  return ans;
}
```

Công thức: Với $d+1$ điểm phân biệt (x_i, y_i) và $\deg f \le d$,

$$f(X) = \sum_{i=0}^{d} y_i \prod_{\substack{0 \le j \le d \\ j \ne i}} \frac{X - x_j}{x_i - x_j}.$$

Code dùng $x_i = i$, nên $\prod_{j \ne i}(i-j) = (-1)^{d-i} i!(d-i)!$:

$$f(X) = \sum_{i=0}^{d} y_i \frac{(-1)^{d-i}}{i!(d-i)!} \prod_{\substack{0 \le j \le d \\ j \ne i}} (X - j).$$

Trong modulo p, phép chia là nhân nghịch đảo; tích tiền tố/hậu tố tính các tích bỏ i trong $O(d)$.

Khi dùng: Cần $f(N)$ với N rất lớn, biết f là đa thức bậc tối đa d và tính được $f(0), \ldots, f(d)$ nhanh. Không suy ra tính đa thức chỉ từ vài giá trị đầu.

Ví dụ: $S_k(N) = \sum_{i=1}^{N} i^k$ có bậc $k+1$: lập $y[0..k+1]$ bằng tổng tiền tố, gọi `lagrange(y,N,p)`. Tương tự, đếm cách chọn k phần tử từ N phần tử: $\binom{N}{k}$ là đa thức bậc k.

Điều kiện code: $y[i] = f(i)$, modulo nguyên tố p, số điểm $d+1 < p$; với tổng lũy thừa cần $k+2 < p$. Mỗi lần tính tốn $O(d + \log p)$, gồm nghịch đảo modulo.

6.19 NTT (Number Theoretic Transform)

Độ phức tạp: $\mathcal{O}(n \log n)$

NTT lặp với bit-reversal, MOD = 998244353 ($G = 3$)

```
// Use: auto c = convolution_ntt({1,2},{3,4});
const int MOD = 998244353, G = 3;
int power(int a, int b) {
  int res = 1;
  for(a%=MOD; b; b>>=1,a=a*a%MOD)if(b&1)res=res*a%MOD;
  return res;
}
int modInverse(int n) { return power(n, MOD - 2); }
void ntt(vector<int>& a, bool invert) {
  int n = a.size(); assert(n > 0 && !(n & (n - 1)) && n <= (1 << 23));
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    int wlen = power(G, (MOD - 1) / len);
    if (invert) wlen = modInverse(wlen);
    for (int i = 0; i < n; i += len) {
      int w = 1;
      for (int j = 0; j < len / 2; j++) {
        int u = a[i + j], v = a[i + j + len / 2] * w % MOD;
        a[i + j] = (u + v >= MOD ? u + v - MOD : u + v);
        a[i+j+len/2]=(u-v<0?u-v+MOD:u-v); w=w*wlen%MOD;
      }
    }
  }
  if (invert) {
    int inv_n = modInverse(n);
    for (int& x : a) x = x * inv_n % MOD;
  }
}
vector<int> convolution_ntt(vector<int> a, vector<int> b) {
  if (a.empty() || b.empty()) return {};
  for (auto& x : a) x = (x % MOD + MOD) % MOD;
  for (auto& x : b) x = (x % MOD + MOD) % MOD;
  int sz = 1, need = a.size() + b.size() - 1;
  while (sz < need) sz <<= 1;
  a.resize(sz); b.resize(sz); ntt(a, false); ntt(b, false);
  for (int i = 0; i < sz; i++) a[i] = 1LL * a[i] * b[i] % MOD;
  ntt(a, true); a.resize(need); return a;
}
```

6.20 FFT (Fast Fourier Transform)

Độ phức tạp: $\mathcal{O}(n \log n)$

FFT Cooley-Tukey trên số phức cho tích chập số thực/tổng quát

```
// Use: auto c = convolution_fft({1,2},{3,4});
// Rounded integer convolution; coefficient magnitudes must allow reliable
    double precision.
using cd = complex<double>; const double PI = acos(-1);
void fft(vector<cd>& a, bool invert) {
  int n = a.size(); assert(n > 0 && !(n & (n - 1)));
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    double ang=2*PI/len*(invert?-1:1); cd wlen(cos(ang),sin(ang));
    for (int i = 0; i < n; i += len) {
      cd w(1);
      for (int j = 0; j < len / 2; j++) {
        cd u = a[i + j], v = a[i + j + len / 2] * w; a[i + j] = u + v;
        a[i + j + len / 2] = u - v; w *= wlen;
      }
    }
  }
  if (invert) for (cd& x : a) x /= n;
}
vector<int> convolution_fft(const vector<int>& a, const vector<int>& b) {
  if (a.empty() || b.empty()) return {};
  int sz = 1, need = a.size() + b.size() - 1;
  while (sz < need) sz <<= 1;
  vector<cd> packed(sz), product(sz);
  for (int i = 0; i < (int)a.size(); ++i) packed[i].real(a[i]);
  for (int i = 0; i < (int)b.size(); ++i) packed[i].imag(b[i]);
  fft(packed, false);
  for (int i = 0; i < sz; ++i) {
      int j = (sz - i) % sz;
      cd A = (packed[i] + conj(packed[j])) * 0.5;
      cd B = (packed[i] - conj(packed[j])) * cd(0, -0.5);
      product[i] = A * B;
  }
  fft(product, true);
  vector<int> res(need);
  for (int i = 0; i < need; ++i) res[i] = llround(product[i].real());
  return res;
}
```

6.21 Công thức Formal Power Series (FPS)

Công thức thao tác đa thức bằng phương pháp Newton và convolution

- **Nghịch đảo** ($Q = P^{-1}$): $Q_k \equiv Q_{k-1}(2 - PQ_{k-1}) \pmod{x^{2^k}}$, khởi đầu $Q_0 = P(0)^{-1}$.
- **Đạo hàm và tích phân**: $P'(x) = \sum i a_i x^{i-1}$, $\int P(x) = \sum \frac{a_i}{i+1} x^{i+1}$.
- **Logarithm**: $\ln P(x) = \int \frac{P'(x)}{P(x)} dx \pmod{x^n}$ (cần $P(0) = 1$).
- **Hàm mũ**: $Q_k \equiv Q_{k-1}(1 - \ln Q_{k-1} + P) \pmod{x^{2^k}}$ (cần $P(0) = 0$).
- **Căn bậc hai**: $Q_k \equiv \frac{1}{2}(Q_{k-1} + PQ_{k-1}^{-1}) \pmod{x^{2^k}}$ ($P(0)$ là thặng dư bậc hai).
- **Lũy thừa**: $P(x)^k = \exp(k \ln P(x))$. Nếu $P(x) = cx^d(1 + Q(x))$, $P(x)^k = c^k x^{kd} \exp(k \ln(1 + Q(x)))$.
- **Chia lấy thương và dư** ($A = BQ + R$): Với $\mathrm{rev}_k(P) = x^k P(1/x)$:

$$\mathrm{rev}_{n-m}(Q) \equiv \mathrm{rev}_n(A) \cdot (\mathrm{rev}_m(B))^{-1} \pmod{x^{n-m+1}}, \quad R = A - BQ$$

- **Taylor Shift** $P(x+c)$: $P(x+c) = \sum_{k=0}^{n} \frac{x^k}{k!} \sum_{i=k}^{n} (i! a_i) \frac{c^{i-k}}{(i-k)!}$. Convolution dãy $A_i = i! a_i$ đảo ngược với $B_j = \frac{c^j}{j!}$.
- **Convolution modulo tùy ý (MTT)**: Tách $A = A_1 B + A_0$, $B = \lceil \sqrt{M} \rceil \approx 32000$. Tính $(A_1 B_1), (A_1 B_0 + A_0 B_1), (A_0 B_0)$ bằng bốn FFT.

6.22 Đại số tuyến tính (Gauss)

Độ phức tạp: $\mathcal{O}(n^3)$

Khử Gauss modulo nguyên tố p: định thức, hạng, nghiệm riêng và cơ sở nghiệm thuần nhất

```
  // Use: auto sol = gauss({{1,2,3},{2,1,3}},2,1000000007);
// All arithmetic is modulo a prime mod. Matrix rows contain m coefficients and
    RHS.
struct LinearSolution {
    int rank; bool consistent; vector<int> particular; vector<vector<int>>
      nullspace;
};
LinearSolution gauss(vector<vector<int>> a, int m, int mod = 1000000007) {
    assert(m >= 0 && mod > 1); int n = a.size(), r = 0; vector<int> where(m, -1);
    for (auto& row : a) {
        assert((int)row.size() == m + 1); for (int& x : row) { x %= mod; if (x <
      0) x += mod; } }
    for (int c = 0; c < m && r < n; ++c) {
        int pivot = r; while (pivot < n && a[pivot][c] == 0) ++pivot; if (pivot ==
      n) continue;
        swap(a[pivot], a[r]); int inv = pow_mod(a[r][c], mod - 2, mod);
        for (int j = c; j <= m; ++j) a[r][j] = mul_mod(a[r][j], inv, mod);
        for (int i = 0; i < n; ++i) if (i != r) {
            int factor = a[i][c]; for (int j = c; j <= m; ++j) {
                a[i][j] -= mul_mod(factor, a[r][j], mod); if (a[i][j] < 0) a[i][j]
      += mod; } }
        where[c] = r++; }
    LinearSolution out{r, true, vector<int>(m), {}}; for (int i = r; i < n; ++i)
      if (a[i][m] != 0) {
        out.consistent = false; return out; }
    for (int c = 0; c < m; ++c) {
        if (where[c] != -1) out.particular[c] = a[where[c]][m]; else {
```

```
            vector<int> v(m); v[c] = 1; for (int j = 0; j < m; ++j) if (where[j] != -1) {
                int x = a[where[j]][c]; v[j] = x ? mod - x : 0; }
            out.nullspace.push_back(v); } }
    return out; }
// Square matrix; mod must be prime. O(n^3 + n log mod).
int determinant_mod(vector<vector<int>> a, int mod = 1000000007) {
    assert(mod > 1); int n = a.size(), ans = 1; for (auto& row : a) {
        assert((int)row.size() == n); for (int& x : row) { x %= mod; if (x < 0) x += mod; } }
    for (int c = 0; c < n; ++c) {
        int pivot_row = c; while (pivot_row < n && a[pivot_row][c] == 0) ++pivot_row;
        if (pivot_row == n) return 0; if (pivot_row != c) {
            swap(a[pivot_row], a[c]); ans = ans ? mod - ans : 0; }
        int pivot = a[c][c]; ans = mul_mod(ans, pivot, mod); int inv_pivot = pow_mod(pivot, mod - 2, mod);
        for (int i = c + 1; i < n; ++i) {
            int factor = mul_mod(a[i][c], inv_pivot, mod); a[i][c] = 0;
            for (int j = c + 1; j < n; ++j) {
                a[i][j] -= mul_mod(factor, a[c][j], mod); if (a[i][j] < 0) a[i][j] += mod; } } }
    return ans; }
```

Nghiệm tổng quát: $x = x_0 + \sum_i t_i v_i \pmod p$, với t_i tùy ý; extttconsistent=false nếu vô nghiệm.

6.23 XOR Linear Basis

Độ phức tạp: $O(B)$ thêm/truy vấn ($B \le 60$)

XOR basis: XOR max/min/thứ k của tập con, sinh 2^{size} giá trị phân biệt

```
// Nonnegative int values.
// Use: XorBasis basis; basis.insert(7); auto best = basis.maximum();
struct XorBasis {
  static constexpr int BITS = numeric_limits<int>::digits;
  array<int, BITS> b {}; int count = 0;
  bool insert(int x) {
    assert(x >= 0);
    for (int i = BITS - 1; i >= 0; --i) if (x >> i & 1) {
      if (b[i]) x ^= b[i];
      else { b[i] = x; ++count; return true; }
    }
    return false;
  }
  int rank() const { return count; }
  bool contains(int x) const {
    assert(x >= 0);
    for (int i = BITS - 1; i >= 0; --i) if (x >> i & 1) x ^= b[i];
    return x == 0;
  }
  int maximum(int x = 0) const {
    assert(x >= 0);
    for (int i = BITS - 1; i >= 0; --i) x = max(x, x ^ b[i]);
    return x;
  }
  int minimum(int x = 0) const {
    assert(x >= 0);
    for (int i = BITS - 1; i >= 0; --i) x = min(x, x ^ b[i]);
    return x;
  }
  optional<int> kth(int k) const {
    if (k < 0 || (count < BITS && k >= (static_cast<int>(1) << count))) return nullopt;
    auto a = b; vector<int> v;
    for (int i = 0; i < BITS; ++i) if (a[i]) {
      for (int j = 0; j < i; ++j) if (a[i] >> j & 1) a[i] ^= a[j];
      v.push_back(a[i]);
    }
    int x = 0;
    for (int i = 0; i < (int) v.size(); ++i) if (k >> i & 1) x ^= v[i];
    return x;
  }
};
```

6.24 Burnside Lemma & Pólya Theorem

Đếm lớp tương đương dưới tác động nhóm: necklace, bracelet

- $|Classes| = \frac{1}{|G|}\sum_{g\in G}|Fix(g)| = \frac{1}{|G|}\sum_{g\in G} k^{c(g)}$.
- **Necklace (chỉ n phép xoay)**: $N(n,k) = \frac{1}{n}\sum_{d|n}\phi(d)k^{n/d}$.
- **Bracelet (xoay và đối xứng)**:
  - n lẻ: $B(n,k) = \frac{1}{2}N(n,k) + \frac{1}{2}k^{(n+1)/2}$.
  - n chẵn: $B(n,k) = \frac{1}{2}N(n,k) + \frac{1}{4}(k^{n/2} + k^{n/2+1})$.

6.25 Game: Invariant, Symmetric, Nim, DP

- **1. Invariant (Bất biến)**: Tìm đại lượng bất biến (parity chẵn/lẻ, modulo $k+1$, số nghịch thế) mà mọi nước đi hợp lệ đều bảo toàn hoặc đổi luân phiên. Người thắng luôn đưa trạng thái về tập bất biến mà đối thủ không thể duy trì. *Ví dụ*: Bốc $1\dots k$ sỏi từ n viên (lấy cuối thắng): Bất biến $n \equiv 0 \pmod{k+1}$ là thế thua (P-pos). Nếu $n \not\equiv 0$, bốc $n \bmod (k+1)$ về bội $k+1$; đối thủ bốc $x \in [1,k]$, ta bốc $k+1-x$ để giữ $0 \pmod{k+1}$.
- **2. Symmetric (Đối xứng / Bắt cặp)**: Ghép các trạng thái hoặc nước đi thành cặp đối xứng rời nhau (qua tâm, trục, hoặc perfect matching). Đối thủ đi vào một nửa thì ta đi vào nửa còn lại để khôi phục đối xứng. Người đầu thắng nếu chiếm tâm/nước lẻ đầu rồi phản chiếu; người sau thắng nếu bàn cờ ban đầu đã hoàn toàn đối xứng. *Ví dụ*: Đặt xu lên bàn tròn: P1 đặt đúng tâm; P2 đặt tại (x,y) thì P1 đặt tại $(-x,-y) \implies$ P1 thắng. Trên đồ thị có perfect matching: P2 luôn đi theo cạnh ghép của đỉnh P1 vừa chọn $\implies$ P2 thắng.
- **3. Nim (XOR & Sprague-Grundy)**: Impartial game dưới normal play (hết nước đi là thua). Mọi game trên DAG tương đương 1 đống Nim kích thước $G(u) = \text{mex}\{G(v) \mid u \to v\}$. Độc lập: $G_{\text{tổng}} = \bigoplus_i G_i$. Thua (P-pos) $\iff G = 0$; Thắng (N-pos) $\iff G \neq 0$ (chọn đống có bit cao nhất của G, giảm $a_i \to a_i \oplus G$). *Misère Nim* (lấy cuối thua): mọi đống $\le 1 \implies$ thua $\iff$ số đống 1 lẻ; có đống $> 1 \implies$ chơi như Nim thường, còn đúng 1 đống > 1 thì giảm nó về 0 hoặc 1 để lại số đống 1 lẻ. *Staircase Nim*: chuyển sỏi từ bậc $i \to i-1$ (bậc 0 loại bỏ); chỉ XOR các bậc *lẻ* ($G = \bigoplus_{i \text{ lẻ}} a_i$), đối thủ dời chẵn $\to$ lẻ thì ta dời tiếp số đó lẻ $\to$ chẵn kề dưới. *Wythoff Nim*: bốc 1 đống tùy ý hoặc 2 đống cùng lượng; thua $\iff a = \lfloor k\varphi \rfloor, b = a+k$ với $k = b-a, \varphi = (1+\sqrt{5})/2$. *Green Hackenbush (cây)*: chặt cạnh, nhánh rời gốc rơi: $g(u) = \bigoplus_{v\in\text{child}(u)}(g(v)+1)$.
- **4. DP (Quy hoạch động & Retrograde)**: Biểu diễn trạng thái trên đồ thị có hướng. Trạng thái lá (hết nước đi) là **LOSE**. Chuyển trạng thái: $\exists v \in$ LOSE $\implies u \in$ WIN; $\forall v \in$ WIN $\implies u \in$ LOSE. *Retrograde (chu trình/hòa)*: BFS ngược từ lá (LOSE), theo dõi bậc ra out_deg[u]: kề LOSE $\implies$ WIN; kề WIN $\implies$ giảm out_deg[u], về 0 $\implies$ LOSE; còn lại $\implies$ **DRAW** ($O(V+E)$). *Ví dụ*: Game chia kẹo/trừ số: $dp[i] =$ LOSE $\iff \forall \text{step}, dp[i-\text{step}] =$ WIN. Game di chuyển quân cờ lặp trạng thái: dùng Retrograde queue.

6.26 Xác suất và kỳ vọng

Tính tuyến tính của kỳ vọng, phân tích biến chỉ báo, Coupon Collector, bất đẳng thức

- **Tuyến tính kỳ vọng**: $E[\sum c_i X_i] = \sum c_i E[X_i]$ (luôn đúng $\forall X_i$, *kể cả khi các biến ngẫu nhiên phụ thuộc nhau*).
- **Biến chỉ báo (Indicator Variable Analysis)**: Đặt $I_A =$ [A xảy ra] $\in \{0,1\}$. Khi đó $E[I_A] = 1\cdot P(A) + 0\cdot P(\overline{A}) = P(A)$.
  - *Kỳ vọng đếm*: Muốn tính kỳ vọng số đối tượng thỏa mãn tính chất, viết $X = \sum_{i=1}^n I_{A_i} \implies E[X] = \sum_{i=1}^n P(A_i)$. Ta chuyển việc tính kỳ vọng toàn cục phức tạp thành tính xác suất xảy ra của từng biến cố thành phần riêng lẻ (nghiệm đúng cả khi các A_i phụ thuộc nhau).
  - *Ví dụ*: (1) Hoán vị ngẫu nhiên: số nghịch thế $E = \binom{n}{2}\cdot\frac{1}{2} = \frac{n(n-1)}{4}$; số điểm bất động $\pi(i) = i$ là $E = n\cdot\frac{1}{n} = 1$. (2) Đồ thị $G(n,p)$: số tam giác K_3 có $E = \binom{n}{3}p^3$; số đỉnh cô lập có $E = n(1-p)^{n-1}$.
  - *Phương sai & Tích*: $I_A^2 = I_A \implies E[I_A^2] = P(A)$; $I_A I_B = I_{A\cap B} \implies E[I_A I_B] = P(A\cap B)$. Khi đó $\text{Cov}(I_A, I_B) = P(A\cap B) - P(A)P(B)$; $\text{Var}(X) = \sum \text{Var}(I_i) + 2\sum_{i<j}\text{Cov}(I_i, I_j)$.
- **Phân phối hình học**: $E[X] = 1/p$, $\text{Var}(X) = (1-p)/p^2$.
- **Coupon Collector**: $E[T] = n\sum_{i=1}^n \frac{1}{i} = nH_n \approx n(\ln n + \gamma)$ ($\gamma \approx 0.5772$).
- **Bất đẳng thức**: **Markov**: $P(X \ge a) \le \frac{E[X]}{a}$ ($X \ge 0$); **Chebyshev**: $P(|X-\mu| \ge k\sigma) \le \frac{1}{k^2}$.

6.27 Bignum

Độ phức tạp: $O(n)$ in (n block)

Các block cơ số 10^9; tính trung gian bằng 64-bit. In block cao nhất bình thường, các block còn lại đủ 9 chữ số.

```
// Use: bignum a{42,1}; print(a); // 1000000042
// Nonnegative integer: base 10^9, least-significant block first.
// Zero = {0}; otherwise the last block must be nonzero.
// Example: 1234567890000000042 -> {42, 234567890, 1}.
using bignum = vector<int>; constexpr int BASE = 1000000000;
ostream & print(const bignum& a, ostream & out = cout) {
  // nonempty, normalized
  assert(!a.empty()); out << to_string(a.back());
  for (size_t i = a.size() - 1; i > 0; --i) {
    std::string block = to_string(a[i - 1]);
    out << std::string(9 - block.size(), '0') << block;
  }
  return out;
}
```

7 Hình học

7.1 Primitive Geometry

Độ phức tạp: $\mathcal{O}(1)$ mỗi thao tác

Phép toán 2D cơ bản: điểm, vector, orientation, giao đường thẳng, hình chiếu

```
// Use: Point a{0,0}, b{2,1}; Line l{a,b-a}; // point + direction
constexpr long double EPS = 1e-12L;
struct Point {
  long double x = 0, y = 0;
  Point operator + (Point p) const { return {x + p.x, y + p.y}; }
  Point operator - (Point p) const { return {x - p.x, y - p.y}; }
  Point operator * (long double k) const { return {x * k, y * k}; }
  Point operator / (long double k) const { return { x / k, y / k }; }
};
long double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
long double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
long double norm2(Point a) { return dot(a, a); }
Point perp(Point a) { return {-a.y, a.x}; }
int sign(long double x) { return(x > EPS) - (x < -EPS); }
struct Line { Point p, d; };
optional<Point> line_intersection(Line a, Line b) {
  long double z = cross(a.d, b.d);
  if (abs(z) <= EPS) return nullopt;
  return a.p + a.d * (cross(b.p - a.p, b.d) / z);
```

```
}
Point projection(Point p, Line l) {
  assert(norm2(l.d)>0); return l.p+l.d*(dot(p-l.p,l.d)/norm2(l.d));
}
bool on_segment(Point p, Point a, Point b) {
  return sign(cross(b - a, p - a)) == 0 && dot(p - a, p - b) <= EPS;
}
```

7.2 Convex Hull

Độ phức tạp: $\mathcal{O}(n \log n)$

Monotone Chain/Graham Scan: đa giác lồi nhỏ nhất chứa tập điểm

```
// Use: auto hull = convex_hull({{0,0},{2,0},{0,2}});
// CCW hull without duplicate/collinear vertices; exact sort, tolerant orientation.
bool point_xy_less(Point a,Point b) { return tie(a.x,a.y)<tie(b.x,b.y); }
bool point_exact_equal(Point a,Point b) { return a.x==b.x && a.y==b.y; }
vector<Point> convex_hull(vector<Point> p) {
  sort(p.begin(),p.end(),point_xy_less);
  p.erase(unique(p.begin(), p.end(), point_exact_equal), p.end());
  if (p.size() < 3) return p;
  vector<Point> h;
  for (Point v : p) {
    while(h.size()>1&&cross(h.back()-h[h.size()-2],v-h.back())<=EPS)h.pop_back();
    h.push_back(v);
  }
  size_t lower = h.size();
  for (int i = (int) p.size() - 2; i >= 0; --i) {
    Point v = p[i];
    while(h.size()>lower&&cross(h.back()-h[h.size()-2],v-h.back())<=EPS)h.pop_back();
    h.push_back(v);
  }
  h.pop_back(); return h;
}
```

7.3 Rotating Calipers

Độ phức tạp: $\mathcal{O}(n)$ trên convex hull không có ba đỉnh thẳng hàng

Antipodal pairs: Cặp đỉnh nằm trên hai đường đỡ song song bao quanh hull. Quét cạnh/cross product trả về cặp chỉ số; có thể dùng tìm đường kính.

```
// Use: auto d2 = diameter_squared({{0,0},{2,0},{0,2}});
// Strict CCW convex hull; emits supporting vertex pairs (duplicates permitted).
vector<pair<int, int>> antipodal_pairs(const vector<Point>& p) {
  int n = p.size(); vector<pair<int, int>> out;
  if (n < 2) return out;
  if (n == 2) return {
    {0, 1}
  }; int j = 1;
  for (int i = 0; i < n; ++i) {
    int k=(i+1)%n;
    while (cross(p[k]-p[i],p[(j + 1) % n]-p[i]) > cross(p[k]-p[i],p[j]-p[i]) + EPS) j = (j + 1) % n;
    out.push_back( {i, j}); out.push_back( {k, j});
    if (abs(cross(p[k]-p[i],p[(j + 1) % n]-p[i]) - cross(p[k]-p[i],p[j]-p[i])) <= EPS) {
      out.push_back( {i,(j+1)%n}); out.push_back( {k,(j+1)%n});
    }
  }
  return out;
}
long double diameter_squared(const vector<Point>& h) {
  long double ans = 0;
  for(auto[i,j]:antipodal_pairs(h))ans=max(ans,norm2(h[i]-h[j]));
  return ans;
}
```

7.4 Minkowski Sum

Độ phức tạp: $\mathcal{O}(|P| + |Q|)$

Minkowski sum của hai đa giác lồi. Khoảng cách nhỏ nhất: $\text{dist}(P, Q) = \text{dist}(O, P - Q)$

```
// Use: auto sum = minkowski_sum(hullA,hullB); // vector<Point>, CCW
// Strict CCW convex polygons. Degenerate hulls use the explicit fallback.
bool point_yx_less(Point a,Point b) { return tie(a.y,a.x)<tie(b.y,b.x); }
void rotate_min(vector<Point>& p) {
    auto it = min_element(p.begin(), p.end(), point_yx_less); rotate(p.begin(), it, p.end());

}
vector<Point> minkowski_sum(vector<Point> a, vector<Point> b) {
  if (a.empty() || b.empty()) return {};
  if (a.size() < 3 || b.size() < 3) {
    vector<Point> s;
    for (auto x : a) for (auto y : b) s.push_back(x + y);
    return convex_hull(s);
  }
rotate_min(a); rotate_min(b); int n=a.size(),m=b.size(),i=0,j=0;
  Point cur = a[0] + b[0]; vector<Point> out {cur};
  while (i < n || j < m) {
    Point u = a[(i + 1) % n] - a[i % n], v = b[(j + 1) % m] - b[j % m];
    int c = i == n ? -1 : j == m ? 1 : sign(cross(u, v));
    if (c >= 0 && i < n) { cur = cur + u; ++i; }
    if (c <= 0 && j < m) { cur = cur + v; ++j; }
    out.push_back(cur);
  }
  out.pop_back(); return out;
}
```

7.5 Point in Convex Polygon

Độ phức tạp: $\mathcal{O}(\log n)$

Kiểm tra điểm trong đa giác lồi bằng binary search góc cực từ P_0

```
// Use: int where = point_in_convex({{0,0},{2,0},{0,2}},{1,0});
// -1 outside, 0 boundary, 1 inside; strict CCW polygon.
int point_in_convex(const vector<Point>& p, Point q) {
  int n = p.size();
  if (!n) return-1;
  if (n == 1) return norm2(q - p[0]) <= EPS * EPS ? 0 : -1;
  if (n == 2) return on_segment(q, p[0], p[1]) ? 0 : -1;
  int a=sign(cross(p[1]-p[0],q-p[0])),b=sign(cross(p.back()-p[0],q-p[0]));
  if (a < 0 || b > 0) return-1;
  if (!a) return on_segment(q, p[0], p[1]) ? 0 : -1;
  if (!b) return on_segment(q, p[0], p.back()) ? 0 : -1;
  int l = 1, r = n - 1;
  while (r - l > 1) {
    int m = (l + r) / 2;
    if (cross(p[m] - p[0], q - p[0]) >= 0) l = m;
    else r = m;
  }
  return sign(cross(p[r] - p[l], q - p[l]));
}
```

7.6 Line Hull Intersection

Độ phức tạp: $O(n)$ tiền xử lý, $O(\log n)$ truy vấn

Tiền xử lý pháp tuyến cạnh; binary search đỉnh cực trị và cạnh bị cắt

```
// Use: LineHullIntersection idx({{0,0},{2,0},{0,2}});
//      auto hits = idx.intersect(Line{{0,1},{1,0}});
// O(n) preprocessing, O(log n) query. Strict CCW hull, n>=3.
struct LineHullIntersection {
  vector<Point> p; vector<pair<long double, int>> normals;
  int extreme(Point d) const {
    long double a = atan2l(d.y, d.x);
    auto it=lower_bound(normals.begin(),normals.end(),pair<long double,int>{a, INT_MIN}); return it == normals.end() ? normals[0].second : it->second;
  }
  LineHullIntersection(vector<Point> h) : p(move(h)) {
    assert(p.size() >= 3);
    for (int i = 0; i < (int) p.size(); ++i) {
      Point e = p[(i + 1) % p.size()] - p[i], out = {e.y, -e.x};
      normals.push_back( {atan2l(out.y, out.x), i});
    }
    auto it = min_element(normals.begin(), normals.end());
    rotate(normals.begin(), it, normals.end());
  }
  long double side(Line l,int i) const { return cross(l.d,p[i]-l.p); }
  vector<Point> intersect(Line l) const {
    assert(norm2(l.d) > 0); Point normal = perp(l.d);
    int hi = extreme(normal), lo = extreme(normal * -1), n = p.size();

    if (side(l,hi) < -EPS || side(l,lo) > EPS) return {};
    // A supporting line may touch a vertex or coincide with one whole edge.
    if (abs(side(l,hi)) <= EPS || abs(side(l,lo)) <= EPS) {
      int u = abs(side(l,hi)) <= EPS ? hi : lo; vector<Point> hits {p[u]};
      int neighbors[] = {(u+n-1)%n, (u+1)%n};
      for(int v : neighbors)if(abs(side(l,v))<=EPS)hits.push_back(p[v]);
      return hits;
    }
    vector<Point> out;
    pair<int,int> arcs[] = {{lo, hi}, {hi, lo}};
    for (auto ends : arcs) {
      int a=ends.first,b=ends.second,steps=(b-a+n)%n,L=0,R=steps;
      long double initial = side(l,a);
      if (abs(initial) <= EPS) { out.push_back(p[a]); continue; }
      while (R - L > 1) {
        int m = (L + R) / 2; long double v = side(l,(a + m) % n);
        if ((v > 0) == (initial > 0)) L = m;
        else R = m;
      }
      Point x=p[(a+L)%n],y=p[(a+R)%n]; auto hit=line_intersection(l,{x,y-x});
      if (hit) out.push_back(*hit);
      else if (abs(side(l,(a + R) % n)) <= EPS) out.push_back(y);
    }
    if(out.size()==2&&norm2(out[0]-out[1])<=EPS*EPS)out.pop_back();
    return out;
  }
};
```

7.7 Polygon Raycast

Độ phức tạp: $O(n)$

Raycasting ngang kiểm tra điểm trong đa giác đơn bất kỳ

```
// Use: int where = point_in_polygon({{0,0},{2,0},{0,2}},{1,0});
int point_in_polygon(const vector<Point>& p, Point q) {
  bool inside = false;
  for (int i = 0, n = p.size(); i < n; ++i) {
    Point a = p[i], b = p[(i + 1) % n];
    if (on_segment(q, a, b)) return 0;
    if ((a.y > q.y) != (b.y > q.y)) {
      long double x = a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
      if (x > q.x) inside = !inside;
    }
  }
  return inside ? 1 : -1;
}
```

7.8 Half-plane Intersection

Độ phức tạp: $O(n \log n)$

Giao các nửa mặt phẳng trái bằng deque, cắt trong hình vuông giới hạn do người gọi chọn

```
// Use: auto polygon = halfplane_intersection(lines,1000000.L);
//      lines: vector<Line>; keeps left side, clips to bounding square.
// Left side of each directed line, clipped to [-bound,bound]^2.
// Empty or lower-dimensional intersections return empty. Choose bound explicitly.
long double halfplane_angle(Line l) {
  long double a=atan2l(l.d.y,l.d.x); return a<0?a+2*acosl(-1.L):a;
}
bool halfplane_angle_less(Line a,Line b) { return halfplane_angle(a)<halfplane_angle(b); }
bool outside(Line l,Point p) { return cross(l.d,p-l.p)<-EPS; }
vector<Point>halfplane_intersection(vector<Line>lines,long double bound){
  assert(bound > 0);
  lines.insert(lines.end(), {
    {
      {-bound, -bound}, {1, 0}
    },{ {bound,-bound},{0,1} },{ {bound,bound},{-1,0} },{ {-bound,bound},{0,-1}
    }
  });
  sort(lines.begin(),lines.end(),halfplane_angle_less);
  deque<Line> q;
  for (Line l : lines) {
    assert(norm2(l.d) > 0);
```

```
    while (q.size() > 1) {
      auto p = line_intersection(q[q.size() - 2], q.back());
      if (!p || outside(l, *p)) q.pop_back();
      else break;
    }
    while (q.size() > 1) {
      auto p = line_intersection(q[0], q[1]);
      if (!p || outside(l, *p)) q.pop_front();
      else break;
    }
    if (!q.empty() && abs(cross(q.back().d, l.d)) <= EPS) {
      if (dot(q.back().d, l.d) < 0) return {};
      if (outside(l, q.back().p)) q.pop_back();
      else continue;
    }
    q.push_back(l);
  }
  while (q.size() > 2) {
    auto p = line_intersection(q[q.size() - 2], q.back());
    if (!p || outside(q.front(), *p)) q.pop_back();
    else break;
  }
  while (q.size() > 2) {
    auto p = line_intersection(q[0], q[1]);
    if (!p || outside(q.back(), *p)) q.pop_front();
    else break;
  }
  if (q.size() < 3) return {};
  vector<Point> out;
  for (int i = 0; i < (int) q.size(); ++i) {
    auto p = line_intersection(q[i], q[(i + 1) % q.size()]);
    if (!p) return {};
    out.push_back(*p);
  }
  long double area = 0;
  for(int i=0;i<(int)out.size();++i)area+=cross(out[i],out[(i+1)%out.size()]);
  if (abs(area) <= EPS) return {};
  return out;
}
```

7.9 Smallest Enclosing Circle

Độ phức tạp: $O(n)$ kỳ vọng

Thuật toán ngẫu nhiên Welzl tìm đường tròn bao nhỏ nhất

```
// Use: mt19937_64 rng(123456);
//      auto c = smallest_enclosing_circle({{0,0},{2,0},{0,2}},rng);
struct Circle { Point center; long double radius; };
Circle diameter_circle(Point a, Point b) {
  Point c = (a + b) / 2; return {c, sqrtl(norm2(a - c))};
}
bool contains(Circle c, Point p) {
  return norm2(p-c.center)<=c.radius*c.radius+EPS*max(1.L,c.radius*c.radius);
}
Circle circle_through(Point a, Point b, Point c) {
  Point u = b - a, v = c - a; long double d = 2 * cross(u, v);
  if (abs(d) <= EPS) {
    Circle ans = diameter_circle(a, b);
    Circle candidates[] = {diameter_circle(a,c), diameter_circle(b,c)};
    for(auto x : candidates)if(x.radius>ans.radius)ans=x;
    return ans;
  }
  Point o=a+Point {v.y*norm2(u)-u.y*norm2(v),u.x*norm2(v)-v.x*norm2(u)}
  /d; return {o, sqrtl(norm2(o - a))};
}
Circle smallest_enclosing_circle(vector<Point> p, mt19937_64 & rng) {
  shuffle(p.begin(), p.end(), rng); Circle c { {0, 0}, -1 };
  for(int i=0; i<(int)p.size(); ++i)if(c.radius<0||!contains(c,p[i])){
    c = {p[i], 0};
    for (int j = 0; j < i; ++j) if (!contains(c, p[j])) {
      c = diameter_circle(p[i], p[j]);
      for(int k=0;k<j;++k)if(!contains(c,p[k]))c=circle_through(p[i],p[j],p[k]);
    }
  }
  if (p.empty()) c.radius = 0;
  return c;
}
```

7.10 Circle Tangents

Độ phức tạp: $\mathcal{O}(1)$

Tìm tối đa bốn tiếp tuyến chung của hai đường tròn

```
// Use: auto tangents = circle_tangents({0,0},1,{4,0},1);
// Tangency points on circles; nullopt means infinitely many (identical circles)
optional<vector<pair<Point,Point>>>circle_tangents(Point a,long double ra,Point
    b,long double rb){
  assert(ra >= 0 && rb >= 0); Point d = b - a; long double z = norm2(d);
  if (z <= EPS * EPS) {
    if (abs(ra - rb) <= EPS) return nullopt;
    return vector<pair<Point, Point>> {};
  }
  vector<pair<Point, Point>> out;
  const int signs[] = {-1, 1};
  for (int s : signs) {
    long double r = ra - s * rb, h = z - r * r;
    if (h < -EPS) continue;
    h = sqrtl(max(0.L, h));
    for (int side : signs) {
      if (side == 1 && h <= EPS) continue;
      Point n=(d*r+perp(d)*(h*side))/z; pair<Point,Point>t {a+n*ra,b+n*(s*rb)};
      bool dup = false;
      for(auto old:out)if(norm2(old.first-t.first)+norm2(old.second-t.second)<=
     EPS*EPS)dup=true;
      if (!dup) out.push_back(t);
    }
  }
  return out;
}
```

7.11 Công thức hình học

Định lý phẳng, đường tròn, tâm tam giác, phép nghịch đảo và phép toán 3D

- **Biến đổi tuyến tính ↔ ma trận**: Mỗi ánh xạ tuyến tính $T : \mathbb{R}^n \to \mathbb{R}^m$ có $A = [T(e_1) \ \cdots \ T(e_n)]$ sao cho $T(x) = Ax$. Các dạng 2D thường dùng:
  - Quay góc θ: $\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$; co giãn: $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$.
  - Đối xứng qua Ox, Oy, $y = x$: $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, $\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$, $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.
  - Shear (trượt): $\begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$ hoặc $\begin{pmatrix} 1 & 0 \\ k & 1 \end{pmatrix}$; chiếu lên vector đơn vị u: $P = uu^T$.
  - Tịnh tiến là affine, không tuyến tính: $x' = Ax + b$. Trong tọa độ thuần nhất, $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.
  - Hợp thành $T_2 \circ T_1$ ứng với A_2A_1; lặp một biến đổi k lần ứng với A^k.
- **Định lý Pick**: $S = I + \frac{B}{2} - 1 \implies I = S - \frac{B}{2} + 1$, với $B = \sum \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$.
- **Công thức Euler cho đồ thị phẳng**: $V - E + F = 1 + C$, với C là số thành phần liên thông, F kể cả mặt ngoài.
- **Định lý Ptolemy**: Tứ giác nội tiếp $ABCD \iff AC \cdot BD = AB \cdot CD + BC \cdot AD$.
- **Định lý Stewart**: $b^2m + c^2n = a(d^2 + mn)$ (cevian d chia cạnh a thành m, n).
- **Công thức Bretschneider** (Diện tích tứ giác bất kỳ):

$$S = \sqrt{(p-a)(p-b)(p-c)(p-d) - abcd\cos^2\left(\frac{A+C}{2}\right)}$$

(Tứ giác nội tiếp $\implies \cos(\frac{A+C}{2}) = 0$, trở thành công thức Brahmagupta).

- **Định lý Descartes** (bốn đường tròn tiếp xúc từng đôi):

$$\left(\sum_{i=1}^{4} k_i\right)^2 = 2\sum_{i=1}^{4} k_i^2 \quad \left(k_i = \pm\frac{1}{R_i}\right)$$

- **Nghịch đảo đường tròn (Circle Inversion)**: Tâm O, bán kính R. $P \mapsto P'$ thỏa $OP \cdot OP' = R^2$. Đổi đường thẳng không qua $O \longleftrightarrow$ đường tròn qua O; bảo toàn góc và tiếp xúc.
- **Giao hai đường tròn**: Đặt $d = |O_1 - O_2|$. Có giao khi $|R_1 - R_2| \le d \le R_1 + R_2$, góc $\alpha = \operatorname{acos}\left(\frac{R_1^2+d^2-R_2^2}{2R_1 d}\right)$.
- **Đường tròn Apollonius**: Quỹ tích $\frac{PA}{PB} = k$ $(k \neq 1)$ là đường tròn có đường kính nối hai điểm chia trong và ngoài AB theo tỉ số k.
- **Tâm ngoại tiếp O và trực tâm H**: Đặt $d_B = x_B^2 + y_B^2 - x_A^2 - y_A^2$, $d_C = x_C^2 + y_C^2 - x_A^2 - y_A^2$:

$$x_O = \frac{d_B(y_C - y_A) - d_C(y_B - y_A)}{D}$$

$$y_O = \frac{(x_B - x_A)d_C - (x_C - x_A)d_B}{D}$$

$$D = 2\left[(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)\right]$$

$$H = A + B + C - 2O$$

- **Phép toán 3D**:
  - Cross product 3D: $\vec{A} \times \vec{B} = (A_yB_z - A_zB_y, \ A_zB_x - A_xB_z, \ A_xB_y - A_yB_x)$.
  - Thể tích tứ diện: $V = \frac{1}{6}\left|((B - A) \times (C - A)) \cdot (D - A)\right|$.
  - Mặt phẳng qua P_0 có pháp tuyến $\vec{n}$: $\vec{n} \cdot (P - P_0) = 0$; Khoảng cách điểm đến mặt phẳng: $\frac{|\vec{n}\cdot(P-P_0)|}{|\vec{n}|}$.

7.12 AREA

Độ phức tạp: $\mathcal{O}(n \log n)$

Diện tích hợp các hình chữ nhật bằng sweep line và segment tree

```
// Use: auto area = rectangle_union_area({{0,0,2,3}});
struct Rectangle { int x1, y1, x2, y2; };
__int128 rectangle_union_area(const vector<Rectangle>& rects) {
  struct Event { int x,y1,y2; int delta;
    bool operator<(const Event& other) const { return x<other.x; }
  }; vector<Event>e; vector<int>ys;
  for (auto r : rects) {
    assert(r.x1 <= r.x2 && r.y1 <= r.y2);
    if (r.x1 == r.x2 || r.y1 == r.y2) continue;
    e.push_back( {r.x1,r.y1,r.y2,1}); e.push_back( {r.x2,r.y1,r.y2,-1});
    ys.push_back(r.y1); ys.push_back(r.y2);
  }
  if (e.empty()) return 0;
  sort(ys.begin(),ys.end()); ys.erase(unique(ys.begin(),ys.end()),ys.end());
  sort(e.begin(),e.end()); int n=ys.size()-1;
  vector<int> cover(4 * n); vector<__int128> len(4 * n);
  struct CoverageTree {
```

```
    vector<int>& cover; vector<__int128>& len; const vector<int>& ys;
    void update(int u,int l,int r,int a,int b,int d) {
      if (b <= l || r <= a) return;
      if (a <= l && r <= b) cover[u] += d;
      else { int m=(l+r)/2; update(u*2,l,m,a,b,d); update(u*2+1,m,r,a,b,d); }
      len[u]=cover[u]?(__int128)ys[r]-ys[l]:r-l==1?0:len[u*2]+len[u*2+1];
  }
  } helper{cover,len,ys}; __int128 ans = 0; int prev = e[0].x;
  for (auto v : e) {
    ans += len[1] * ((__int128) v.x - prev);
    helper.update(1,0,n,lower_bound(ys.begin(),ys.end(),v.y1)-ys.begin(),
     lower_bound(ys.begin(),ys.end(),v.y2)-ys.begin(),v.delta);
    prev = v.x;
  }
  return ans;
}
```

7.13 Manhattan MST

Độ phức tạp: $\mathcal{O}(n \log n)$

Manhattan MST bằng quét tám miền góc

```
// Use: auto [cost,edges] = manhattan_mst({{0,0},{2,3}});
struct ManhattanEdge { int u, v; int weight;
  bool operator<(const ManhattanEdge& other) const { return weight<other.weight;
    }
};
// Coordinates, sums/differences, distances and total weight must fit int.
vector<ManhattanEdge>manhattan_candidates(vector<pair<int,int>>p){
  auto original=p; int n=p.size(); vector<int>ids(n); iota(ids.begin(),ids.end()
    ,0);
  vector<ManhattanEdge> edges;
  for (int direction = 0; direction < 4; ++direction) {
    vector<pair<int,int>> order;
    for (int id:ids) order.push_back({p[id].first+p[id].second,id});
    sort(order.begin(),order.end());
    for (int i=0;i<n;++i) ids[i]=order[i].second;
    map<int, int> sweep;
    for (int i : ids) {
      auto[x, y] = p[i];
      for (auto it = sweep.lower_bound(-y); it != sweep.end();) {
        int j = it->second; auto[u, v] = p[j];
        if (x - u < y - v) break;
        edges.push_back( {
          i,j,llabs(original[i].first-original[j].first)+llabs(original[i].
   second-original[j].second)
        }); it = sweep.erase(it);
      }
      sweep[-y] = i;
    }
    for (auto& [x, y] : p) if (direction & 1) x = -x;
    else swap(x, y);
  }
  return edges;
}
pair<int,vector<ManhattanEdge>>manhattan_mst(const vector<pair<int,int>>&p){
  auto e = manhattan_candidates(p);
  sort(e.begin(),e.end());
  vector<int>parent(p.size()),size(p.size(),1); iota(parent.begin(),parent.end()
    ,0);
  struct DSUFind {
    vector<int>& parent;
    int find(int u) { return parent[u]==u?u:parent[u]=find(parent[u]); }
  } dsu{parent}; int total = 0; vector<ManhattanEdge> tree;
  for (auto edge : e) {
    int a = dsu.find(edge.u), b = dsu.find(edge.v);
    if (a == b) continue;
    if (size[a] < size[b]) swap(a, b);
    parent[b]=a; size[a]+=size[b]; total+=edge.weight; tree.push_back(edge);
  }
  return {total, tree};
}
```

7.14 Planar Dual Graph

Độ phức tạp: $\mathcal{O}(E \log E)$

Từ embedding phẳng liên thông với cạnh thẳng sang đồ thị đối ngẫu: tách mặt (kể cả mặt ngoài) và cạnh dual bằng sắp góc các half-edge (dart).

```
// Use: auto dual = planar_dual(points,edges);
//      points: vector<Point>; edges: vector<pair<int,int>>.
struct PlanarDual {
  vector<vector<int>>face_darts; vector<int>face_of; vector<pair<int,int>>
    dual_edges;
  int outer_face = -1;
};
// Connected straight-line embedding, no crossings/overlapping rays/self-loops.
// Bridges yield loops in the dual. Isolated single vertex has one empty face.
PlanarDual planar_dual(const vector<Point>&p,const vector<pair<int,int>>&edges){
  int n = p.size(), m = edges.size(); PlanarDual out;
  if(!m){ if(n){ out.face_darts.push_back( {}); out.outer_face=0; } return out;
    }
  vector<vector<int>>adj(n); vector<int>from(2*m),to(2*m),pos(2*m),next(2*m);
  for (int i = 0; i < m; ++i) {
    auto[u,v]=edges[i]; assert(u!=v); from[2*i]=to[2*i+1]=u; to[2*i]=from[2*i
    +1]=v;
    adj[u].push_back(2 * i); adj[v].push_back(2 * i + 1);
  }
  struct AngleOrder {
    const vector<Point>& p; const vector<int>& to; int u;
    static bool upper(Point a) { return a.y>0 || (a.y==0 && a.x>=0); }
    bool operator()(int a,int b) const {
      Point x=p[to[a]]-p[u],y=p[to[b]]-p[u];
      if (upper(x)!=upper(y)) return upper(x)>upper(y);
      return cross(x,y)>0;
    }
  };
  for (int u = 0; u < n; ++u) {
    sort(adj[u].begin(),adj[u].end(),AngleOrder{p,to,u});
    for (int j = 0; j < (int) adj[u].size(); ++j) pos[adj[u][j]] = j;
  }
  for (int d = 0; d < 2 * m; ++d) {
    int v=to[d],k=adj[v].size(); next[d]=adj[v][(pos[d^1]+k-1)%k];
  }
  out.face_of.assign(2 * m, -1);
  long double minimum = numeric_limits<long double>::infinity();
  for (int d = 0; d < 2 * m; ++d) if (out.face_of[d] < 0) {
    int id=out.face_darts.size(); vector<int>face; long double area=0; int v=d;
    do {
      out.face_of[v]=id; face.push_back(v); area+=cross(p[from[v]],p[to[v]]); v=
     next[v];
    }
    while (v != d);
    out.face_darts.push_back(move(face));
    if (area < minimum) minimum = area, out.outer_face = id;
  }
  for(int i=0;i<m;++i)out.dual_edges.push_back( {out.face_of[2*i],out.face_of[2*
    i+1]});
  return out;
}
```

8 Tiện ích và kỹ thuật

8.1 Template

```
#include<bits/stdc++.h>
#pragma GCC optimize("O3,unroll-loops")
// Enable only when the judge CPU supports these instructions.
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
bool VERBOSE = false;
// Use cerr (not std::cerr); false skips log expressions too.
#define cerr if (!VERBOSE) {} else std::cerr
#define FOR(i, a, b) for (int i = a; i <= b; i++)
#define FORD(i, a, b) for(int i = a; i >= b; i--)
#define endl '\n'
#define pb push_back
#define pf push_front
#define fastio ios_base::sync_with_stdio(NULL); cin.tie(NULL); cout.tie(NULL);

//Data structure
#define vi vector<int>
#define int long long
#define ii pair<int, int>
#define iii pair<int, ii>

//Bitmask (absolutely have a () outside of every thing)
#define LSB(x) (x & (-x))
#define ON(x, i) ((x >> i)&1)
#define OFF(x, i) !ON(x, i)
#define SET(x, i) ((x) |= (1LL << (i)))
#define UNSET(x, i) ((x) &= ~(1LL << (i)))

bool maximize(int &a,int b) { if (a<b) { a=b; return true; } return false; }
bool minimize(int &a,int b) { if (a>b) { a=b; return true; } return false; }
using namespace std;

const int MOD = 1e9 + 7;
void add(int &a, int b){
    a += b;
    if (a >= MOD) a -= MOD;
}

void sub(int &a, int b){
    a -= b;
    if (a < 0) a += MOD;
}
void solve(){

}
// Remember: return 0; at the end of main.
```

8.2 Debugging

```
#define what_is(x) cerr << #x << " = " << x << endl;
// Use: deb(distance); print one value per call.
#define deb(x) what_is(x)
```

8.3 Script biên dịch và chạy

Cần **g++** trong PATH (Windows: MinGW-w64). Truyền tên file nguồn; dừng nếu biên dịch lỗi.

```
# Linux / macOS: save as run.sh
#!/bin/sh
[ "$#" -eq 1 ] || { echo 'Usage: sh run.sh file.cpp' >&2; exit 1; }
src="$1"
exe="${src%.*}.exe"
g++ -std=c++17 -O2 -Wall "$src" -o "$exe" || exit 1
case "$exe" in /*) ;; *) exe="./$exe" ;; esac
"$exe"
# Run: sh run.sh "my solution.cpp"
# Files: sh run.sh main.cpp < input.txt > output.txt

:: Windows: save as run.bat (omit the .sh part)
@echo off
if "%~1"=="" (echo Usage: run.bat file.cpp & exit /b 1)
g++ -std=c++17 -O2 -Wall "%~1" -o "%~dpn1.exe"
if errorlevel 1 exit /b 1
"%~dpn1.exe"
:: Run in cmd: run.bat "my solution.cpp"
:: Files: run.bat main.cpp < input.txt > output.txt
```

8.4 Phân tích biểu thức (Expression Parsing)

```
#include <bits/stdc++.h>
  // Use: auto value = evaluate_expression("2*(3+4)");

// + - * / ^, parentheses, unary signs; ^ is right-associative.
// Throws invalid_argument on syntax errors, domain_error on undefined arithmetic.
struct ExpressionParser {
    std::vector<long double> values; std::vector<char> ops;
    int precedence(char op) {
        if (op == '^') return 4;
        if (op == 'n' || op == 'p') return 3;
        if (op == '*' || op == '/') return 2;
        if (op == '+' || op == '-') return 1;
        return 0;
    }
    void syntax() { throw std::invalid_argument("expression syntax"); };
    void apply() {
        char op = ops.back(); ops.pop_back();
        if (values.empty()) syntax();
        long double b = values.back(); values.pop_back();
        if (op == 'n' || op == 'p') {
            values.push_back(op == 'n' ? -b : b); return;
        }
        if (values.empty()) syntax();
        long double a = values.back(); values.pop_back();
        if (op == '/' && b == 0) throw std::domain_error("division by zero");
        long double result = op == '+' ? a + b : op == '-' ? a - b :
            op == '*' ? a * b : op == '/' ? a / b : std::pow(a, b);
```

```
        if (!std::isfinite(result)) throw std::domain_error("nonfinite result");
        values.push_back(result);
    }
    long double evaluate(const std::string& text) {
    bool operand = true;
    for (size_t i = 0; i < text.size();) {
        char c = text[i];
        if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }
        if (std::isdigit(static_cast<unsigned char>(c)) || c == '.') {
            if (!operand) syntax();
            char* end;
            long double value = std::strtold(text.c_str() + i, &end);
            if (end == text.c_str() + i || !std::isfinite(value)) syntax();
            i = static_cast<size_t>(end - text.c_str());
            values.push_back(value); operand = false; continue;
        }
        ++i;
        if (c == '(') {
            if (!operand) syntax();
            ops.push_back(c); continue;
        }
        if (c == ')') {
            if (operand) syntax();
            while (!ops.empty() && ops.back() != '(') apply();
            if (ops.empty()) syntax();
            ops.pop_back(); operand = false; continue;
        }
        if (c != '+' && c != '-' && c != '*' && c != '/' && c != '^') syntax();
        if (operand) {
            if (c != '+' && c != '-') syntax();
            ops.push_back(c == '-' ? 'n' : 'p'); continue;
        }
        while (!ops.empty() && ops.back() != '(' &&
               (precedence(ops.back()) > precedence(c) ||
                (precedence(ops.back()) == precedence(c) && c != '^'))) apply();
        ops.push_back(c); operand = true;
    }
    if (operand) syntax();
    while (!ops.empty()) {
        if (ops.back() == '(') syntax();
        apply();
    }
    if (values.size() != 1) syntax();
    return values.back();
    }
};
long double evaluate_expression(const std::string& text) {
    ExpressionParser parser; return parser.evaluate(text);
}
```

8.5 Lập lịch Johnson

Lập lịch trên hai máy.

```
#include <bits/stdc++.h>
  // Use: auto schedule = johnson_schedule({{0,2,3},{1,4,1}});

struct FlowShopJob { int id; int first, second; };
struct FlowShopSchedule { std::vector<int> order; int makespan; };

bool johnson_less(const FlowShopJob& a,const FlowShopJob& b) {
        bool early_a = a.first <= a.second, early_b = b.first <= b.second;
        if (early_a != early_b) return early_a;
        int x = early_a ? a.first : a.second;
        int y = early_b ? b.first : b.second;
        if (x != y) return early_a ? x < y : x > y;
        return a.id < b.id;

}

// Nonnegative durations; unique job IDs. O(n log n).
FlowShopSchedule johnson_schedule(std::vector<FlowShopJob> jobs) {
    std::sort(jobs.begin(),jobs.end(),johnson_less);
    FlowShopSchedule result{{}, 0};
    int first_finish = 0;
    for (const auto& job : jobs) {
        result.order.push_back(job.id);
        first_finish += job.first;
        result.makespan = std::max(result.makespan, first_finish) + job.second;
    }
    return result;
}
```

8.6 Fracturing Search (k-best)

Oracle tìm nghiệm tốt nhất trong miền; heap dùng cùng tiêu chí xếp hạng.

```
kBest(initialDomain, k):
    heap = priority_queue()  // best solution first

    push(domain):
        best = oracle(domain)
        if best exists:
            heap.push({domain, best})

    push(initialDomain)
    answers = []
    while heap not empty and answers.size < k:
        {domain, S} = heap.pop()
        answers.append(S)

        prefix = copy(domain)
        for i in unfixed decisions of domain:
            child = copy(prefix)
            child.fix(i, opposite(S[i]))
            push(child)
            prefix.fix(i, S[i])

    return answers
```

Principle: thứ tự quyết định cố định; child khác S lần đầu tại i, prefix giữ các quyết định trước đó giống S. Các miền con rời nhau và phủ phần còn lại; không gộp nghiệm theo cost. Chi phí phụ thuộc số miền và oracle.

8.7 Monte Carlo và Las Vegas

- **Monte Carlo** (*Thời gian cố định, xác suất lỗi* $\le \epsilon$*; lặp* k *lần để giảm lỗi xuống* $(1-p)^k$):
  - **Freivalds** ($AB \stackrel{?}{=} C$): Kiểm tra $P = A(Bv) - Cv$ với $v \in \{0,1\}^N$ ngẫu nhiên trong $O(N^2)$. Nếu $AB \neq C$, $\Pr(P = 0) \le 1/2$. Lặp k lần $\implies$ lỗi $\le 2^{-k}$.
  - **Identity Testing / PIT (Schwartz-Zippel)**: Đa thức khác 0 bậc d, $P(x_1..x_n)$, tại $r \in S^n$ ngẫu nhiên có $\Pr(P(r) = 0) \le d/|S|$. General matching: kiểm tra $\det(\text{Tutte Matrix}) \neq 0$ trong $O(V^3)$.
  - **Fingerprinting / Hashing**: Ánh xạ $v \mapsto r_v \in [0, 2^{64}-1]$. Hash tập bằng $\bigoplus r_v$, multiset bằng $\sum cnt(v) r_v \pmod{2^{64}}$, dãy bằng rolling hash 61-bit. Xác suất va chạm $\le 2^{-64}$.
  - **Color Coding** ($k \le 8$): Tìm đồ thị con/đường đi k đỉnh bằng tô ngẫu nhiên k màu. Xác suất mục tiêu đủ màu là $k!/k^k \ge e^{-k}$. DP trên màu mất $O(2^k(V+E))$. Lặp $O(e^k \ln(1/\epsilon)) \approx 300$ lần để lỗi $< 10^{-6}$.
- **Las Vegas** (*Luôn đúng, thời gian ngẫu nhiên, kỳ vọng* $1/p$ *lần thử*):
  - **Tìm và kiểm chứng witness**: Nếu nghiệm có mật độ $\ge p$, chọn ứng viên ngẫu nhiên rồi kiểm tra trong $O(T) \implies$ thời gian kỳ vọng $O(T/p)$. Ví dụ: chọn phần tử ngẫu nhiên để tìm majority $> n/2$ trong kỳ vọng $O(n)$ ($p > 1/2 \implies \le 2$ lần thử); chọn cặp ngẫu nhiên tìm ước chung.
  - **Cấu trúc và chọn ngẫu nhiên**: Pivot QuickSelect (kỳ vọng $O(n)$), priority Treap ngẫu nhiên (chiều cao kỳ vọng $O(\log n)$), Pollard Rho (kỳ vọng $O(n^{1/4})$).

8.8 Lịch (Calendar)

```
#include <bits/stdc++.h>
  // Use: int weekday = day_of_week(2026,10,8);

// Valid Gregorian date, year >= 1. Returns Sunday=0, ..., Saturday=6.
int day_of_week(int year, int month, int day) {
    if (month < 3) { month += 12; --year; }
    int y = year % 100, century = year / 100;
    int h = (day + 13 * (month + 1) / 5 + y + y / 4 +
             century / 4 + 5 * century) % 7;
    return (h + 6) % 7;
}
```

8.9 Josephus

Đánh số từ 0, mỗi lần loại người thứ k: $f(1) = 0$, $f(n) = (f(n-1) + k) \bmod n$. Duyệt kích thước $2, \ldots, n$: $O(n)$ thời gian, $O(1)$ bộ nhớ. Với $k = 2$: $f(n) = 2(n - 2^{\lfloor \log_2 n \rfloor})$.

8.10 Kỹ thuật interactive

```
#include <bits/stdc++.h>
  // Use: InteractiveSession judge(cin,cout,100);

// Protocol adapter: supply serialization and invalid-reply rules for the task.
struct InteractiveSession {
    using Reply = int; // Change reply type to match the protocol.
    std::istream& input;
    std::ostream& output;
    size_t limit, used = 0;
    InteractiveSession(std::istream& in, std::ostream& out, size_t budget)
        : input(in), output(out), limit(budget) {}
    size_t queries() const { return used; }
    Reply ask(std::function<void(std::ostream&)> write,
              std::function<bool(Reply)> valid) {
        if (used == limit) throw std::runtime_error("query budget exhausted");
        write(output); output << '\n' << std::flush; ++used;
        Reply reply;
        if (!(input >> reply) || !valid(reply))
            throw std::runtime_error("invalid judge reply");
        return reply;
    }
    void answer(std::function<void(std::ostream&)> write) { write(output); output
      << '\n' << std::flush; }
};

// Monotone predicate: test(lo)=false, test(hi)=true; known endpoints not queried.
int first_true(int lo,int hi,std::function<bool(int)> test) {
    assert(lo < hi);
    while (__int128(hi) - lo > 1) {
        int mid = static_cast<int>(__int128(lo) + (__int128(hi) - lo) / 2);
        if (test(mid)) hi = mid; else lo = mid;
    }
    return hi;
}

// Adjacent values differ by 1; read(lo)<=target<=read(hi), lo<hi.
int find_unit_step_value(int lo,int hi,int target,
                         std::function<int(int)> read) {
    while (__int128(hi) - lo > 1) {
        int mid = static_cast<int>(__int128(lo) + (__int128(hi) - lo) / 2);
        auto value = read(mid);
        if (value == target) return mid;
        if (value < target) lo = mid; else hi = mid;
    }
    return read(lo) == target ? lo : hi;
}

// Distinct values, n>=2; better(i,j) returns true iff value[i]>value[j].
// Returns {maximum index, second-maximum index}; balanced tournament.
std::pair<int,int> top_two(int n,std::function<bool(int,int)> better) {
    assert(n >= 2);
    std::vector<std::vector<int>> defeated(n);
    std::vector<int> round(n);
    std::iota(round.begin(), round.end(), 0);
    while (round.size() > 1) {
        std::vector<int> next;
        for (size_t i = 0; i < round.size(); i += 2) {
            int winner = round[i];
```

```
            if (i + 1 < round.size()) {
                int loser = round[i + 1];
                if (!better(winner, loser)) std::swap(winner, loser);
                defeated[winner].push_back(loser);
            }
            next.push_back(winner);
        }
        round.swap(next);
    }
    int best = round[0], second = defeated[best][0];
    for (size_t i = 1; i < defeated[best].size(); ++i)
        if (better(defeated[best][i], second)) second = defeated[best][i];
    return {best, second};
}
```

- **Kiểm tra ba phía**: So sánh hai nhóm A, B cùng kích thước; khi có một mục tiêu và baseline bằng nhau đã biết, trả lời xác định mục tiêu ở A, B hoặc nhóm còn lại C.
- **Gộp truy vấn**: Hỏi $\gcd(X, \prod p_i^{e_i})$ để khôi phục số mũ các nguyên tố đã chọn; tránh tràn tích. Suy ra nhóm cuối khi biết tổng.
- **Chuẩn hóa**: Với $1 \le x \le 10^d$, tổng chữ số $S(x(10^d-1)) = 9d$; đưa trạng thái về $9d$, rồi cộng mục tiêu$-9d$.
- **Top hai theo chi phí**: Xáo trộn phần tử còn lại; so với phần tử nhì trước phần tử nhất để ít truy vấn cực đại thật.
- **Đỉnh cực đại/lan truyền**: Cache giá trị đã hỏi, giữ witness cho đỉnh còn khả thi. Dò mìn: số mìn còn lại $= 0$ thì các ô chưa biết an toàn; bằng số ô chưa biết thì tất cả là mìn. Tiếp tục trừ các phương trình chồng nhau.

8.11 Giao tiếp hai lần chạy (Run-Twice)

Nhận diện: Pha 1 gửi thông điệp giới hạn; Pha 2 dùng nó cùng dữ liệu riêng để suy ra đáp án. Phân biệt S khả năng cần ít nhất $\lceil \log_2 S \rceil$ bit. Kiểm tra kênh có giữ thứ tự, mất dữ liệu hay đổi nhãn không.

- **Đóng gói**: $0 \le x_i < b_i$, gửi $X = \sum_i x_i \prod_{j<i} b_j$; giải mã lần lượt $x_i = X \bmod b_i$, $X \leftarrow \lfloor X/b_i \rfloor$. Ví dụ $(2,1,3)$, cơ số $(3,2,4)$ cho $X = 23$. Cần $\prod b_i - 1$ vừa miền truyền; `long long` chứa tối đa 39 chữ số hệ 3.
- **Mất thứ tự**: Gửi $v_i = iB + x_i$ với $0 \le x_i < B$, $0 \le i < k$; lấy lại $i = \lfloor v_i/B \rfloor$, $x_i = v_i \bmod B$. Ví dụ $7, 12, 24$ với $B = 10$ mã hóa $(7,2,4)$. Cần miền truyền chứa $kB - 1$.
- **Mất số, checksum nguyên vẹn**: Mất một số: XOR toàn bộ trước khi gửi, XOR tiếp phần còn lại để khôi phục. Mất hai số: gửi $A = \sum x_i$, $B = \sum x_i^2$; trừ phần còn lại được $s = u + v$, $q = u^2 + v^2$. Lấy $d = \sqrt{2q - s^2} = |u - v|$, suy ra $(s \pm d)/2$, chưa biết thứ tự. Dùng số nguyên chính xác, `__int128` nếu cần; không thay tổng bằng tổng modulo.
- **Hoán vị**: k phần tử phân biệt truyền $k!$ trạng thái, $0 \le t < k!$. Hai pha biết cùng tập phần tử, sort làm chuẩn. Còn m phần tử: chọn hạng $\lfloor t/(m-1)! \rfloor$ từ 0, xóa nó, cập nhật $t \leftarrow t \bmod (m-1)!$. Giải mã cộng $c_m(m-1)!$, c_m là số phần tử còn lại nhỏ hơn phần tử đang đọc. Ví dụ $a < b < c$: bca truyền $1 \cdot 2! + 1 \cdot 1! = 3$.
- **Lật một ô báo đích**: Bàn 2^r bit, chỉ số $0, \ldots, 2^r - 1$. Đặt $P = \bigoplus_{b_i=1} i$, lật ô $P \oplus T$; Pha 2 XOR chỉ số bit 1 được T. Ví dụ $P = 5, T = 3$: lật ô 6. Nếu $P = T$, lật ô 0. Cần được tự chọn ô lật và giữ nguyên chỉ số ở hai pha.

8.12 Random, Bit Tricks, Bitset

```
#include <bits/stdc++.h>
  // Use: for_each_submask(13,onMask); for_each_k_subset(5,2,onMask);
  //      void onMask(int mask)

// Nonnegative masks. Calls visit(submask), including zero, in descending order.
void for_each_submask(int mask,std::function<void(int)> visit) {
    assert(mask >= 0);
    int sub = mask;
    for (;;) {
        visit(sub);
        if (sub == 0) break;
        sub = (sub - 1) & mask;
    }
}

// Gosper: n<=63 with #define int long long; k<=n.
void for_each_k_subset(int n,int k,std::function<void(int)> visit) {
    const int BITS = std::numeric_limits<int>::digits;
    assert(0 <= k && k <= n && n <= BITS);
    if (k == 0) { visit(0); return; }
    __int128 limit = (__int128)1 << n;
    int mask = ((__int128)1 << k) - 1;
    for (;;) {
        visit(mask);
        int low = mask & -mask;
        __int128 next = (__int128)mask + low;
        if (next >= limit) break;
        mask = next | (((next ^ mask) >> 2) / low);
    }
}

// Count bits in the full int storage width; zero has no set bit.
int trailing_zeros(int x) {
    return x ? __builtin_ctzll(x) : sizeof(int) * CHAR_BIT;
}
int leading_zeros(int x) {
    return x ? __builtin_clzll(x) - (sizeof(long long)-sizeof(int))*CHAR_BIT
             : sizeof(int) * CHAR_BIT;
}
int popcount(int x) { return __builtin_popcountll(x); }
int lowest_bit(int x) { assert(x >= 0); return x & -x; }
// Permutation/matching DP: next position = popcount(mask), no extra step state.
```

8.13 Biến đổi bài toán (Transformations)

Khi tìm hướng tiếp cận

- Xuất phát từ các đại lượng trong đề.
- Thay đổi mức độ cụ thể của suy nghĩ hiện tại (từ tổng quát đến chi tiết, từ trừu tượng đến cụ thể).
- **Xuất phát từ đích, suy luận ngược**: Muốn tính/chứng minh kết quả thì cần biết gì trước? Truy ngược đến khi nối được với dữ kiện trong đề.
- **Xuất phát từ vét cạn**: Viết lời giải hiển nhiên, tìm tính toán lặp lại hoặc dư thừa để tối ưu.
- **Thay đổi cách biểu diễn**: Chuyển sang đồ thị, hình học, đại số hoặc xét phần bù để lộ cấu trúc dễ xử lý hơn.
- **Khai thác ràng buộc đặc biệt**: Mỗi giả thiết có tác dụng gì, vì sao đề cần nó, bỏ đi thì điều gì thay đổi?
- **Liên hệ bài toán đã biết**: Tìm cấu trúc quen thuộc và xác định điểm khác biệt cần xử lý.
- **Khi bí**: Đọc lại đề! Xét trường hợp nhỏ ($N \le 6$) để tìm quy luật/bất biến; suy ra độ phức tạp ($N \le 20 \to 2^N, 500 \to N^3, 5000 \to N^2, 2 \cdot 10^5 \to N \log N$).
- **Tối ưu → quyết định (Binary Search)**: Nếu điều kiện $P(X)$ đơn điệu, binary search X qua kiểm tra khả thi.
- **Hoán vị toàn cục → đổi chỗ cục bộ**: Lập luận trao đổi greedy: đổi $(i,j) \to (j,i)$ cải thiện mục tiêu $\iff f(i,j) < f(j,i)$.
- **Ràng buộc → Graph / Flow / Cut**: 2-SAT, difference constraints ($x_j - x_i \le w$), Project Selection (min-cut), Hall, Dilworth (DAG path cover $= |V| -$ MaxMatching).
- **Đại số → hình học/tính lồi**: Đường bao trên/dưới, Convex Hull, Slope Trick, Minkowski sum, CHT.
- **Trung bình có trọng số → Convex Hull**: Xem λ_i là tỉ lệ trộn ($\lambda_i \ge 0$, $\sum_i \lambda_i = 1$). Khi đó $x = \sum_i \lambda_i x_i$ nằm giữa các đầu vào: hai điểm cho đoạn AB, ba điểm cho miền tam giác, nhiều điểm cho convex hull. Mọi điểm trong convex hull đều biểu diễn được như vậy. Trong 1D: $x \in [\min x_i, \max x_i]$. Chuẩn hóa trọng số dương: $\lambda_i = w_i / \sum_j w_j$.
- **Tỉ số tổng có trọng số → trung bình tỉ số**: Với $w_i \ge 0$, $b_i > 0$, $\frac{\sum_i w_i a_i}{\sum_i w_i b_i}$ là trung bình có trọng số của a_i/b_i, trọng số mới $w_i b_i$. Vì vậy nằm trong $[\min_i(a_i/b_i), \max_i(a_i/b_i)]$. Không trọng số: $w_i = 1$.
- **Trộn trước → Jensen**: Với f lồi (hình cái bát), tính sau khi trộn rẻ hơn: $f(\sum_i \lambda_i x_i) \le \sum_i \lambda_i f(x_i)$. Đọc là f(trung bình) $\le$ trung bình của f.
- **Đoạn toàn cục → Divide & Conquer**: Chia $[l,r] \to [l,m]$ và $[m+1,r]$, cộng đóng góp chéo (CDQ D&C, D&C DP).
- **Xóa offline → thêm (đảo thời gian)**: Đảo thứ tự truy vấn $Q \to 1$ để đổi xóa thành thêm DSU. Game DP: đi ngược từ trạng thái kết thúc.
- **Chia miền tìm kiếm (Meet-in-Middle)**: Chia $N = N/2 + N/2$, giảm $2^N \to 2^{N/2}$ bằng hash table/two pointers.
- **Đúng k nhóm → hệ số phạt Lagrange**: Tối ưu $cost + \lambda m$, m là số nhóm. Binary search λ khi m đơn điệu; cần tính lồi và xử lý hòa để khôi phục đúng k. Bài *đếm* đúng k tập dùng Binomial Inversion/PIE từ số cách thuộc ít nhất k tập.
- **Tổng toàn cục → đóng góp phần tử**: Đổi góc nhìn: cộng đóng góp mỗi phần tử/cạnh/cặp (u,v) trên mọi trạng thái hợp lệ (Fubini).
- **Chia theo ngưỡng ($\sqrt{N}$ Heavy/Light)**: Nhóm $\le B$: duyệt trực tiếp; $> B$: tối đa N/B nhóm, cache riêng. Cân bằng chi phí theo B; khác kỹ thuật Mo.
- **Nâng cấp cuối → chặn trạng thái**: Bỏ nâng cấp cuối, lập lại kế hoạch còn lại; nếu chi phí tăng $\le$ giá nâng cấp thì bỏ được. Với $L \le N$, $k > \lceil\sqrt{N}\rceil$: $\lceil L/(k-1)\rceil - \lceil L/k\rceil \le 1$. Giá nâng cấp ≥ 1, mỗi điểm dừng giá $1 \implies$ chặn tài nguyên ở $\lceil\sqrt{N}\rceil$.
- **Gom nhóm thương (Quotient Grouping)**: $\lfloor N/k \rfloor$ có $O(\sqrt{N})$ giá trị khác nhau. Với $1 \le l \le N$, nhóm cùng thương kết thúc ở $r = \lfloor N/\lfloor N/l \rfloor \rfloor$; tiếp theo $l = r + 1$.
- **Đếm chính xác → bù/PIE**: Đếm trạng thái hợp lệ qua tổng − không hợp lệ, Stars & Bars, PIE.
- **Impartial Game → Nim Sum**: Chuyển game trên DAG thành XOR các giá trị Sprague-Grundy ($SG(u) = \text{mex}\{SG(v)\}$).
- **Undirected Vertex Geography → Matching**: Quân bắt đầu tại s; hai người luân phiên đi tới hàng xóm chưa thăm, xóa đỉnh vừa rời. Người đầu thắng iff s thuộc mọi maximum matching của G, tương đương $\nu(G) > \nu(G - s)$.
- **Tổng quát → trường hợp đặc biệt**: Giải giới hạn dễ trước ($A_i \in \{0,1\}$, cây, $N \le 6$) để tìm quy luật rồi tổng quát hóa.
- **Vét cạn → Branch & Bound**: Cắt nhánh DFS bằng cận hợp lệ.
- **Manhattan ↔ Chebyshev (xoay 45°)**: $(x,y) \mapsto (x+y, x-y) \implies |x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$. Tách hai chiều thành hai trục 1D độc lập.
- **Phân rã max/min và trị tuyệt đối**: $\max(A,B) = \frac{A+B+|A-B|}{2}$, $\min(A,B) = \frac{A+B-|A-B|}{2}$; $|x-a| = \max(x-a, a-x)$. Với một khoảng cách, xét bốn cặp dấu ($\pm x \pm y$). Để tối thiểu $\sum_i(|x-a_i| + |y-b_i|)$, tách hai tổng 1D rồi chọn median của a_i và b_i.

- **Đếm theo ngưỡng/lớp (Fubini)**: $\sum_{i=1}^{n} A_i = \sum_{v=1}^{\max A} \text{count}(A_i \geq v)$ (cộng giá trị $\longleftrightarrow$ đếm phần tử $\geq v$).
- **Bình phương → tổng số lẻ**: $x^2 = 1 + 3 + 5 + \cdots + (2x-1) = \sum_{i=1}^{x}(2i-1)$.
- **Dãy chữ số 1 → tổng cấp số nhân**: $\underbrace{11\ldots1}_{i \text{ chữ số}} = 1+10+\cdots+10^{i-1} = \frac{10^i-1}{9}$.
- **Dãy tăng/giảm → biến không âm**: Bỏ ràng buộc thứ tự bằng chênh lệch kề. Nếu $A_1 \leq \cdots \leq A_n$, đặt $D_i = A_i - A_{i-1} \geq 0$; nếu $A_1 \geq \cdots \geq A_n$, đặt $D_i = A_{i-1} - A_i \geq 0$ ($i \geq 2$). Thứ tự nghiêm ngặt cần $D_i \geq 1$. Khôi phục $A_i = A_1 + \sum_{j=2}^{i} D_j$ (tăng), hoặc $A_i = A_1 - \sum_{j=2}^{i} D_j$ (giảm).
- **Range Add → mảng hiệu/Prefix Sum**: $A_i = \sum_{j=1}^{i} D_j$ ($D_i = A_i - A_{i-1}$); cộng đoạn $[l,r]$ → hai cập nhật điểm trên D. Tổng tĩnh → $S_r - S_{l-1}$.
- **Cửa sổ cố định → lớp modulo**: XOR hai phép flip cửa sổ dài x bắt đầu tại $i, i+1$: phần giao triệt tiêu, chỉ đảo $i, i+x$ → cùng lớp modulo x. Mỗi cửa sổ chứa đúng một vị trí mỗi lớp; đặt $P_r = \bigoplus_{i \equiv r \pmod x} s_i$. Flip đảo mọi P_r, nên $P_r \oplus P_0$ ($1 \leq r < x$) bất biến và đủ phân loại trạng thái; từ toàn 0 đạt được s iff mọi P_r bằng nhau. Cue: triệt tiêu hai cửa sổ kề → hai đầu cách x → modulo. Cần phép triệt tiêu hợp lệ (XOR, hoặc cộng với phép nghịch đảo); không suy rộng sang gán/max.
- **Thu nhỏ điều kiện: toàn cục → cục bộ**: Tìm điều kiện trên các phần tử kề/cửa sổ nhỏ tương đương điều kiện cả dãy. Sau sắp xếp, $\min_{i \neq j} |A_i - A_j| = \min_i (A_{i+1} - A_i)$; $A_j - A_i \geq (j-i)D \iff A'_{i+1} \geq A'_i$ ($A'_i = A_i - iD$). Cấp số cộng $\iff$ mọi bộ ba liên tiếp thỏa $A_i + A_{i+2} = 2A_{i+1}$; áp dụng ngang/dọc biến hình chữ nhật cấp số cộng thành vùng cửa sổ 3×3 hợp lệ (LFAC2 2011A). Đoạn dài ≥ 2 có majority ($> 50\%$) luôn chứa witness dài 2 (x,x) hoặc 3 (x,y,x).
- **Tích → tổng**: $ab = \binom{a+b}{2} - \binom{a}{2} - \binom{b}{2}$; $\prod A_i \to \sum \log A_i$ (hoặc log xác suất).
- **Đẳng thức bitwise**: $A+B = (A \oplus B) + 2(A \text{ AND } B) = (A \text{ OR } B) + (A \text{ AND } B)$; $A \oplus B = (A \text{ OR } B) - (A \text{ AND } B)$. Lật bit hai số kề: $\sum_{i=1}^{n} \text{popcount}((i-1) \oplus i) = 2n - \text{popcount}(n)$.
- **Kỹ thuật ngẫu nhiên (Monte Carlo / Las Vegas)**: Xem §8.7. Nghiệm mật độ p: Monte Carlo lặp k lần, lỗi $\leq (1-p)^k$; Las Vegas kiểm chứng trong kỳ vọng $O(T/p)$. Fingerprinting $(\bigoplus r_v, \sum r_v)$ kiểm tra tập/multiset; Color Coding tách đồ thị con; Schwartz-Zippel kiểm tra tồn tại đại số (Tutte).
- **Dirichlet/Pigeonhole (4-Sum $a_x + a_y = a_z + a_w$)**: Các giá trị nguyên dương phân biệt: Có tối đa $2 \max A$ giá trị tổng. Nếu $\binom{N}{2} > 2 \max A$, pigeonhole bảo đảm hai cặp trùng tổng. Hash tổng → cặp chỉ số trong $O(N^2)$; tính phân biệt của các giá trị bảo đảm bốn chỉ số khác nhau.
- **Boyer-Moore Majority Vote**: Tìm phần tử majority ($> n/2$ lần) trong $O(n)$ thời gian, $O(1)$ bộ nhớ bằng một bộ đếm.
- **Không bao nhau → Antichain (Sperner)**: Chọn các tập con/bitmask sao cho không tập nào chứa tập khác: tối đa $\binom{n}{\lfloor n/2 \rfloor}$. **Construction**: lấy tất cả tập có đúng $\lfloor n/2 \rfloor$ phần tử (cùng popcount nên không bao nhau). Đạt cận khi được chọn từ toàn bộ tập con; có ràng buộc thêm phải kiểm tra lại. Ví dụ: N là tích n prime khác nhau, chọn các ước không chia nhau → lấy tích của đúng $\lfloor n/2 \rfloor$ prime.

Invariant: nhận diện và kiểm tra

- **Cách tìm**: Đổi biểu diễn để thao tác thành swap/triệt tiêu. Khác bất biến → không thể; giống nhau cần chứng minh đủ bằng construction/dạng chuẩn.
- **Tổng / tô màu / modulo**: Chuyển lượng → $\sum a_i$; đối xứng → tổng xen kẽ/theo màu. Tìm w sao cho $w \cdot \Delta a = 0$ (hoặc $0 \pmod m$) với mọi thao tác. XOR cùng lượng hai lần triệt tiêu.
- **Hiệu kề / prefix → swap**: $a'_i = a_{i-1} + a_{i+1} - a_i$ swap hai hiệu kề. XOR a_i vào hai hàng xóm swap hai prefix XOR P_{i-1}, P_i. Mọi swap kề được phép → so multiset; giữ biên a_1 (hiệu), P_n (prefix XOR).
- **Hoán đổi → nhóm vị trí / cặp đối xứng**: Swap từng cặp vị trí: cạnh là swap được phép, độc lập trạng thái → so multiset mỗi component. Swap cách k → modulo k; nhiều bước nhảy thử gcd, kiểm tra biên. Swap nguyên khối prefix/suffix cùng độ dài l, $2l \leq n$, giữ thứ tự trong khối → các cặp đối xứng (a_i, a_{n+1-i}) chỉ đổi chỗ/đảo hướng: so multiset các cặp $(\min, \max)$, giữ phần tử giữa nếu n lẻ. Điều kiện đủ khi mọi $1 \leq l \leq \lfloor n/2 \rfloor$ được phép (đảo thứ tự l cặp đầu và đảo hướng từng cặp); chỉ cho vài l thì chưa chắc đủ.
- **Parity**: Tìm lượng có tính chẵn/lẻ không đổi: tổng, số bit 1, số vật theo màu, số nghịch thế. Kiểm tra mỗi thao tác làm lượng đó đổi một số chẵn. Với hoán vị phân biệt: parity $= (n - \#\text{cycles}) \bmod 2$; k-cycle tương đương $k-1$ swap, nên k lẻ giữ parity. Mọi 3-cycle sinh đúng hoán vị chẵn; phần tử trùng cho phép đổi parity gán nhãn đích.
- **Cùng lớp dư → GCD các hiệu**: Đặt $d_i = a_{i+1} - a_i$, $g = \gcd_{i<n} |d_i|$. Khi đó $g = \gcd_{i>1} |a_i - a_1| = \gcd_{i<j} |a_i - a_j|$ (hiệu cặp là tổng hiệu kề). Với $g > 0$, mọi a_i nằm trên lưới $a_1 + g\mathbb{Z}$; g là modulo lớn nhất để tất cả cùng dư. Tịnh tiến cả mảng hoặc đổi thứ tự không đổi g. Cue: khoảng cách giữa giá trị/cùng lớp dư → xét gcd hiệu, không phải gcd giá trị. Thêm $2u - v$ giữ lưới vì $u, v \equiv a_1 \pmod g \implies 2u - v \equiv a_1 \pmod g$ (CF1477A). $g = 0$: mọi giá trị bằng nhau, xét riêng.

Monovariant: mỗi bước tiến gần hơn đến đích

- **Cách dùng**: Chọn một số đo Φ (số lỗi, khoảng cách, độ lệch); chứng minh sau mỗi bước nó giảm. Nếu Φ là số nguyên không âm và giảm ít nhất 1, quá trình dừng sau tối đa Φ ban đầu bước. Dừng chưa chắc đạt đích: cần chứng minh chưa đạt đích thì vẫn chọn được bước làm giảm Φ.
- **Sửa thứ tự → số nghịch thế**: Swap hai phần tử kề $a_i > a_{i+1}$ làm số cặp sai thứ tự giảm đúng 1. Số vị trí sai không chắc giảm khi swap.
- **Di chuyển → khoảng cách/vị trí**: Mỗi bước đưa vật gần đích hơn → tổng khoảng cách giảm. Vật chỉ dịch trái → tổng vị trí của các vật giảm, dù số vật giữ nguyên.
- **San bằng → tổng bình phương**: Chuyển 1 từ số lớn u sang số nhỏ v, $u \geq v+2$: $(u-1)^2 + (v+1)^2 - u^2 - v^2 = -2(u-v-1) < 0$. Tổng không đổi, nhưng độ lệch giảm.
- **Cố định dần → số chỗ còn phải sửa**: Mỗi bước kéo dài prefix đúng và không phá nó → số vị trí ngoài prefix giảm. Nếu số đo chính đôi khi không đổi, thêm số đo phụ: bước nào cũng giảm số chính, hoặc giữ số chính và giảm số phụ (cả hai nguyên không âm).
- **Kiểm tra**: Chỉ "không tăng" có thể lặp; số thực giảm như $1, 1/2, 1/4, \ldots$ cũng có thể vô hạn. Nếu chỉ các bước mình chọn làm Φ giảm, kết luận chỉ áp dụng cho thuật toán đó.
- **Chia miền → giảm một nửa số mũ (LFAC2 2014H, 4 sao)**: Tem RGB ngắn nhất phủ chuỗi có k wildcard; chồng tại wildcard được khác màu. $L \geq n/2$: chỉ cần prefix/suffix tương thích. $L \leq n/2$: hai đầu dài L rời nhau, chọn đầu ít wildcard hơn, tối đa $\lfloor k/2 \rfloor$; liệt kê $3^{k/2}$ thay vì 3^k.
- **Bộ ba Pythagoras**: Nguyên thủy: $(a,b,c) = (u^2 - v^2, 2uv, u^2 + v^2)$, $u > v \geq 1$, $\gcd(u,v) = 1$, khác parity. Sinh mọi bộ bằng lấy bội $k(a,b,c)$; nếu $c \leq C$, duyệt $u^2 + v^2 \leq C$, $k \leq C/c$.
- **Factorial: nhánh nhỏ và nhánh khó (CRRC K)**: Tìm $a!/(b!c!) = N$, chọn $b \geq c$, đặt $d = a - b$. Nếu $d \geq c$, $N \geq \binom{2c}{c}$ chặn c; nhánh $d < c$ phải khai thác chia hết theo prime valuations. Với $N \leq 10^9$, editorial dùng $d \leq 20$; cutoff thực nghiệm, chưa chứng minh ở đây.

8.14 Phím tắt VS Code (Linux)

Mặc định; dấu phẩy: bấm hai tổ hợp lần lượt. `Ctrl+K, Ctrl+S`: tra/gán phím nếu keymap khác hoặc hệ điều hành chặn.

- **Nhiều con trỏ**: `Ctrl+D` chọn lần xuất hiện kế; `Ctrl+K, Ctrl+D` bỏ qua lần vừa chọn; `Ctrl+U` hoàn tác con trỏ; `Ctrl+Shift+L` chọn mọi lần xuất hiện của vùng chọn; `Esc` thoát.
- **Đổi tên**: `F2` đổi tên symbol theo scope (cần hỗ trợ ngôn ngữ); `Ctrl+H` thay chuỗi trong file.
- **Nhiều dòng**: `Alt+Click` thêm con trỏ; `Shift+Alt+I` thêm con trỏ cuối các dòng được chọn (ví dụ thêm dấu phẩy hàng loạt).
- **Dòng**: `Alt+↑ / ↓` chuyển dòng/khối; `Ctrl+Shift+K` xóa dòng. Không chọn gì: `Ctrl+C/X` copy/cắt dòng.
- **Chèn / format**: `Ctrl+Enter` dòng mới dưới; `Ctrl+Shift+Enter` dòng mới trên; `Ctrl+/` comment dòng; `Ctrl+Shift+I` format file (cần formatter).
- **Nhảy**: `Ctrl+G` tới dòng; `Ctrl+Shift+O` tìm symbol; `F12` tới định nghĩa; `Ctrl+Shift+\` tới ngoặc khớp; `F8` lỗi/cảnh báo kế.
- **File / lệnh**: `Ctrl+P` tìm file; `Ctrl+Tab` đổi file; `Ctrl+Shift+T` mở lại tab đóng; `Ctrl+Shift+P` tìm lệnh.
- **Terminal**: `Ctrl+`` hiện/ẩn; `Ctrl+Shift+`` tạo mới; `Ctrl+Shift+C/V` copy/paste trong terminal.

Nguồn: VS Code, Keyboard shortcuts for Linux. Alt+Click bị chặn: đổi Multi Cursor Modifier.

8.15 Checklist trước khi nộp

- **Đã đọc giới hạn và edge case ($N = 0, 1$, âm, mất liên thông)?**
- Đã reset/xóa trạng thái toàn cục giữa các test case?
- Đúng giới hạn mảng, chỉ số từ 0 hay 1?
- Có tràn số nguyên (dùng `long long`/`__int128`)?
- Hàm thiếu `return` (Undefined Behavior)?
- Độ chính xác số thực/so sánh epsilon (`eps = 1e-9`)?
- Có fast I/O (`cin.tie(NULL)`, `'\n'` thay `endl`)?